
Software Requirements Specification

for

Designing a Privacy-Preserving, Offline Machine Learning System for Phishing Detection Using URL Features

Version 2.6 approved

Prepared by Connie Rodriguez

Saint Martin's University

March 25, 2026

Executive Summary

This Software Requirements Specification (SRS) document defines the design and functional scope of a machine learning-based phishing detection system that classifies URLs as either malicious or legitimate using URL-string-derived features exclusively. The system is designed as a standalone, offline pipeline, with no reliance on external services, live network queries, or third-party reputation systems during execution.

The system supports the study of phishing detection using URL-derived features under strict operational constraints, including air-gapped deployment, privacy preservation, and independence from external data sources. While existing phishing detection systems often achieve strong performance, many depend on supplementary inputs such as Domain Name System (DNS) queries, WHOIS data, webpage content analysis, or cloud-based threat intelligence feeds. These dependencies introduce limitations related to data exposure, reproducibility, and deployability in restricted environments.

To investigate this problem, the system extracts a defined set of lexical features from URLs and applies multiple supervised machine learning classifiers. The classification outputs are aggregated using a majority voting ensemble mechanism, enabling comparative analysis of individual model performance and overall system resilience. The pipeline supports end-to-end experimentation, including dataset construction, preprocessing, feature extraction, model training, and cross-dataset evaluation.

The primary contribution of this work is the development of a constrained experimental framework for the systematic evaluation of phishing detection performance in a fully offline, privacy-preserving, and lightweight environment. Rather than introducing a novel algorithm, this project focuses on analyzing the effectiveness and limitations of URL-only detection when external data sources are intentionally excluded.

This SRS document outlines the system's functional requirements, data structures, processing workflow, and operational constraints. The implementation is intended as a research-oriented prototype designed to support empirical evaluation, reproducibility, and controlled experimentation. The results of this system aim to contribute to cybersecurity research by examining the trade-offs between detection accuracy, generalization, privacy preservation, and system independence.

Table of Contents

1. Introduction	6
1.1 Purpose	6
1.2 Scope	7
1.3 Overview	8
2. Overall Description	9
2.1 Product Perspective	9
2.2 Product Functions	10
2.3 User Classes & Characteristics	12
2.4 Operating Environment	24
2.5 Design & Implementation Constraints	25
2.6 Assumptions & Dependencies	26
3. Software Architecture	28
3.1 Architectural Overview	28
3.2 Systems Architecture Diagram	32
3.3 Architectural Components	33
3.4 Integration Layer	35
3.5 Technology Stack	36
3.6 Data Architecture	37
3.7 Security Architecture	39
3.8 Integration Architecture	40
3.9 Scalability & Performance Architecture	41
4. Software Features & Requirements	42
4.1 Functional Requirements	42
4.2 External Interface Requirements	48
4.3 Non-Functional Requirements	50
4.4 Security Requirements	53
4.5 Software Quality Attributes	55
5. Software Models	58

5.1 Use Case Diagrams	58
5.2 Activity Diagrams	59
5.3 Sequence Diagrams	60
5.4 Class Diagrams	61
5.5 State Diagrams	62
5.6 Entity-Relationship Diagrams	63
5.7 Wireframe/Prototypes	64
6. Data Requirements	65
6.1 Logical Data Model	65
6.2 Data Dictionary	66
6.3 Data Retention	69
6.4 Data Migration	70
6.5 Backup & Recovery	70
7. Other Software Requirements	70
7.1 Internationalization & Localization	71
7.2 Installation & Deployment	71
7.3 Licensing Requirements	72
7.4 Legal & Regulatory Requirements	72
7.5 Documentation Requirements	73
8. Appendices	74
Appendix A: Detailed Architecture Diagrams	74
Appendix B: Technical Specifications and Algorithmic Details	83
Appendix C: User Interface Prototype	89
Appendix D: Sample Inputs/Outputs, Test Scenarios	94
Appendix E: Definitions, Acronyms, and Abbreviations	112
Appendix F: References	114
Appendix G: AI Use	116

Revision History

Name	Date	Reason For Changes	Version
Connie Rodriguez	3/09/26	Created Layout with Table of Contents; Updated: Section 1	1.1
Connie Rodriguez	3/11/26	Updated: Section 2.1- 2.2; Section 2.4 – 2.5	1.2
Connie Rodriguez	3/12/26	Updated: Section 2.3; Section 3.1 – 3.2	1.3
Connie Rodriguez	3/13/26	Updated: Section 3.3 – 3.9	1.4
Connie Rodriguez	3/14/26	Updated: Section 4	1.5
Connie Rodriguez	3/15/26	Updated: Section 4	1.6
Connie Rodriguez	3/16/26	Updated: Section 6	1.7
Connie Rodriguez	3/17/26	Updated: Section 6	1.8
Connie Rodriguez	3/18/26	Updated: Section 7	1.9
Connie Rodriguez	3/19/26	Updated: Section 8	2.0
Connie Rodriguez	3/20/26	Updated: Section 5	2.1
Connie Rodriguez	3/21/26	Updated Section 5	2.2
Connie Rodriguez	3/22/26	Finalized: Sections 1-3	2.3
Connie Rodriguez	3/23/26	Finalized: Sections 4-6	2.4
Connie Rodriguez	3/24/26	Finalized: Section 7	2.5
Connie Rodriguez	3/25/26	Finalized: Section 8	2.6

1.Introduction

1.1 Purpose

The purpose of this Software Requirements Specifications (SRS) document is to define the functional and non-functional requirements for the Privacy-Preserving, Offline Machine Learning System for Phishing Detection using URL Features. The system is implemented as an offline, eight-script machine learning prototype that classifies URLs as malicious or legitimate, using 24 features derived from the URL string. The system supports both command-line interaction and a lightweight graphical user interface (GUI) for URL classification during inference. This document establishes the formal requirements governing the system's behavior, interfaces, constraints, data handling, architectural structure, and quality expectations.

This SRS serves as the primary reference for the design, implementation, testing, and validation of the current prototype. It provides a structured and traceable description of what the system is required to do, the conditions under which it operates, the assumptions on which it depends, and the criteria used to assess whether it satisfies its intended design objectives. In this role, the document supports requirements clarity, implementation consistency, reproducibility, and alignment between the implemented pipeline and the stated project goals.

The document is written for the current prototype rather than for a fully deployed commercial product. Accordingly, the mandatory requirements described apply to the present offline research and development implementation. Where appropriate, the document also identifies future deployment considerations, such as possible API exposure or end-user interfaces, in order to demonstrate how the current architecture could be extended without altering the core classification workflow. These forward-looking considerations are included for architectural context only and do not represent implemented capabilities of this prototype system unless explicitly stated.

The intended audience for this document includes academic evaluators, future student researchers or replicators, developers maintaining or extending the codebase, and technical reviewers assessing the methodological soundness of the system. Academic evaluators use the document to assess project rigor, interpretability, and requirements traceability. Future students or researchers may use it to reproduce, validate, and extend the pipeline in new environments. Developers and maintainers may use it as a requirements baseline for implementation, modification, testing, and documentation. Collectively, these audiences rely on the SRS as a formal description of the system's required behavior and design boundaries.

1.2 Scope

The Privacy-Preserving, Offline Machine Learning System for Phishing Detection Using URL Features is a standalone, offline machine learning pipeline that classifies URLs as malicious or legitimate using features derived exclusively from the URL string. The system operates without live network access, Domain Name System (DNS) resolution, or external API calls during both training or inference. This design supports reproducibility, privacy preservation, and operation in isolated or resource-constrained environments.

The system encompasses the end-to-end processing pipeline required for dataset construction, feature engineering, model training, evaluation, and inference. The primary objectives of the system are as follows:

- To construct balanced, labeled datasets from three real-world threat intelligence sources: PhishTank (Cisco Talos Intelligence Group, 2024), URLhaus (abuse.ch, 2024), and Tranco (Le Pochat et al., 2023).
- To engineer 24 URL-string features capturing length, character composition, structural patterns, and entropy across the datasets using a consistent feature schema.
- To train and evaluate three machine learning classifiers, Logistic Regression, Random Forest, and Decision Tree, under controlled and reproducible conditions, using stratified splitting and 5-fold cross-validation.
- To evaluate cross-dataset generalization by training models on one threat source and testing them on another in both directions.
- To produce structured outputs including classification reports, visual dashboards, timestamped training logs, and Comma-Separated Values (CSV) results suitable for academic reporting and iterative analysis.

In its current form, the system is a prototype developed for academic and research purposes rather than a production-deployed application. It is designed to support the study of phishing detection using features derived from URL strings and to enable controlled experimentation within a constrained, offline environment.

The system does not include web-based interface, REST API, browser extension, or mobile application. The prototype supports interaction through both command-line

execution and the lightweight graphical user interface (GUI). All training and pipeline execution tasks are performed through command-line scripts, while the GUI is limited to inference functionality only. Future system extension may expose the classification functionality through a REST API, integrate with enterprise security workflows, or provide a richer end-user interface. These potential deployment pathways are considered future extensions and are not a part of the present implementation unless explicitly stated.

The system emphasizes accessibility, reproducibility, and deployment flexibility. Because the pipeline operates entirely on features derived from the URL string and does not require external network interaction during execution, it can be run in a standard Python environment without specialized infrastructure. This makes it suitable for academic evaluation, research replication, and potential adaptation into lightweight security tools, while still maintaining strict control over data handling and execution conditions.

1.3 Overview

The remainder of this document is organized as follows:

- **Section 2: Overall Description**
Provides a high-level description of the system, including its context, major functions, user classes, operating environment, design constraints, and assumptions.
- **Section 3: Software Architecture**
Describes the system's architectural structure, including the pipeline design, component relationships, technology stack, data flow, and supporting architectural principles.
- **Section 4: Software Features and Requirements**
Defines the system's functional and non-functional requirements, external interface requirements, security requirements, and key software quality attributes.
- **Section 5: Software Models**
Presents supporting models and diagrams, including use case, activity, sequence, class, state, and data models, as well as interface and prototype presentations where applicable.
- **Section 6: Data Requirements**
Specifies the logical data model, data dictionary, and policies related to data storage, retention, and management.

- **Section 7: Other Software Requirements**
Describes additional requirements, including installation, deployment, licensing, legal considerations, and supporting documentation.
- **Section 8: Appendices**
Contains supplementary materials such as detailed diagrams, technical specifications, sample inputs and outputs, glossary terms, and references.

2. Overall Description

2.1 Product Perspective

The Privacy-Preserving, Offline Machine Learning System for Phishing Detection Using URL Features is a standalone, offline pipeline that operates independently of any external platforms or services during its execution. It does not integrate with web browsers, email clients, network firewalls, intrusion detection systems, or cloud-based security platforms at runtime. All data ingestion, feature engineering, model training, cross-dataset evaluation, and output generation occur locally within a Python virtual environment on the host machine.

The system is positioned within the broader domain of URL-based threat detection research. Existing commercial solutions, such as browser-integrated phishing filters and enterprise secure web gateways, typically rely on reputation databases, live DNS resolution, or HTML content inspection to classify URLs. This system constrains feature extraction exclusively to information derived from the URL string. This design enables operation without network access while supporting reproducibility and privacy-preserving execution in controlled environments.

In its current form, the system is a prototype developed for academic purposes and is not a production security tool. Its primary purpose is to support the assessment of the effectiveness and generalization capability of features derived from the URL string across independently sourced threat intelligence datasets. The system produces outputs including trained model artifacts, classification reports, visual dashboards, and structured CSV result files for academic analysis.

In addition to command-line interaction, the prototype includes a lightweight graphical user interface (GUI) implemented using Tkinter (phishing_gui.py). This interface enables users to submit a URL for classification and view prediction outputs in a structured, user-friendly format.

This system is designed with a potential production deployment pathway in mind. The inference component (predict.py) classifies URLs using locally stored model artifacts and does not require retraining during operation. This component could be extended to support deployment through a REST API, web interface, or integration into enterprise security systems. These extensions are considered future capabilities and are not part of the current prototype implementation.

The system utilizes three external data sources during the data acquisition phase only:

- PhishTank (Cisco Talos Intelligence Group, 2024) – a community-sourced, crowd-verified database of active phishing URLs. PhishTank provides an XML export (online_valid.xml) containing verified phishing submissions with URL, submission date, and verification status. Only URLs labeled as 1 (verified phishing) are retained.
- URLhaus (abuse.ch, 2024)- a threat intelligence feed operated by the Swiss non-profit abuse.ch in collaboration with Bern University of Applied Sciences. URLhaus tracks malware delivery URLs actively used in botnet campaigns and is integrated into Security Operations Centers (SOCs) workflows and SIEM platforms worldwide. The feed is provided as a comment-prefixed plain-text file (urlhaus_abuse_ch.txt).
- Tranco (Le Pochat et al., 2023)- a research-grade list designed for academic use. Unlike the Alexa Top Sites list, Tranco is hardened against manipulation by averaging rankings from multiple sources over a rolling time window, reducing the risk of SEO-inflated or attacker-controlled domains appearing as legitimate. The top-ranked domains are used as the legitimate class (label=0, legitimate).

Once downloaded, these datasets are treated as immutable, static inputs, and all subsequent processing occurs offline. No network calls, API requests, or external dependencies are required during pipeline execution. All inter-stage communication is performed through structured CSV files stored in the local data directory, and all evaluation processes occur locally within a Python virtual environment.

2.2 Product Functions

The system provides the following major functional capabilities, each implemented as a distinct Python script adhering to the single-responsibility principle:

Script	Single Responsibility	Input	Output
--------	-----------------------	-------	--------

convert_phishtank.py	Parses the PhishTank XML export and converts it into a standardized two-column CSV (url, label=1).	online_valid.xml	phishtank_urls.csv
convert_urlhaus.py	Parses the URLhaus text feed, skips comment lines (#), and produces a standardized two-column CSV (url, label=1).	urlhaus_abuse_c h.txt	urlhaus_urls.csv
build_dataset.py	Merges each malicious source with Tranco legitimate URLs, enforces a 50/50 class balance, shuffles the data, and generates two balanced datasets.	phishtank_urls.c sv urlhaus_urls.csv tranco_top_sites .csv	phishtank_tranco_data set.csv urlhaus_tranco_dataset. csv
prepare_data.py	Cleans, validates, deduplicates, normalizes labels across both datasets in one run.	phishtank_tranc o_dataset.csv urlhaus_tranco_ dataset.csv	cleaned_phishtank_tran co_dataset.csv cleaned_urlhaus_tranco_ dataset.csv
extract_features.py	Engineers all 24 URL string features across both datasets in one run.	cleaned_phishta nk_tranco_data set.csv cleaned_urlhaus_ tranco_dataset. csv	features_phishtank_tran co_dataset.csv features_urlhaus_tranco_ dataset.csv
train_models.py	Trains the 3 classifiers on each dataset; 80/20 stratified split; 5-fold cross-validation; saves output files.	features_phisht ank_tranco_data set.csv features_urlhau s_tranco_datase t.csv	Dashboards, model_results.csv training_results_date_ti me.txt model_lr.pkl model_rf.pkl model_dt.pkl

			scaler.pkl
cross_dataset_eval.py	Trains on PhishTank → test on URLhaus (Direction 1); train on URLhaus → test on PhishTank (Direction 2). Save dashboards and log.	features_phishtank_tranco_data set.csv features_urlhaus_tranco_data set.csv	Cross-eval dashboards, cross_eval_results.csv cross_eval_results_datetime.txt
predict.py	Loads saved .pkl model files; engineers 24 URL string features from a user-supplied URL; runs through all three classifiers; returns per-model classification and confidence scores with majority-vote summary. No model retraining occurs.	outputs/models_name.pkl user-supplied URL string	Console output: Per model classification (malicious/legitimate) confidence score, and majority-vote summary
phishing_gui.py	Provides a graphical interface for URL classification using trained models; calls predict.py logic without retraining.	User-entered URL string	GUI display of per-model classification, confidence scores, and majority-vote result

2.3 User Classes & Characteristics

The system supports multiple user classes across its current prototype implementation and anticipated future deployment contexts. The primary and secondary user classes correspond to the current academic and research use of the system. The tertiary and quaternary user classes represent future deployment scenarios and are included to define forward-looking requirements without implying full implementation in the current prototype.

Primary User: Academic Evaluator / Supervisor

The primary user class for the current prototype is the academic evaluators and supervisor, including project advisors, committee members, and defense examiners. This user class evaluates the system through generated artifacts rather than direct interaction with the pipeline.

These artifacts include trained model outputs, classification reports, visual dashboards, structured results files, and this SRS document. The purpose of this interaction is to assess system design soundness, reproducibility, and the overall implementation quality.

This user class imposes requirements related to interpretability, documentation quality, and output clarity. All outputs must be clearly labeled, self-contained, and understandable without requiring code execution or detailed knowledge of the implementation. Evaluation metrics and visualizations must be presented in a manner consistent with standard machine learning reporting practices.

Technical proficiency required: No script execution is required. Familiarity with machine learning evaluation metrics (e.g., precision, recall, F1-score) and experimental methodology is expected at the graduate academic level.

Secondary User: Future Student / Academic Replicator

The secondary user class consists of graduate students, cybersecurity researchers, and academic practitioners who seek to reproduce, validate, or extend the system. This user interacts directly with the system by executing scripts, modifying components, and analyzing outputs.

This user class installs the system in a local environment, acquires input data, and executes the pipeline to reproduce results or adapt the system to new research questions. They may operate in different environments, including alternative operating systems or updated raw data.

This user class drives requirements related to reproducibility, modularity, and portability. The system must support independent execution of each pipeline stage, maintain consistent data formats across stages, and provide sufficient documentation to enable accurate replication. The architecture must allow individual components, such as feature engineering or model training, to be modified without affecting other stages.

Technical proficiency required: Proficiency in Python 3.11.x, virtual environment management, command-line execution, and a working understanding of supervised machine learning concepts.

Tertiary User: Corporate Security Team / IT Department (Future Deployment Context)

The tertiary user class represents enterprise security personnel, including Security Operations Center (SOC) analysts and IT administrators. This user class is associated

with future deployment scenarios and does not fully interact with the current prototype beyond command-line-based inference.

In a production context, this user class would utilize the system to classify URLs received through email gateways, web proxies, or user-reported incidents. Interaction would occur through an API, command-line tool, or integration with a Security Information and Event Management (SIEM) system.

This user class imposes requirements related to inference performance, reliability, and operational usability. The system must provide rapid classification, high recall for malicious detection, and clear output that is suitable for decision making. Additionally, the system must operate without requiring live interaction with potentially malicious URLs.

In the current prototype, interaction is limited to use of `predict.py` for single-URL classification via the command line or through the Tkinter-based GUI interface.

Technical proficiency required: In the prototype, command-line proficiency is required. In a production deployment, familiarity with API integration or security tooling is expected.

Quaternary User: Individual End User / General Public (Future Deployment Context)

The quaternary user class represents non-technical individuals who may interact with a future deployment version of the system through a web interface, browser extension, or mobile application. This user class is not supported in the current prototype and is included to define future usability and interface requirements.

This user submits a URL for classification and receives a plain-language safety assessment without exposure to technical details. The system must provide intuitive interaction, near-instantaneous response, and assurance that URLs are not visited or executed as part of the classification process.

This user class drives requirements related to usability, accessibility, and privacy. The system must provide intuitive interaction, near-instantaneous response, and assurance that URLs are not visited or executed during classification.

Technical proficiency required: None.

2.3.1 Use Cases

The following use cases describe how the four user classes interact with the system across its major functional scenarios. Use Cases 1-4 describe interactions within the

current research prototype. Use Cases 5-7 describe future deployment scenarios and are included to define forward-looking system requirements.

Use Case 1- Execute Full Pipeline

Actor:

Future Student / Academic Replicator

Goal:

Execute all eight of the pipeline scripts in sequence to produce the trained models, evaluation dashboards, and results files from raw data sources.

Description:

The user downloads raw data files from PhishTank, URLhaus, and Tranco, places them in the data/ directory, then executes each of the eight pipeline scripts in order, seven processing and training pipeline scripts followed by a validation of the inference component predict.py to confirm the inference path is working. The system converts the raw formats, builds balanced datasets, cleans and validates data, engineers 24 URL-string features, trains three ML classifiers, evaluates within-dataset performance, and produces visual dashboards and results files.

Preconditions:

- Python 3.11.x virtual environment is created and activated
- All required packages are installed via pip install -r requirements.txt
- Data/online_valid.xml, data/urlhaus_abuse_ch.txt, and data/tranco_top_sites.csv are all present in the data/ directory
- The outputs/ directory exists or will be created by the pipeline

Main Success Scenario:

1. User runs: python src/convert_phishtank.py
The system parses XML and saves data/phishtank_urls.csv
2. User runs: python src/convert_urlhaus.py
The system skips comment lines and saves data/urlhaus_urls.csv.
3. User runs: python src/build_dataset.py
The system merges sources with Tranco, with caps at 10,000 per class, shuffles, and saves two balanced 20,000 row datasets
4. User runs: python src/prepare_data.py
The system cleans, deduplicates, and normalizes labels across both datasets, saving two cleaned CSVs
5. User runs: python src/extract_features.py
The system engineers all 24-URL string features across both datasets, saving two feature CSVs
6. User runs: python src/train_models.py

The system trains Logistic Regression, Random Forest, and Decision Tree classifiers on each dataset with 80/20 stratified split and 5-fold cross-validation; saves dashboards, model_results.csv, and timestamped training log

7. User runs: `python src/cross_dataset_eval.py`
The system trains on one dataset and tests on another for each dataset; saves cross-eval dashboards and results
8. User runs: `python src/predict.py --url https://example-phishing-url.com`
The system loads the saved .pkl model files from outputs/, engineers 24 URL string features from the test URL, and returns per-model classifications with confidence scores and a majority-vote summary, confirms that the inference path is operational
9. System confirms all outputs saved to outputs/ directory

Alternative Scenarios:

- Missing Input File: if any raw data file is missing, the relevant converter or build script exits with a descriptive error message identifying the missing file and providing the download URL. The pipeline does not proceed until the file is present.
- Encoding Error: if a source file contains non-UTF-8 characters, the system automatically retries with latin-1 encoding and logs a warning message.

Postconditions:

- All pipeline outputs are present in outputs/: two dashboards from train_models.py, two dashboards from cross_dataset_eval.py, model_results.csv, cross_eval_results.csv, and timestamped text logs; predict.py inference confirmed operational.
- Feature CSVs (26 columns each: url + label + 24 features) are present in the data/ directory
- Trained model .pkl files are present in outputs/ and predict.py is confirmed operational for subsequent inference use by User Classes 2, 3, and 4; results are reproducible through re-execution under identical conditions

Use Case 2- Evaluate New Data Using Saved Models

Actor:

Future Student / Academic Replicator

Goal:

Classify new URLs using previously trained models via predict.py without retraining.

Description:

The user invokes predict.py with a URL argument. The system loads the previously trained model files (.pkl) from outputs/, engineers the 24 URL-string features from the input URL using the same logic as extract_features.py, runs the URL through all three classifiers, and returns a malicious/legitimate classification with a confidence score from each model. Unlike scripts 1-7, predict.py does not require the full pipeline to be re-executed; it operates independently on any previously trained model artifacts.

Preconditions:

- train_models.py has been previously executed and saved model .pkl files to outputs/
- Python virtual environment is activated
- The user has a URL to classify

Main Success Scenario:

1. User submits a URL either via command-line execution (python src/predict.py --url <https://example-phishing-url.com>) or through the GUI interface (phishing_gui.py)
2. System loads saved model file from outputs/
3. System engineers the 24 URL-string features from the provided URL
4. System runs the URL through Logistic Regression, Random Forest, and Decision Tree classifiers
5. System returns a classification (malicious/legitimate) and confidence score for each model
6. System displays the majority vote classification across the three models

Alternative Scenarios:

- Missing Model Files: if no saved model files are found in outputs/, the system notifies the user to run train_models.py first.

Postconditions:

- Classification result and confidence scores are displayed for all three models
- A majority-vote summary classification is provided

Use Case 3- Compare Model Performance Within Datasets

Actor:

Academic Evaluator / Supervisor

Goal:

Review and compare the classification performance of all three trained models on within dataset evaluation results.

Description:

After train_models.py has been executed, the user reviews the saved model_results.csv, the timestamped training log, and the visual dashboard files to compare Logistic Regression, Random Forest, and Decision Tree performance on the PhishTank and URLhaus datasets.

Preconditions:

- train_models.py has been executed and outputs are present in outputs/
- outputs/model_results.csv contains results for all three models across both datasets

Main Success Scenario:

1. User opens outputs/model_results.csv to review accuracy, precision, recall, F1-score, and confusion matrix values
2. User opens training_results_*name*.txt to review full classification report and cross-validation scores
3. User opens dashboard files to view the metrics comparison bar chart, feature importance chart, and confusion matrix heatmaps
4. User compares model performance across accuracy, precision, recall, and F1-score columns
5. User identifies the best performing model for each dataset based on F1-score and recall balance

Alternative Scenarios:

- Results File Not Found: if outputs/model_results.csv is missing, the user is directed to run train_models.py to generate results.

Postconditions:

- User has identified the relative strengths and weaknesses of each model on each dataset
- The feature importance ranking from Random Forest is available for interpretation

Use Case 4- Evaluate Cross-Dataset Generalization

Actor:

Academic Evaluator / Supervisor

Goal:

Determine whether models trained on one threat intelligence source maintain detection performance when tested on a different source.

Description:

The user executes `cross_dataset_eval.py`. The system trains each of the three models on the full PhishTank + Tranco feature dataset and evaluates on the full URLhaus + Tranco feature dataset (direction 1), then reverses the process (direction 2). The results enable analysis of whether the 24 URL-string features evaluate model performance across datasets to assess generalization capability.

Preconditions:

- Both `features_*name*_tranco_dataset.csv` files are present in `data/` (produced by `extract_features.py`)
- Python virtual environment is activated

Main Success Scenario:

1. User runs: `python src/cross_dataset_eval.py`
2. System loads both full feature datasets (20,000 rows each)
3. System trains Logistic Regression, Random Forest, and Decision Tree on PhishTank features (direction 1 training phase)
4. The system applies features scaling based on training data only, ensuring no data leakage occurs
5. System evaluates all three models on URLhaus features and reports metrics
6. System trains all three models on URLhaus features (direction 2 training phase)
7. System evaluates all three models on PhishTank features and reports metrics
8. System saves two cross-eval dashboards, `cross_eval_results.csv`, and timestamped text log to `outputs/`

Alternative Scenarios:

- Feature Schema Mismatch: if the two feature CSVs have different column sets, the system exits with an error identifying the schema difference and instructing the user to rerun `extract_features.py` on both datasets

Postconditions:

- Cross-dataset performance metrics are saved to `outputs/cross_eval_results.csv`
- User has quantitative evidence of whether the 24 URL-string features generalize across the threat sources
- The generalization asymmetry between the two directions is documented in the results

Note: The following use cases are included for architectural completeness and do not represent implemented functionality in the current prototype.

Use Case 5- Corporate Security Team Screens a Batch of Suspicious URLs

This use case represents a future deployment scenario.

Actor:

Corporate Security Team / IT Department

Goal:

Rapidly screen a set of URLs flagged by an email gateway, web proxy, or employee report to determine which require immediate investigation.

Description:

A corporate SOC analyst or IT administrator receives a set of URLs flagged as potentially suspicious by their organization's security tooling or reported by employees. The analyst uses the deployed phishing URL detection tool to classify each URL using predict.py (current prototype) or an equivalent API endpoint (future production deployment). The system returns a per-URL classification with confidence scores near-instantaneously, allowing the analyst to prioritize which URLs warrant a deeper forensic investigation. Because the system operates entirely on URL-string features without visiting or resolving the URL, the analyst is not exposed to potentially malicious content during the screening process.

Preconditions:

- The system has been trained and model .pkl files are present in the outputs/ (prototype), or the deployed model endpoint is available (future production)
- Python virtual environment is activated (prototype) or REST API is running (future production)
- The analyst has a list of one or more URLs to classify

Main Success Scenario:

1. Analyst submits a URL for classification: `python src/predict.py --url [name of the suspicious url]`
2. System engineers 24 URL string features from the URL without contacting the URL
3. System loads trained model artifacts and runs the URL through all three classifiers
4. System returns: per-model classification (malicious/legitimate), per-model confidence score, and a majority-vote summary
5. Analyst repeats for each URL in the batch, or in a production deployment, they would submit a batch request to the API endpoint
6. Analysts use the classification results to prioritize which URLs require escalation, blocking, or further investigation

Alternative Scenarios:

- High-Confidence Malicious: if all three classifiers return “malicious”, with confidence scores above 0.90, the analyst will flag the URL for immediate blocking without further review.
- Mixed Classification: if classifiers disagree, the analyst shall escalate the URL for manual review rather than relying solely on the majority vote.
- URL Without HTTP Scheme: if the submitted URL lacks a scheme prefix, the system processes it as a string feature input and notes the absence of https/http in the is_https feature; classification then proceeds as normal

Postconditions:

- Each submitted URL has received a “malicious” or “legitimate” classification with a confidence score from each of the three models
- The analyst has a prioritized listing of URLs requiring further investigation or immediate blocking
- No potentially malicious URL was visited or resolved during the classification process

Use Case 6- Individual User Verifies a Suspicious URL Link Before Clicking

This use case represents a future deployment scenario.

Actor:

Individual End User / General Public

Goal:

Verify whether a URL received in an email, text message, or social media post is safe to click before opening it in a browser.

Description:

An individual user receives a URL from an unknown source/sender or in an unexpected context and is uncertain whether it is safe to open. Using a future web interface, browser extension, or mobile application built on the phishing URL detection engine, the user pastes or types the URL into the interface and receives a plain-language safety verdict within one second. The system classifies the URL using the same 24 URL string features and trained models as the current prototype, abstracted behind a non-technical interface. The user does not need to understand machine learning, feature engineering, or classification confidence, they receive a clear, actionable recommendation.

Preconditions:

- A production-deployed web interface, browser extension, or mobile app is available (future deployment; current prototype uses predict.py CLI or GUI)
- The user has a URL they wish to verify
- The user does not need to visit the URL to have it classified

Main Success Scenario:

1. User pastes or types the URL into the verification interface
2. System engineers 24 URL-string features from the URL string without visiting or resolving the URL
3. System applies trained models and returns a majority-vote classification
4. Interface displays a plain-language result: e.g., “This URL appears safe” or “This URL has been flagged as potentially malicious, we recommend not clicking on it”
5. User decides whether to open the URL based on the classification result

Alternative Scenarios:

- **Malicious Classification:** the interface displays a warning with a recommendation not to click and a suggestion to report the URL to the sender’s email provider.
- **Low Confidence or Mixed Vote:** the interface displays a caution message: “This URL could not be confidently classified, proceed with caution”.
- **URL Shortened or Obfuscated:** the system classifies the URL as follows; it notes any entropy anomaly, unusual character counts, or suspicious structural patterns in the URL string. It does not resolve URL shorteners because resolving them would require visiting the URL.

Postconditions:

- The user has received a safety verdict for the URL without having visited or resolved it
- The URL string was processed entirely locally or on a secure server; it was not browsed, resolved, or executed at any point

Use Case 7- Future Student Extends the Pipeline with a New Feature Set

This use case represents a future deployment scenario.

Actor:

Future Student / Academic Replicator

Goal:

Adapt the existing pipeline to investigate a new research question by modifying the feature engineering stage while reusing all other pipeline stages unchanged.

Description:

A graduate student working on a cybersecurity research project wishes to extend the phishing URL detection pipeline by replacing or augmenting the 24 URL-string features with a new set of features. For example, adding lexical n-gram features, WHOIS-based registration age features, or domain reputation scores. Because the pipeline is built on the single-responsibility-principle with CSV-based inter-stage contracts, the student would only need to modify `extract_features.py` to produce a new feature schema. All other scripts would require no modification, provided the output CSV retains the URL and label columns alongside the newly added feature columns.

Preconditions:

- The student has cloned or downloaded the pipeline repository
- Python 3.11.x virtual environment is created, and all packages have been installed from `requirements.txt`
- Raw data files have been obtained from PhishTank, URLhaus, and Tranco, and placed in the `data/` directory
- The student has read Section 3.6 (Data Architecture) of this SRS to understand the feature schema contract

Main Success Scenario:

1. Student runs Scripts 1-4 unchanged to produce cleaned datasets
2. Student modifies the `extract_features.py` to implement the new or extended feature set, maintaining the URL and label columns in the output schema
3. Student runs the modified `extract_features.py`, the new features CSVs are produced and placed in the `data/` directory
4. Student runs `train_models.py` unchanged, the script reads feature column names dynamically from the CSV header and trains on whatever features are now present
5. Student runs `cross_dataset_eval.py` unchanged, cross-dataset evaluation proceeds with the new feature schema
6. Student reruns `predict.py` (Script 8) with a test URL to confirm the inference script is compatible with the newly trained `.pkl` files before concluding that the extension works
7. Student compares new results against the baseline results documented in this SRS to assess whether the new features improve cross-dataset generalization

Alternative Scenarios:

- Schema Mismatch Between Datasets: if the two feature CSVs have different column counts, `cross_dataset_eval.py` exits with a schema mismatch error and instructs the student to rerun `extract_features.py` on both datasets.

- **predict.py Compatibility:** the student must rerun train_models.py to regenerate the .pkl files trained on the new feature schema before using predict.py, loading the old .pkl files with a new feature schema will cause a feature count mismatch error.

Postconditions:

- New features CSVs reflecting the extended feature set are present in data/ directory
- New trained model .pkl files (compatible with the new feature schema) and evaluation dashboards reflecting the extended features are present in outputs/ directory, and predict.py has been confirmed operational with the new models
- The student has quantitative results comparing baseline and extended feature set performance for inclusion in their own research report

2.4 Operating Environment

The system operates within a local execution environment and requires no network connectivity during pipeline execution. The following environment specifications apply:

Title	Specifications
Operating System	Windows 10/11 (future compatibility with macOS and Linux)
Python Version	Python 3.11.x
Virtual Environment	Python venv (standard library)
Machine Learning Library	Scikit-learn >= 1.1
Development Environment	Visual Studio 2022 or an equivalent IDE (optional); any terminal capable of executing Python scripts
Execution Interface	Command-line interface (CLI) via terminal or IDE-integrated console, and graphical user interface (GUI) via the Tkinter-based application (phishing_gui.py)
Hardware (minimum)	4 GB RAM, 2 GB free disk space, dual-core CPU
Hardware (recommended)	8 GB RAM, 4 GB free disk space, quad-core CPU
Network Requirements	Internet access required only for initial data download; not required during pipeline execution

2.5 Design & Implementation Constraints

The following constraints are binding design and implementation decisions that apply to all components of the system. Each constraint supports reproducibility, system consistency, and controlled execution. These constraints apply to the current prototype implementation unless otherwise specified.

- **Programming Language- Python 3.11.x:** the entire pipeline is implemented in Python 3.11.9. No R, Julia, MATLAB, or other languages are used in any pipeline stage. This constraint ensures that a single virtual environment with one requirements.txt file is sufficient to reproduce all results. This ensures consistent dependency management and reproducible execution across environments.
- **URL-String-Only Feature Constraint:** All 24 engineered features must be derivable from the raw URL string alone. No DNS resolution, WHOIS lookups, HTTP/HTTPS content fetching, certificate inspection, or calls to external threat intelligence APIs are permitted at any pipeline stage. This constraint defines the feature engineering boundary of the system. It ensures:
 - The pipeline runs fully offline
 - Both datasets use an identical 24-feature schema, enabling valid cross-dataset comparison
 - Inference via predict.py is near-instantaneous with no network latency
 - Results are reproducible regardless of the current network reachability of the URLs.
- **Single-Responsibility Principle:** Each of the eight Python scripts performs exactly one well-defined transformation. No script combines multiple responsibilities, for example:
 - convert_phishtank.py: performs only format conversion
 - extract_features.py: performs only feature engineering
 - train_models.py: performs only within-dataset training and evaluation

This constraint enables independent testing, replacement, and debugging of each stage.

- **Balanced Class Distribution:** Both PhishTank + Tranco, and URLhaus + Tranco datasets must maintain a strict 50/50 class split (malicious label=1,

legitimate label=0), capped at 10,000 samples per class per dataset (20,000 rows total per dataset). This constraint prevents class imbalance from artificially inflating accuracy metrics and ensures that all evaluation metrics, especially precision, recall, and F1-score, reflect the genuine model discrimination rather than majority-class bias.

- **Full Reproducibility via a Fixed Random Seed:** All random operations throughout the pipeline use `random_state=42`, including dataset shuffling, train-test splitting, cross-validation, and model initialization (Logistic Regression, Random Forest, Decision Tree). All file paths are constructed using `os.path.join()` for cross-platform compatibility. Pipeline re-execution on identical input files must produce functionally equivalent outputs.
- **No External Machine Learning APIs or Pre-Trained Models:** All modeling is performed using scikit-learn exclusively. No cloud ML services (e.g. AWS, SageMaker, Google Vertex AI), AutoML platforms, large language models (LLMs), or pre-trained feature extractors are used. This constraint ensures that the pipeline is self-contained, free of API key dependencies, and executable without cloud accounts or internet access during execution.
- **Filesystem-Only Inter-Stage Communication:** No script communicates with another through function calls, shared memory, sockets, or databases. All inter-stage data transfer is performed through CSV files stored in the `data/` directory and model artifacts stored in the `outputs/` directory. This architectural constraint ensures each stage is independently executable and ensures that the pipeline state is fully inspectable at every intermediate step.
- **Academic Integrity and Data Origin:** All three data sources are publicly available and freely licensed for research use. PhishTank data is available under community terms, URLhaus data is made freely available by abuse.ch for security research, and Tranco is published under an open research license. No proprietary, confidential, or personally identifiable data is used at any stage of the pipeline.

2.6 Assumptions & Dependencies

This section documents the assumptions required for the system to operate correctly, and the external dependencies that must be satisfied before pipeline execution can begin. These assumptions and dependencies apply to the current prototype implementation unless otherwise specified. Violations of these assumptions, such as malformed source files or a missing package, will cause pipeline stages to fail with a descriptive error message.

Data Source Assumptions

- The PhishTank XML export (online_valid.xml) is manually downloaded from phishtank.com and placed in the data/ directory before running convert_phishtank.py. The file is assumed to be a valid PhishTank XML export containing an <entries> root element with <entry> child elements, each carrying <url>, <phish_id>, and <verified> fields. All entries in this dataset are assumed to be community-verified as phishing. The system does not verify dataset recency.
- The URLhaus malicious URL feed (urlhaus_abuse_ch.txt) is manually downloaded from urlhaus.abuse.ch and placed in the data/ directory before running convert_urlhaus.py. The file is assumed to follow the URLhaus plain-text format with comment lines prefixed by the # character. The pipeline skips all comment lines and processes only data rows. URL records with an empty or missing URL field are silently skipped.
- The Tranco Top Sites list (tranco_top_sites.csv) is manually downloaded from tranco-list.eu and placed in the data/ directory before running build_dataset.py. The file is assumed to follow the standard two-column Tranco format of rank and domain. The pipeline prepends "http://" to each domain to construct a URL-format string for consistent feature extraction. The top 10,000 domains are used as the legitimate class.
- All three source files are assumed to be encoded in UTF-8. If a file contains non-UTF-8 characters, the pipeline will automatically retry with latin-1 encoding and logs a warning. If both encodings fail, the script exits with a descriptive error.

Pipeline Execution Dependency

- run_pipeline.bat is a Windows batch file provided as a convenience execution wrapper. It executes all eight pipeline scripts (including predict.py with a sample URL) in defined sequence with a single double-click invocation. This eliminates the need to run each script individually from the command line. The batch file assumes:
 - The virtual environment has been created and is accessible at .\venv\.

- All three raw data files are present in data/ directory.
- That the PowerShell execution policy permits script execution.

This script supports automated execution of all pipeline stages in sequence. On macOS/Linux, an equivalent shell script (run_pipeline.sh) may be substituted, but is not provided.

Python Package Dependency

All Python package dependencies are declared in requirements.txt in the project root. The following table documents each package, its minimum required version, and its role within the pipeline. All packages are installable via `pip install -r requirements.txt` within the activated virtual environment. No packages require compilation from source or system-level installation.

pandas >= 1.5.0	Data loading, manipulation, and CSV I/O
NumPy >= 1.23.0	Numerical operations and array handling
scikit-learn >= 1.1.0	Model training, evaluation, preprocessing (StandardScaler, StratifiedKFold)
matplotlib >= 3.6.0	Dashboard and chart generation
seaborn >= 0.12.0	Confusion matrix heatmap rendering

3. Software Architecture

3.1 Architectural Overview

3.1.1 High-Level Architecture

This architectural style describes the architectural structure of the current prototype implementation. The system is structured as an eight-script sequential data processing pipeline, with seven scripts forming a linear training path and one script (predict.py) providing a separate, post-training inference path. Each script accepts well-defined input from the local filesystem and produces well-defined outputs back to the local filesystem. All inter-stage data transfer is performed through CSV files in the data/ directory and model artifacts in the outputs/ directory.

The pipeline progresses through four logical phases along the training path, followed by a standalone inference path:

- Phase 1- Data Ingestion: This phase includes convert_phishtank.py, which parses the PhishTank XML export and normalizes phishing URLs into a standardized two-column CSV (url, label=1). convert_urlhaus.py parses the

URLhaus plain-text feed, skips any comment lines, and produces an equivalent two-column CSV for phishing URLs. These scripts isolate format-specific parsing logic, ensuring that upstream format changes affect only a single script.

- Phase 2- Dataset Construction: `build_dataset.py` independently merges each malicious source with Tranco legitimate URLs, enforcing a 50/50 class balance that is capped at 10,000 samples per class, and shuffles the combined 20,000-row datasets. Both datasets are constructed in a single script execution, producing two parallel datasets that flow identically through all downstream stages.
- Phase 3- Preparation and Feature Engineering: `prepare_data.py` cleans, deduplicates, validates, and normalizes labels across both datasets in a single pass. `extract_features.py` derives all 24 URL-string features across both cleaned datasets in a single pass, producing two feature datasets containing 26 columns (url, label, and 24 engineered features). The identical feature schema applied to both datasets ensures consistency required for cross-dataset comparison.
- Phase 4- Modeling and Evaluation: `train_models.py` trains three classifiers (Logistic Regression, Random Forest, Decision Tree) on each feature dataset independently using an 80/20 stratified split and 5-fold cross-validation, producing dashboards, results CSVs, and timestamped logs.
- Inference Path: `predict.py` operates independently of the training pipeline. It loads the serialized model files (.pkl) saved by `train_models.py`, derives the same 24 URL-string features from a single user-supplied URL, runs the URL through all three classifiers, and returns a per-model classification with confidence scores and a majority-vote summary. This path supports potential future deployment scenarios without requiring pipeline re-execution.

The training pipeline and inference path share the same feature engineering logic but operate independently. The training pipeline is sequential and batch oriented. The inference path is interactive and single-record-oriented. This architectural separation ensures that the `predict.py` can be used after an initial training run without re-executing the remaining pipeline scripts.

3.1.2 Architectural Style/Pattern

The system follows a Layered Pipeline architecture, aligned to the ETL (extract, transform, load) pattern adapted for machine learning workflows. The mapping is defined as follows: the extract phase corresponds to the two converter scripts that ingest raw source data; the transform phase spans dataset construction, cleaning, and feature engineering; and the load phase corresponds to model training,

evaluation, and output generation. Each layer has a clearly defined responsibility and consumes only the output of the preceding layer. No layer re-reads raw inputs or bypasses an intermediate stage. (IBM, n.d.)

This pattern is used in preference to alternative approaches based on system requirements for reproducibility, modularity, and offline execution. A monolithic script would limit the ability to independently verify or audit individual pipeline stages (Amazon Web Services, 2026). A Jupyter notebook approach, while common in exploratory data science, does not enforce stage boundaries and makes reproducibility dependent on cell execution order rather than file contracts (Wang et al., 2020). A microservices or event-driven architecture would introduce unnecessary infrastructure complexity without providing proportional benefit for a single-user offline system (Amazon Web Services, 2026).

The architecture does not implement MVC (model-view-controller), as the graphical user interface is minimal and does not introduce a distinct controller or view layer separate from the core pipeline logic. It does not use an event-driven pattern, as there are no asynchronous triggers or reactive data flows (Microsoft Build 2026, 2026a). It does not use a microservices pattern, as all logic executes on a single host without containerization or inter-process communication. These architectural patterns are not used due to the batch-processing, single-user, offline nature of the current system. This distinction reflects that the corporate security team and individual end user described in Section 2.3 interact with a future deployed system rather than the current prototype. Production deployment may introduce a service layer, (e.g., REST API, containerization, and user interface), without altering the core pipeline architecture.

3.1.3 Architectural Principles

The following architectural principles govern every design decision in the system. Each principle defines a guided constraint applied throughout the system design.

- Separation of Concerns: Data format conversion, dataset construction, data quality assurance, feature engineering, within-dataset modeling, cross-dataset evaluation, and inference are implemented as entirely separate components with no overlapping logic. This ensures that modifications to one component (e.g, feature engineering) do not affect other components. This also ensures that output at each stage can be independently inspected, validated, and archived. (Microsoft Build 2026, 2026b).
- Single Responsibility Principle (SRP): Every script is designed to have one primary responsibility and one primary reason to change. `convert_urlhaus.py` changes only if the URLhaus file format changes. `extract_features.py` changes

only if the feature set changes. `train_models.py` changes only if the modelling strategy changes. This principle ensures that each script can be independently tested, modified, and described without ambiguity. (Microsoft Build 2026, 2026b)

- **Schema Validation Enforcement:** Every script enforces a documented input schema at startup. If an expected column is absent from an input CSV, the script exits immediately with a descriptive error identifying the missing column and the expected schema. This prevents silent data corruption and ensures data integrity across pipeline stages.
- **Fail-Fast Input Validation and Defensive Programming:** All inputs are validated prior to the execution of any computational logic. Conditions such as missing files, malformed CSV structures, schema mismatches, and encoding errors are detected at the earliest possible stage and reported through descriptive, actionable error messages. This fail-fast approach helps to prevent error propagation across pipeline stages, avoiding scenarios in which faults introduced in early stages remain undetected until later stages have executed. This approach improves its ability to be debugged, reduces wasted computation, and supports reproducibility.
- **Reproducibility by Design:** All randomly selected operations are seeded with `random_state=42`, including dataset shuffling, test/train splitting, cross-validation fold assignment, and all three classifier initialization. File naming follows two patterns: result CSVs (`model_results.csv`, `cross_eval_results.csv`), dashboard PNGs, and `.pkl` model artifacts use fixed names and overwrite any previous runs, ensuring that there is always exactly one current result set for each; the training and cross-evaluation logs use timestamped file names (`YYYYMMDD_HHMMSS`) and accumulate across each run. Raw source files in the `data/` directory are never modified by any pipeline script. File paths use `os.path.join()` for cross-platform compatibility. These constraints together ensure that re-executing the pipeline on identical inputs will yield functionally equivalent outputs.
- **File-Based Pipeline Integration:** Pipeline stages are fully decoupled and do not share any in-memory state, communicate through direct function invocation, or rely on shared objects. Instead, the local filesystem serves as the primary mechanism for data exchange between stages, where each script reads its inputs from disk and writes its outputs back to disk. This design ensures that all intermediate artifacts are fully observable, persistent, and independently reproducible. As a result, any stage of the pipeline can be executed or re-executed in isolation without requiring prior stages to be run (unless an input artifact created from a previous stage is needed).

3.1.4 Rationale

This section describes the rationale for the selected architectural decisions implemented in the current prototype. The layered pipeline architecture is used in preference to monolithic script or Jupyter notebook approach for the following reasons. First, it enables independent testing, modification, and inspection of each stage. Second, it allows evaluators to inspect intermediate outputs without re-running the full pipeline and allows individual pipeline stages to be modified independently. Third, it mirrors professional data engineering practice, aligning with established data engineering practices and supporting potential extension to a production deployment.

CSV was selected as the inter-stage data format because it is human-readable and directly inspectable without requiring specialized tools. CSV files are universally supported across Python versions and operating systems and introduce no version dependency risk. They also enable intermediate datasets to be opened in spreadsheet applications for manual verification during evaluation and review.

scikit-learn was selected in preference to deep learning frameworks and gradient boosting libraries based on system requirements for interpretability and controlled evaluation. The system is designed to evaluate whether lightweight, interpretable classifiers can achieve high classification performance using URL-string-derived features alone. Accordingly, Logistic Regression, Decision Tree, and Random Forest models were chosen due to their interpretability, well-established theoretical foundations, and ability to support transparent feature importance analysis and human-readable decision structure. More complex model families, such as deep learning and gradient boosting methods, were not included, as they would increase model complexity without providing proportional benefit given the system's focus on interpretability and controlled evaluation.

3.2 Systems Architecture Diagram

This section describes the structural and interactional architecture of the current prototype implementation through a set of complementary system diagrams. These diagrams collectively illustrate system boundaries, internal component organization, and execution flow across the pipeline.

The system is represented using three primary diagram types: a Context Diagram, a Component Diagram, and a Logical Deployment Diagram. Each diagram captures a different level of abstraction and supports both technical and non-technical understanding of the system.

Component Diagram:

The component diagram represents the internal structure of this system as a sequence of eight modular components corresponding to the pipeline scripts. These components are grouped into four logical phases: Data Ingestion, Data Construction, Data Preparation and Feature Engineering, and Modeling and Evaluation, followed by an independent Inference component. Each component consumes structured input from the file system and produces output artifacts that serve as input to subsequent components. Data flow is strictly unidirectional, and no component shares in-memory state or directly invokes another component.

Logical Deployment Diagram:

The logical deployment diagram represents the execution environment of the prototype. All components execute within a single host machine using a Python 3.11.x virtual environment. The system relies on a local file-based architecture, with data/ serving as the input and intermediate storage directory, and outputs/ serving as the location for model artifacts, evaluation results, and logs. No distributed infrastructure, containerization, or cloud-based services are used in the prototype. This deployment model supports reproducibility, portability, and ease of evaluation.

These diagrams are provided in Appendix A and are referenced throughout Section 3 to support architectural traceability.

3.3 Architectural Components

This section describes the major architectural components of the current prototype system, organized according to logical responsibilities rather than traditional web-based layering. As the system is a standalone pipeline rather than an interactive application, the architecture is composed of execution modules, data processing components, and data storage components.

Presentation Layer:

The prototype includes a lightweight graphical user interface (GUI) implemented using Tkinter, in addition to command-line interaction. The GUI provides a simple user-facing interface for URL classification, allowing the user to input a URL and view classification results including per-model predictions, confidence scores, and a majority-vote summary.

Command-line interfaces remain the primary mechanism for executing the training pipeline and inference scripts. Output artifacts, including CSV files, logs, and visualization dashboards, serve as additional interaction points and are designed to be human-readable for inspection by academic evaluators and future researchers.

The GUI represents a thin presentation layer over the existing inference logic and does not introduce additional processing functionality beyond what is provided by `predict.py`.

Application/Processing Layer:

The core logic of the system resides in a set of eight Python scripts, each implementing a single stage of the pipeline. These components are organized according to the following functional responsibilities:

- **Data Ingestion Components:**
`convert_phishtank.py` and `convert_urlhaus.py` handle parsing and normalization of raw source data into a consistent CSV schema
- **Dataset Construction Component:**
`build_dataset.py` merges malicious and legitimate sources, enforces class balance constraints, and produces structured datasets suitable for downstream processing.
- **Data Preparation Component:**
`prepare_data.py` performs cleaning, deduplication, validation, and label normalization to ensure data quality.
- **Feature Engineering Component:**
`extract_features.py` generates 24 URL-string-derived features from each record, producing datasets used for model training and evaluation.
- **Modeling and Evaluation Component:**
`train_models.py` performs model training, cross-validation, and within-dataset evaluation. `cross_dataset_eval.py` performs cross-dataset generalization testing.
- **Inference Component:**
`predict.py` provides a standalone inference capability by loading trained model artifacts and classifying user-supplied URLs.

Each component operates independently, consuming inputs from the filesystem and producing outputs for subsequent stages.

Data Access Layer:

The system uses a file-based data access model rather than a database or ORM framework. All data is stored as CSV files within the data/ directory and accessed using pandas library. Model artifacts are serialized using pickle and stored in the outputs/ directory.

Data access is performed through direct file I/O operations, with explicit schema validation at each stage. This approach ensures transparency, reproducibility, and ease of inspection, as all intermediate artifacts are persisted and can be independently reviewed or reused.

No caching layer, database system, or external data storage service is used in the current prototype.

3.4 Integration Layer

The current prototype does not include a formal integration layer. The system operates as a standalone, offline pipeline with no external service communication during execution.

API Architecture:

No REST, GraphQL, or gRPC interfaces are implemented in the prototype. All functionality is executed locally through command-line invocation of Python scripts or a lightweight GUI. The inference capability provided by predict.py represents a logical boundary where an API may be introduced in future deployment; however, no such interface is present in the current system.

Third-Party Integration:

The system does not integrate with external services, APIs, or live data sources at runtime. All input data is manually downloaded from publicly available sources (PhishTank, URLhaus, and Tranco) and stored locally in the data/ directory prior to execution. No external calls are made during the pipeline operation.

Messaging Queuing/Event Bus:

No message queuing or event-driven communication mechanism is used. The system follows a strictly sequential execution model, where each pipeline stage is invoked explicitly and completes before the next stage begins.

Service Communication Patterns:

No inter-process or network-based communication occurs within the system. Components do not communicate via HTTP, sockets, or remote procedure calls.

Instead, all inter-stage communication is performed through file-based data exchange using CSV files stored on the local filesystem.

This design reflects the requirements of the current prototype, prioritizing reproducibility, transparency, and offline execution. A future production deployment could introduce an integration layer, including API endpoints and service-based communication, without requiring modification of the core pipeline architecture.

3.5 Technology Stack

The current prototype is implemented using a lightweight, local technology stack designed to support offline execution, reproducibility, and ease of evaluation.

Programming Languages:

The system is implemented in Python 3.11.x. No additional programming languages are used in any component of the pipeline.

Frameworks and Libraries:

The system relies on a small set of established Python libraries for data processing, machine learning, and visualization.

- pandas ($\geq 1.5.0$) for data loading, manipulation, and CSV input/output
- NumPy ($\geq 1.23.0$) for numerical operations
- scikit-learn ($\geq 1.1.0$) for model training, evaluation, preprocessing, and cross-validation
- Matplotlib ($\geq 3.6.0$) for visualization and dashboard generation
- seaborn ($\geq 0.13.0$) for enhanced statistical visualization

No web frameworks, frontend frameworks, or testing frameworks are used.

Database Systems:

No relational or NoSQL database system is used. All data is stored as CSV files within the data/ directory, and model artifacts are serialized using pickle and stored in the outputs/ directory. No caching systems are implemented.

Infrastructure:

The system executes on a single local machine and does not rely on cloud infrastructure, virtualization, or containerization. No cloud services (e.g., AWS, Azure, Google Cloud) are used. Docker, Kubernetes, and other orchestration tools are not part of the current prototype.

Development Tools:

Development and execution are performed using local development tools:

- Python virtual environment (venv) for dependency isolation
- pip for package management (via requirements.txt)
- Command-line interface (CLI) for script execution
- Visual Studio 2022 or equivalent IDE for development (optional, not required for execution)

No CI/CD pipelines, automated tools, or build systems are implemented in the current prototype.

This technology stack supports simplicity, transparency, and reproducibility. The system may be extended in future implementations to include web frameworks, APIs, and cloud-based infrastructure without altering the core pipeline design.

3.6 Data Architecture

This section describes how data is stored, transformed, and managed throughout the current prototype. The system follows a file-based data architecture designed to support transparency, reproducibility, and ease of inspection.

Data Storage Strategy:

The system does not use a database management system. All data is stored as flat CSV files within the data/ directory. These files represent both intermediate and final datasets produced at each stage of the pipeline. Model artifacts generated during training are serialized using pickle and stored in the outputs/ directory. This file-based approach ensures that all intermediate data states are persisted, human-readable, and directly inspectable without specialized tooling. It also eliminates dependencies on database configuration, supporting portability, and reproducibility.

Data Flow:

Data flows through the system in a sequential, stage-based pipeline. Raw source data files are manually placed in the data/ directory and are transformed through a series of processing stages:

- Raw source data (XML, TXT, CSV formats)
 - XML and TXT are converted to normalized CSV format (url, label)
 - → merged and balanced into structured datasets
 - → cleaned and validated datasets
 - → feature datasets containing 24 engineered features
 - → model training and evaluation outputs
 - → serialized model artifacts and evaluation results

All data movement occurs through file-based read/write operations. Each stage consumes input files from disk and produces output files that serve as input to subsequent stages. No in-memory data sharing occurs between stages.

Data Models:

The system uses simple tabular data models represented as CSV files. Three primary data structures are used:

- **Normalized URL Data:**
Columns: url (string), label (integer)
Represents phishing and legitimate URL records after ingestion.
- **Dataset:**
Columns: url, label
Represents balanced and structured data prepared for processing.
- **Feature Dataset:**
Columns: url, label, feature_1 through feature_24
Represents model-ready data with engineered features derived from the URL string.

Relationships between these data models are linear and transformation-based with each stage producing a new dataset rather than modifying existing files.

Caching Strategy:

No caching layer is implemented in the current prototype. All intermediate results are persisted to disk as CSV files and can be reused without re-computation if the corresponding files are present. This approach serves as a form of explicit, file-based caching while still maintaining full transparency of intermediate stages.

Data Partitioning / Sharding:

Data partitioning or sharding is not implemented. All datasets are processed as single, in-memory DataFrame objects within each script execution. Dataset sizes (20,000 rows per dataset) are intentionally constrained to ensure efficient processing within a single-machine environment.

This data architecture is appropriate for the current prototype, prioritizing clarity, reproducibility, and ease of validation. More advanced storage mechanisms, such as

databases or distributed processing frameworks, may be introduced in future implementations if scalability requirements increase.

3.7 Security Architecture

This section describes the security considerations of the current prototype. As the system is a standalone, offline pipeline executing in a controlled local environment, security requirements are minimal and focus on safe data handling and execution integrity.

Authentication Mechanism:

No authentication mechanisms (e.g., OAuth, JWT, SAML) are implemented. The system does not provide user accounts, login functionality, or multi-user access control. All scripts are executed locally by a single user with direct access to the system.

Authorization Model:

No authorization model (e.g., RBAC, ABAC, ACLs) is implemented. All operations are performed with the privileges of the existing user. There are no role-based restrictions or permission layers within the system.

Data Encryption:

No encryption mechanisms are applied to data at rest or in transit. All data is stored locally as CSV files and processed within the local execution environment. No data is transmitted over the network during pipeline execution.

Security Layers:

The system does not implement distinct network, application, or data security layers. Security is instead derived from offline execution model, absence of external connectivity during runtime, and use of publicly available, non-sensitive raw data. The system does not process personally identifiable information (PII) or other sensitive data.

API Security:

No API endpoints are implemented in the current prototype. Therefore, no API security mechanisms such as rate limiting, API keys, or token-based authentication are required.

Operational Safety Considerations:

Although the system processes potentially malicious URLs, it does not resolve, visit, or execute any URL. All feature extraction is performed on the URL string alone, ensuring that no network interaction occurs and reducing exposure to malicious content during execution.

The current security architecture reflects the scope and constraints of the prototype. A future production deployment would require authentication, authorization, secure communication (HTTPS), input validation at API boundaries, and additional security controls appropriate for a multi-user and network-accessible environment.

3.8 Integration Architecture

The current prototype does not implement a formal integration architecture. The system operates as a standalone, offline pipeline with no external service communication during execution. This section describes the absence of integration mechanisms in the prototype and identifies potential integration points for future deployments.

External System Interfaces:

No APIs or web services are consumed at runtime. All external data sources (PhishTank, URLhaus, and Tranco) are obtained through manual download and local file placement in the data/ directory prior to execution. The system does not perform live data retrieval, API calls, or service-based queries.

Communication Protocols:

No network communication protocols (e.g., HTTP, HTTPS, WebSocket) are used during pipeline execution. All processing occurs locally within a single host environment. There is no inter-process communication over a network and no use of message queues or event-driven mechanisms.

Data Exchange Formats:

Data exchange within the system is performed through CSV files stored on the local filesystem. While the original source data may be provided in XML (PhishTank) or plain-text formats (URLhaus), these formats are converted into a standardized CSV schema during the data ingestion phase. No JSON, Protocol Buffers, or other structured interchange formats are used within the pipeline.

Integration Patterns:

The system follows a synchronous, sequential execution pattern. Each pipeline stage is executed sequentially and completes before the next stage begins. There is no asynchronous processing, parallel execution, or event-driven triggering between the components.

This integration architecture reflects the requirements of the current prototype, prioritizing reproducibility, simplicity, and offline execution. A future production deployment may introduce external system interfaces, API-based communication, and asynchronous processing models without requiring modification to the core data process pipeline.

3.9 Scalability & Performance Architecture

This section describes the scalability and performance characteristics of the current prototype. The system is designed for execution in a single-machine environment with moderate dataset sizes and does not implement distributed or large-scale processing mechanisms.

Scaling Strategy:

The system uses a vertical scaling model limited to the resources of a single host machine (CPU and memory resources). Horizontal scaling, distributed computation, and multi-node execution are not implemented. Dataset sizes are intentionally constrained (20,000 rows per dataset) to ensure efficient processing within this environment.

Load Balancing:

No load balancing mechanisms are implemented. The system does not expose services or handle concurrent user requests, as all processing is performed locally by a single user through sequential execution.

Caching Layers:

No dedicated caching layers (e.g., memory caches, CDN, or database caching) are implemented. Instead, intermediate results are persisted to disk as CSV files and can be reused across executions if the corresponding files are present. This file-based persistence provides a transparent and reproducible alternative to traditional caching mechanisms.

Performance Optimization:

Performance is supported through design decisions that prioritize efficiency within the constraints of the prototype environment:

- Dataset size constraints ensure that all processing can be completed in memory without excessive resource consumption.
- Efficient data manipulation is achieved using pandas and NumPy operations.
- Model training uses computationally efficient algorithms (Logistic Regression, Decision Tree, Random Forest) appropriate for tabular data and moderate dataset sizes.
- Feature engineering is limited to URL-string-derived features, eliminating the need for network calls or external data retrieval during execution.
- Sequential execution ensures predictable performance and simplifies debugging and reproducibility.

The current architecture does not support high-throughput or real-time processing. A future production deployment may introduce horizontal scaling, parallel processing, load balancing, and caching mechanisms.

4. Software Features & Requirements

4.1 Functional Requirements

Feature 1: Data Ingestion and Conversion

The system shall convert raw data from external sources into a standardized two-column (url, label) CSV format using dedicated single-responsibility converter scripts.

ID	Requirement Description
FR-1.1 PhishTank XML Parsing and Extraction	The system shall parse the PhishTank XML export file (online_valid.xml) and extract all phishing URLs, outputting phishtank_urls.csv with columns url and label (integer, value=1).
FR-1.2 URLhaus Feed Parsing and Extraction	The system shall parse the urlhaus_abuse_ch.txt feed, skipping all lines beginning with “#”, and extract all phishing URLs regardless of url_status (online or offline status), outputting urlhaus_urls.csv with columns url and label (integer, label=1). Both online and offline URLs are retained.
FR-1.3 Encoding Handling and Fallback Support	The system shall support UTF-8 encoding with automatic latin-1 fallback, logging a warning message if the fallback encoding is used.

FR-1.4 Duplicate URL Removal	The system shall remove duplicate URLs within the converted output and shall log the number of duplicates removed.
FR-1.5 Missing Input File Detection	The system shall terminate execution with a descriptive error message, identifying the missing file and providing the download URL, if the required input file is not found.

Feature 2: Dataset Construction

The system shall merge each malicious source with an equal number of legitimate Tranco URLs to produce two balanced datasets.

ID	Requirement Description
FR-2.1 Balanced Dataset Generalization	The system shall produce two output datasets: phishtank_tranco_dataset.csv and urlhaus_tranco_dataset.csv, each containing equal numbers of malicious (label=1) and legitimate (label=0) URLs.
FR-2.2 Dataset Size Limitation Configuration	The system shall cap malicious URLs at DEFAULT_LIMIT=10,000 per dataset, configurable via a --limit command-line argument. The same number of Tranco URLs shall be selected to maintain a 50/50 balance.
FR-2.3 Dataset Source Isolation	The system shall independently read the Tranco list for each dataset, ensuring that no Tranco URL appears in both datasets simultaneously.
FR-2.4 Dataset Shuffling Determinism	The system shall shuffle the combined dataset using random_state=42 before saving.
FR-2.5 Class Distribution Logging	The system shall log the final class balance (malicious count, legitimate count, percentage split) for each output dataset.

Feature 3: Data Cleaning and Validation

The system shall clean and validate both balanced datasets before feature extraction.

ID	Requirement Description
FR-3.1 URL Column Standardization	The system shall detect URL column names from a predefined candidate list (url, urls, link, domain, address) and standardize the column name to 'url'.
FR-3.2 Label Normalization	The system shall normalize label values to binary integers (0 or 1), mapping string variants including 'phishing', 'malicious', 'legitimate', and 'benign' to their integer equivalents.
FR-3.3 Invalid URL Removal	The system shall remove rows containing null, empty, or whitespace-only URL values and shall log the number of removed counts.

FR-3.4 Invalid Label Removal	The system shall remove rows containing invalid or missing label values and shall log the number of removed rows.
FR-3.5 Duplicate Removal	The system shall remove duplicate URLs and shall log the number of duplicates removed.
FR-3.6 Multi-Dataset Processing Support	The system shall process both datasets in a single execution without requiring separate per-dataset command line calls.
FR-3.7 Empty Dataset Validation	The system shall terminate execution with a descriptive error if no valid rows remain after cleaning.

Feature 4: URL Feature Engineering

The system shall derive 24 URL string features from each cleaned dataset using deterministic string-processing methods. All 24 features shall be computable from the raw URL string alone using deterministic string-processing methods.

ID	Requirement Description
FR-4.1 Length-Based Feature Extraction	The system shall compute length-based features (4): url_length, hostname_length, path_length, query_length. Providing integer character counts of the full URL string and each parsed URL component. hostname_length shall measure the full host including subdomains. For URLs with no explicit path or query, path_length and query_length are zero.
FR-4.2 Character Count Feature Extraction	The system shall compute character count features (10): num_dots, num_hyphens, num_underscores, num_slashes, num_qmarks, num_equals, num_ampersands, num_at, num_percent, num_digits. Providing integer counts of each character type in the full URL.
FR-4.3 Structural Feature Extraction	The system shall compute structural/boolean features (6): has_https (0/1), has_ip (0/1), has_port (0/1), has_www (0/1), has_at_in_url (0/1), subdomain_depth (integer). Providing binary flags and one structural measurement. has_ip flags URLs where the host is a raw IP address rather than a domain name. has_https records whether the scheme is HTTPS. has_at_in_url detects if the @ character is in the URL. subdomain_depth counts the number of subdomain levels that are above the registered domain.
FR-4.4 Entropy and Ratio Feature Computation	The system shall compute entropy and ratio features (4): url_entropy, hostname_entropy (both Shannon entropy values), digit_ratio (proportion of digit characters in the full URL), special_char_ratio (portion of special characters, such as dots, hyphens, underscores in the full URL). All values are represented

	as floating-point numbers. Shannon entropy quantifies uncertainty or randomness in the character distribution of a string (algorithmically generated or obfuscated malicious URLs tend to exhibit a higher entropy than human readable legitimate domain names). digit_ratio and special_char_ratio capture structural obfuscation patterns that entropy alone may not have isolated.
FR-4.5 Input Schema Validation	The system shall validate that input datasets contain 'url' and 'label' columns prior to processing, exiting with an instruction to run prepare_data.py first if columns are missing.
FR-4.6 Label Retention in Feature Dataset	The system shall retain the 'label' column alongside all 24 features for downstream modeling, within the output feature CSVs.
FR-4.7 Consistent Feature Schema Output	The system shall process both datasets in a single invocation, producing identical column schemas for both outputs.

Feature 5: Model Training and Within-Dataset Evaluation

The system shall train and evaluate multiple machine learning models using within-dataset validation.

ID	Requirement Description
FR-5.1 Logistic Regression Training	The system shall train Logistic Regression (max_iter=1000, class_weight='balanced') with feature scaling applied to training data only.
FR-5.2 Random Forest Training	The system shall train Random Forest (n_estimators=200, class_weight='balanced', n_jobs=-1) without feature scaling.
FR-5.3 Decision Tree Training	The system shall train Decision Tree (max_depth=10, min_samples_split=20, class_weight='balanced') without feature scaling.
FR-5.4 Stratified Train/Test Splitting	The system shall perform an 80/20 stratified train/test split using a fixed random seed.
FR-5.5 Performance Metric Computation	The system shall compute and report the accuracy, precision, recall, F1-score, and a full confusion matrix (true negative, false positive, false negative, true positive) for each model on the dataset.
FR-5.6 Cross-Validation Evaluation	The system shall perform 5-fold stratified cross-validation on the training set and report mean and standard deviation for F1-score.
FR-5.7	The system shall save a visual dashboard per dataset to the outputs/ directory, including a metrics table, grouped bar chart, feature importance chart (RF), and confusion matrix heatmaps.

Dashboard Visualization Generation	
FR-5.8 Result and Log Persistence	The system shall save all results to outputs/model_results.csv and all terminal output to a timestamped outputs/training_results_YYYYMMDD_HHMMSS.txt file.

Feature 6: Cross-Dataset Generalization Evaluation

The system shall evaluate whether models trained on one threat source generalize to a different threat source in both directions.

ID	Requirement Description
FR-6.1 PhishTank-to-URLhaus Evaluation	The system shall train models on the PhishTank+Tranco dataset and evaluate on the URLhaus+Tranco dataset (direction 1).
FR-6.2 URLhaus-to-PhishTank Evaluation	The system shall train models on the URLhaus+Tranco dataset and evaluate on the PhishTank+Tranco dataset (direction 2).
FR-6.3 Preprocessing Isolation for Evaluation	The system shall apply preprocessing transformations (e.g., StandardScaler) using training data only to prevent data leakage.
FR-6.4 Cross-Dataset Metric Reporting	The system shall report accuracy, precision, recall, F1-score, and confusion matrix (true negative, false positive, false negative, true positive) for each model in each direction.
FR-6.5 Cross-Evaluation Artifact Persistence	The system shall save two dashboards (one per direction), outputs/cross_eval_results.csv, and a timestamped outputs/cross_eval_results_YYYYMMDD_HHMMSS.txt file.

Feature 7: Experimental Output Generation and Artifact Persistence

The system shall generate, display, and persistently save all experimental outputs and intermediate artifacts to support reproducibility, academic reporting, and iterative analysis. All outputs are stored in the outputs/ directory. No pipeline stage overwrites the outputs of a different stage, however, re-running the same stage shall overwrite its own fixed-name output (result CSVs, dashboard PNGs, .pkl model artifacts) while timestamped log files accumulate across every run.

ID	Requirement Description
FR-7.1 Real-Time Metric Display	The system shall display classification performance metrics for each trained model in the terminal during pipeline execution, including accuracy, precision, recall, F1-score, full scikit-learn

	classification report (per-class precision/recall/F1), confusion matrix counts (true negative, false positive, false negative, true positive), and 5-fold cross-validation mean +- standard deviation.
FR-7.2 Execution Logging and Output Capture	The system shall save all terminal output, including print statements, classification reports, and cross-validation summaries, to a timestamped plain-text log file (training_results_YYYYMMDD_HHMMSS.txt) in outputs/ directory including console display, using a mechanism that mirrors standard output to a file.
FR-7.3 Experimental Artifact Persistence	The system shall persistently store the following experimental artifacts to the outputs/ directory: (1) cleaned datasets in data/ directory for downstream reuse; (2) extracted feature datasets in the data/ directory; (3) model_results.csv containing aggregated metric for all models and datasets; (4) cross_eval_results.csv containing all cross-dataset evaluation metrics; (5) four PNG dashboard visualizations (two within-dataset, two cross-dataset); (6) timestamped training log and timestamped cross-evaluation log.
FR-7.4 Model Serialization and Storage	The system shall save trained model objects (serialized via pickle as .pkl files) to outputs/ directory after training_models.py execution, enabling predict.py to load models and generate predictions without retraining.
FR-7.5 Raw Data Preservation Enforcement	Raw source datasets in data/ directory shall be preserved without modification by any pipeline script. All transformations shall produce new output files rather than overwriting inputs.

Feature 8: URL Inference and Classification (predict.py)

The system shall provide a standalone inference capability that classifies a single user-supplied URL using the trained model artifacts from Feature 5, without requiring any retraining or pipeline re-execution.

Inference shall be accessible through both:

1. A command-line interface (predict.py)
2. A graphical user interface (GUI, phish_gui.py)

Both interfaces shall utilize the same underlying features engineering and model prediction logic, ensuring consistent and deterministic results across interaction modes. This feature represents the prototype inference interface and a logical boundary for future user-facing interfaces.

ID	Requirement Description
----	-------------------------

FR-8.1 URL Input Handling via CLI	The system shall accept a single URL string via a --url command-line argument and shall not require any other arguments for a standard classification run.
FR-8.2 Model Loading and Feature Reuse	The system shall load the serialized .pkl model files from outputs/ directory and apply the identical 24-feature engineering logic as extract_features.py to the input URL, without re-executing the training pipeline. The StandardScaler loaded from outputs/ directory shall be applied (transform only) to the single-row feature vector before Logistic Regression inference.
FR-8.3 Human-Readable Prediction Output	The system shall output a per-model verdict (malicious/legitimate), confidence score (class probability as a percentage), and a majority-vote summary classification for all three trained classifiers. Outputs shall be displayed in the terminal in plain language without requiring interpretation of raw numerical probability arrays.
FR-8.4 Offline Inference Execution	The system shall operate fully offline. The URL string is treated as a passive data record from which only string-derived features are computed.
FR-8.5 Deterministic Prediction Consistency	The system shall produce deterministic output for any given URL: repeated invocations with the same URL and the same .pkl model files shall produce consistent outputs for identical inputs when using the same trained model artifacts.
FR-8.6 GUI URL Input Handling	The system shall provide a graphical user interface that allows the user to input a single URL string through a text input field and initiate classification through a user-controlled action (e.g., button selection).
FR-8.7 GUI Prediction Execution	The system shall invoke the same underlying feature extraction and model inference logic used by predict.py when a classification request is initiated through the graphical user interface, ensuring consistent results across both CLI and GUI interfaces.
FR-8.8 GUI Output Display	The system shall display classification results within the graphical user interface, including per-model predictions (malicious/legitimate), confidence scores, and a final majority-vote classification in a structured and human-readable format.

4.2 External Interface Requirements

4.2.1 User Interfaces

The system provides both a command-line interface (CLI) and a graphical user interface (GUI) for user interaction. The CLI supports execution of all pipeline stages, including data processing, model training, evaluation, and inference.

The GUI, implemented using Tkinter (phishing_gui.py), provides a simplified interface for URL classification during inference. Users may input a URL and view classification results, including per-model predictions, confidence scores, and a majority-vote summary.

All training, dataset construction, and evaluation functionality remains accessible only through the CLI.

All visual outputs generated by the system are saved as image files in the outputs/ directory for review.

Example command-line usage:

- `python src/convert_phishtank.py` (no arguments needed, reads data/online_valid.xml by default)
- `python src/build_dataset.py --limit 5000` (optional override of the default 10,000 row cap)
- `python src/predict.py --url https://example.phishing.domain.com` (classifies a URL using saved models)

4.2.2 Hardware Interfaces

The system does not require any specialized hardware and is designed to operate on a standard personal computer.

The following hardware resources are utilized:

- **CPU:** Used for all data processing, feature extraction, and model training operations. Random Forest enables parallel tree construction via `n_jobs=-1`, utilizing available CPU cores.
- **RAM:** Used to store datasets and feature representation during execution. Peak consumption occurs during Random Forest training.
- **Local storage:** Used to store input data, intermediate datasets, model artifacts, visual outputs, and logs. Approximately 200-400 MB required for a full pipeline run.

No GPU acceleration, specialized networking hardware, and peripheral devices (cameras, printers, etc.) are required or used in the current prototype.

4.2.3 Software Interfaces

The system interacts with the following software components:

- Python 3.11.x runtime environment for execution of all scripts

- pandas ($\geq 1.5.0$) for data loading, DataFrame manipulation, and CSV input and output
- NumPy ($\geq 1.23.0$) for numerical array operations, Shannon entropy computation
- scikit-learn ($\geq 1.1.0$) for model training, preprocessing, and evaluation
- Matplotlib ($\geq 3.6.0$) for visualization and dashboard generation
- seaborn ($\geq 0.13.0$) for statistical visualization

The system also utilizes standard Python libraries for command-line interaction, file system operations, and regular expression processing, XML parsing and mathematical computations.

All feature engineering operations are performed using deterministic string processing and do not rely on external services during execution.

4.2.4 Communication Interfaces

The system does not implement runtime communication interfaces. No HTTP, HTTPS, WebSocket, or message-based communication protocols are used during execution.

All inter-stage data exchange occurs through file-based interactions using CSV files stored in the data/ directory and output artifacts stored in the outputs/ directory.

External data acquisition is performed manually prior to execution. The system does not perform automated data retrieval at runtime.

4.3 Non-Functional Requirements

4.3.1 Performance

ID	Requirement Description
NFR-P-1 End-to-End Pipeline Execution Time	The system shall complete full pipeline execution (all 8 scripts), from raw data ingestion through to saved model artifacts and evaluation dashboards, within 10 minutes on the recommended hardware. Random Forest training is the dominant computational cost, which executes four times across the within-dataset and cross-dataset evaluation stages.

NFR-P-2 Individual Script Execution Time	Each individual pipeline script shall complete execution within 3 minutes on the recommended hardware for datasets of up to 20,000 rows.
NFR-P-3 Dataset Processing Capacity	The system shall process phishing URL datasets containing up to 50,000 URLs within a reasonable execution time (under 20 minutes) without code modification, controlled using the --limit parameter in build_dataset.py.
NFR-P-4 Memory Utilization Constraint	Memory consumption shall not exceed 2 GB during execution on datasets of up to 20,000 rows.
NFR-P-5 Visualization Output Generation Time	The system shall generate and save visualization outputs within 30 seconds of model evaluation completion.
NFR-P-6 Hardware Independence	The system shall execute all computations without requiring GPU acceleration or cloud-based resources.

4.3.2 Reliability

ID	Requirement Description
NFR-R-1 Continuous Pipeline Execution	The system shall execute the full machine learning pipeline without interruption when valid inputs and all required dependencies are present.
NFR-R-2 Deterministic Output Consistency	The system shall produce consistent outputs when executed multiple times with identical inputs and configurations. All randomly determined operations shall use a fixed random seed.
NFR-R-3 Input Validation and Error Detection	The system shall detect missing files, invalid formats, encoding errors, and schema mismatches at the start of each script and shall notify the user with a specific, actionable error message before any processing begins.
NFR-R-4 Execution and Output Integrity	The system shall not produce partial or corrupted output. Scripts shall either complete successfully and write all outputs or exit with an error before writing any output.

4.3.3 Usability

ID	Requirement Description
NFR-U-1 Ease of Pipeline Execution and Inference	The system shall allow users to execute the full training pipeline by placing raw data files in the data/ directory and executing the seven pipeline scripts in sequence and support URL inference after model training by either running the predict.py script with a --url

	argument in CLI or using the graphical user interface (GUI). No code modification shall be required for a standard pipeline run or for a standard inference query on a previously trained model.
NFR-U-2 Multi-Format Result Presentation	The system shall present results in both numerical form (console output, CSV files) and graphical form (PNG dashboard visualization) to facilitate interpretation of machine learning model performance.
NFR-U-3 Informative Error Messaging	All error messages shall include: (1) the specific file or column that caused the error; (2) the reason for the failure; (3) the corrective action required (e.g., download URL or script to run first).
NFR-U-4 Default Configuration Usability	The system shall operate with default configurations and shall not require mandatory command-line arguments for standard execution.

4.3.4 Maintainability

ID	Requirement Description
NFR-M-1 Modular Script Architecture	The system shall maintain a modular architecture where each of the eight scripts is implemented as a separate, independently executable Python module without cross-imports between scripts.
NFR-M-2 Source Code Documentation Standards	Each source file shall include a header block describing script purpose, inputs, outputs, and usage example.
NFR-M-3 Independent Component Modifiability	Modifications to one pipeline stage shall not require changes to any other pipeline script.
NFR-M-4 Centralized Logging Support	The system shall use Python's logging module throughout to enable log-level control without modification of source code.
NFR-M-5 System Extensibility	The system shall support extension by allowing new features, models, or data sources to be added without modifying existing components.

4.3.5 Portability

ID	Requirement Description
NFR-PO-1 Cross-Platform Execution Support	The system shall execute without modification on Windows (with the run_pipeline.bat file), macOS, and Linux (with the correct run_pipeline.sh file not provided currently) systems capable of executing Python 3.11.x

NFR-PO-2 Platform-Independent File Handling	The system shall use platform-independent file path handling (<code>os.path.join()</code>) to ensure cross-platform compatibility.
NFR-PO-3 Dependency Environment Reproducibility	All dependencies shall be defined in the <code>requirements.txt</code> and installable in an isolated virtual environment using a single command (<code>pip install -r requirements.txt</code>), ensuring consistent execution across systems and preventing conflicts with system-level Python packages. The single <code>pip install</code> command shall be sufficient to recreate the full dependency environment on any supported platform.

4.3.6 Data Integrity

ID	Requirement Description
NFR-DI-1 Raw Data Preservation	Raw source data files (<code>phishtank_urls.csv</code> , <code>urlhaus_urls.csv</code> , <code>tranco_top_sites.csv</code>) shall not be unmodified by any pipeline script. All transformation operations shall produce new output files in <code>data/</code> directory.
NFR-DI-2 Persistent Artifact Storage	All intermediate processing outputs (cleaned dataset, feature datasets) and final outputs (model results, dashboards, logs) shall be stored in designated directories and shall not be deleted by pipeline scripts during normal execution.
NFR-DI-3 Dataset Validation Enforcement	The system shall validate dataset structure (required columns, non-null values, valid label values) before executing feature extraction or model training. Scripts shall exit with an informative error if required data integrity conditions are not met.
NFR-DI-4 Training Data Isolation (No Data Leakage)	The <code>StandardScaler</code> fitted during model training shall be fitted exclusively on training data. The scaler shall not be fitted on the test data or cross-dataset evaluation data at any point, preventing data leakage.

4.4 Security Requirements

This section defines the security requirements for the current prototype. The system operates as a standalone, offline application within a local execution environment. Security controls focus on data handling integrity, controlled execution, and minimizing exposure to external threats.

4.4.1 Authentication

ID	Requirement Description
SR-A-1	The system shall not require user authentication, as it operates within a local single-user environment.

No Authentication Requirement	
SR-A-2 OS-Level Access Control	Access to the system shall be controlled by the host operating system's user account permissions.

4.4.2 Authorization

ID	Requirement Description
SR-AU-1 No Role-Based Authorization	The system shall not implement a role-based authorization model in the current prototype.
SR-AU-2 User-Level Execution Permissions	All operations shall be performed with the privileges of the executing user.

4.4.3 Data Protection

The following data protection practices are applied:

ID	Requirement Description
SR-DP-1 Offline Data Processing Constraint	The system shall not initiate any network connections during pipeline execution. All data must be pre-downloaded and placed in data/ directory before execution begins.
SR-DP-2 No Sensitive Data Handling	The system shall not store, log, or transmit any personally identifiable information. The system processes only publicly available URL strings. No user credentials or personal data are handled.
SR-DP-3 Raw Data Integrity Preservation	Raw source data files shall be preserved without modification throughout the pipeline. All processing outputs are written to separate files, ensuring that the original data remains intact, and the preprocessing steps can be repeated from scratch if necessary.
SR-DP-4 Local Artifact Storage Only	All experimental artifacts (feature datasets, model results, dashboards, logs) shall be stored locally within the project directory and shall not be transmitted to external services.
SR-DP-5 URL-Only Feature Processing	The system shall perform all feature extraction using URL-string data only and shall not resolve or access URLs.

4.4.4 Secure Execution Environment

The system's execution environment provides the following security controls:

ID	Requirement Description
SR-SEE-1 No Elevated Privilege Requirement	The system shall operate without requiring elevated or administrative privileges.
SR-SEE-2 Virtual Environment Isolation	The system shall use an isolated Python virtual environment for dependency management.
SR-SEE-3 Dependency Predefinition Requirement	All dependencies shall be defined in requirements.txt and installed prior to execution.
SR-SEE-4 No Network Exposure During Execution	The system shall not open network ports, host services, or accept inbound connections during execution.

4.4.5 Future Security Considerations (Future Implementation)

The following controls are not implemented in the current prototype but may be required in future deployment:

- Authentication mechanism
- Secure communication (HTTPS/TLS)
- Input validation for externally submitted URLs
- Rate limiting and request throttling
- Security logging and monitoring

4.5 Software Quality Attributes

Software quality attributes describe the overarching characteristics that define how effectively the system satisfies its intended purpose. These attributes complement the functional and non-functional requirements by specifying system-level quality expectations.

4.5.1 Reliability

ID	Requirement Description
SQA-R-1 Consistent Execution	The system shall execute the machine learning pipeline without failure when valid datasets and all required dependencies are provided. Pipeline execution shall produce consistent results when inputs are identical.

SQA-R-2 Reproducible Results	The system shall produce consistent, reproducible experimental results when identical datasets, preprocessing steps, and model configuration are used. Fixed random seeds shall be applied to all randomly determined operations.
SQA-R-3 Error Detection and Graceful Failure	The system shall validate all inputs at the start of execution and shall terminate with descriptive error messages if invalid conditions, such as invalid datasets, missing required input files, incompatible feature schemas, and encoding errors, are detected.
SQA-R-4 Output Integrity Assurance	The system shall not produce partial or corrupted outputs; each script shall either complete successfully or terminate prior to writing output files.

4.5.2 Availability

ID	Requirement Description
SQA-A-1 Local System Availability	The system shall be available for execution whenever the host system and required software environment are operational, including the required virtual environment and installed dependencies.
SQA-A-2 Offline Operational Capability	The system shall not depend on external services during execution and shall function fully offline once input data has been obtained.

4.5.3 Maintainability

ID	Requirement Description
SQA-M-1 Modular Code Architecture	The system shall maintain a modular architecture consisting of independently executable scripts, each responsible for a single processing stage.
SQA-M-2 Code Documentation Completeness	All source code modules shall include file-level documentation describing purpose, inputs, outputs, and usage.
SQA-M-3 Model and Code Extensibility	Modification to one pipeline stage shall not require modification of other stages, provided input/output data contracts are preserved.
SQA-M-4 Architectural Extensibility Support	The system shall support extension through the addition of new models, features, or data sources without requiring architectural redesign.

4.5.4 Portability

ID	Requirement Description
----	-------------------------

SQA-P-1 Cross-Platform Compatibility	The system shall execute on Windows 10/11, macOS, and Linux without platform-specific code changes. All path handling uses <code>os.path.join()</code> , all file I/O specifies explicit encoding, and no Windows-only or Unix-only system calls are used.
SQA-P-2 Platform- Independent File Operations	All file operations shall use platform-independent path handling mechanisms.
SQA-P-3 Environment Replicability	The system shall support environment replication through a <code>requirements.txt</code> file and isolated Python virtual environment.

4.5.5 Scalability

ID	Requirement Description
SQA-S-1 Dataset Scalability Support	The system shall support processing datasets of varying sizes by adjusting the <code>DEFAULT_LIMIT</code> parameter in <code>build_dataset.py</code> .
SQA-S-2 Experimental Scalability Support	The system architecture shall support expansion of experimental scope, including additional datasets, features, and models.
SQA-S-3 Performance Scalability Constraint	The system shall maintain acceptable execution performance on datasets up to at least 50,000 records per class within a single-machine environment, requiring no code changes.

4.5.6 Usability

ID	Requirement Description
SQA-U-1 Ease of System Execution	The system shall support system execution through command-line operations for full pipeline processing and provide a graphical user interface (GUI) for simplified inference interaction.
SQA-U-2 Multi-Format Result Interpretability	The system shall provide outputs in multiple formats, including human-readable classification reports in the terminal, machine-readable CSVs for further analysis, and visual dashboards as PNG files for reporting.
SQA-U-3 Clear Error Reporting	The system shall generate clear, descriptive error messages that identify the cause of failure and required corrective action.
SQA-U-4	The system shall allow users to perform inference using a single command without requiring retraining or pipeline re-execution.

Simplified Inference Execution	
---------------------------------------	--

4.5.7 Interpretability

ID	Requirement Description
SQA-I-1 Feature Importance Transparency	The system shall provide model interpretability through feature importance metrics and model coefficients where applicable. These scores shall be included in the visual dashboard output and training logs.
SQA-I-2 Human-Readable Prediction Output	The system shall output per-model verdicts as plain-language labels (malicious/legitimate) alongside confidence scores (class probability) and a majority-vote summary, without requiring the user to interpret raw numerical probability arrays.
SQA-I-3 Intermediate Artifact Transparency	The system shall enable inspection of intermediate data artifacts to support transparency and reproducibility.

4.5.8 Compliance

ID	Requirement Description
SQA-C-1 Open-Source License Compliance	The system shall comply with the licensing terms of all third-party open-source libraries. No General Public License licensed dependencies that would impose distribution restrictions are used.
SQA-C-2 Academic Research Compliance	The system shall support reproducible research practices and transparent reporting of machine learning results in compliance with academic research standards. All dataset sources are documented with full academic citations and access methodology.
SQA-C-3 Documentation Completeness and Reproducibility	System documentation (SRS) shall describe the machine learning pipeline, dataset usage, feature engineering decisions, evaluation methodology, and system architecture in sufficient detail to support independent review and reproduction.

5. Software Models

5.1 Use Case Diagrams

Two use case diagrams are provided, both in Appendix A.

The first diagram (current prototype) covers the use cases implemented in the current system:

- UC-1 Execute Full Pipeline (Scripts 1-8)
- UC-2 Evaluate New Data Using Saved Models (predict.py)
- UC-3 Compare Model Performance Within Dataset
- UC-4 Evaluate Cross-Dataset Generalization
- UC-7 Extend Pipeline with New Feature Set

UC-7 is performed by the Future Student / Academic Replicator and represents system extensibility within the prototype.

All <<extend>> relationships are defined, where the extending use case points to the base use case:

- UC-2 extends UC-1 (predict.py is an optional post-pipeline inference step)
- UC-4 extends UC-3 (cross-dataset evaluation represents a deeper optional analysis)
- UC-7 extends UC-2 (pipeline extensions are validated through inference using predict.py)

The second diagram (future deployment) introduces:

- UC-5 Screen Batch of Suspicious URLs (Corporate Security Team/ IT Department)
- UC-6 Verify Suspicious URL Before Clicking (Individual End User/ General Public)

Both:

- UC-5 <<extend>> UC-2
- UC-6 <<extend>> UC-2

In this context, UC-2 serves as the shared inference base use case. UC-2 may be executed through either command-line interaction (predict.py) or the graphical user interface (phishing_gui.py), both of which utilize the same underlying inference logic.

Full use case specifications are documented in Section 2.3.1.

5.2 Activity Diagrams

The activity diagram is provided in Appendix A.

The diagram models the two independent execution paths of the system.

The Training Pipeline (Scripts 1-7) illustrates the sequential control flow from raw ingestion through model training, evaluation, and artifact generation. The diagram includes two decision nodes:

- Are Source Files Available?
- Is Dataset Valid?

Both implement fail-fast error handling with explicit exit paths for Missing Input File and Validation Failure.

The Inference Path diagram (Script 8) models the on-demand inference flow, beginning with user-supplies input and concluding with classification output. It includes a decision node:

- Run Saved Model Inference?

And an error branch:

- Model Load Error (triggered when required .pkl artifacts are absent)

Both paths utilize identical 24-feature engineering logic but operate independently.

Training Pipeline path (Scripts 1-7 executed once per dataset version)

1. convert_phishtank.py → data/phishtank_urls.csv
2. convert_urlhaus.py → data/urlhaus_urls.csv
3. build_dataset.py → two balanced raw datasets
4. prepare_data.py → cleaned datasets
5. extract_features.py → two feature CSVs
6. train_models.py → within-dataset results, dashboards, .pkl files
7. cross_dataset_eval.py → cross-dataset results and dashboards

Inference Path (Script 8 executed on demand)

User supplies URL via --url, predict.py loads .pkl model artifacts, applies identical 24-feature extraction, runs classification across all models, returns per-model verdict, confidence scores, and majority vote-summary, exits with no files written.

Branching occurs only in error conditions:

- Missing input files (training path)
- Missing model artifacts (inference path)

5.3 Sequence Diagrams

The sequence diagram is provided in Appendix A.

The diagram models the interaction between system components across both execution paths.

The Training Pipeline sequence (Scripts 1-7) begins with the User initiating pipeline execution. The conceptual PipelineController coordinates sequential interactions with:

- PhishTankConverter
- URLhausConverter
- DatasetBuilder
- DataCleaner
- FeatureExtractor
- ModelTrainer
- CrossDatasetEvaluator

Each component produces output artifacts passed forward in the pipeline.

ModelTrainer includes a nested interaction with ModelRepository to persist trained .pkl model artifacts.

The Inference Path sequence (Script 8) begins with the User initiating inference. For GUI-based interaction, the graphical interface acts as an intermediary between the User and the InferenceEngine, forwarding user input and displaying classification results without altering the underlying inference sequence. The InferenceEngine:

- Loads model artifacts via ModelRepository
- Applies feature extraction
- Performs classification
- Returns predictions result to the User

Note: The PipelineController shown in the sequence is a conceptual abstraction used to represent the logical orchestration. In the implemented system, modules communicate through file-based CSV artifacts, rather than direct method invocation.

5.4 Class Diagrams

The class diagram is provided in Appendix A.

The diagram models the system using primary module classes corresponding to the pipeline architecture.

- PhishTankConverter
- URLhausConverter
- DatasetBuilder
- DataCleaner

- FeatureExtractor
- ModelTrainer
- ModelRepository
- CrossDatasetEvaluator
- InferenceEngine
- GUIInterface
- PipelineController (conceptual)

Supporting data classes include:

- URLRecord
- Dataset
- FeatureVector
- ModelArtifact
- EvaluationResult
- CVResult

Composition relationships indicate ownership:

- Dataset composes URLRecord

Association relationships represent logical dependencies between components. These dependencies are conceptual; in the implemented system, integration occurs through file-based data exchange rather than direct call methods.

The system is modular and loosely coupled. Each module:

- Reads input artifacts from disk
- Processes data independently
- Writes output artifacts back to disk

The InferenceEngine maintains a read-only dependency on ModelRepository and does not modify stored artifacts.

5.5 State Diagrams

Two state diagrams are provided in Appendix A.

The system is modelled as two independent machines:

- Training Pipeline (Scripts 1-7)
- Inference Path (Script 8)

Both originate from an initial idle / initialized state but diverge thereafter.

The Training Pipeline includes states for:

- Dataset Loading
- Preprocessing
- Feature Extraction
- Model Training
- Evaluation
- Visualization
- Completion

It includes terminal error states for:

- Missing Input File
- Validation Failure
- Schema Mismatch

The Inference Path includes:

- Model Loading
- Feature Extraction
- Classification
- Completion

and a terminal error state:

- Model Load Error

The inference state machine operates independently but requires trained model artifacts produced by the training pipeline.

5.6 Entity-Relationship Diagrams

The entity-relationship (logical data relationship) diagram is provided in Appendix A.

The system is modelled using flat-file data entities rather than a relational database.

Training Path Entities (persisted)

- RawURL- initial ingested data
- BalancedDataset- class-balanced dataset
- CleanedURL- validated and de-duplicated dataset
- FeatureRecord – feature-engineered dataset (24 features per record)
- ModelArtifact- serialized classifier objects and StandardScaler (.pkl files)
- ModelResult- within-dataset evaluation results
- CrossEvalResult- cross-dataset evaluation results

Inference Path Entities (transient)

- InferenceResult- classification output

Relationships follow a sequential transformation:

RawURL → BalancedDataset → CleanedURL → FeatureRecord → ModelResult/
CrossEvalResult

Inference operates independently:

User Inputs Request → FeatureRecord (in-memory) → InferenceResult

The inference process maintains a read-only dependency on ModelArtifacts outputs from the training pipeline.

5.7 Wireframe/Prototypes

Two interface models are provided in Appendix A:

- Future web interface wireframe
- Current tkinter-based GUI prototype

The wireframe represents planned, simplified interface for non-technical end users, providing a single input field for URL submission and a clear classification result display.

The current system does not implement a web-based interface: instead, it provides multiple interaction modes supported through command-line and graphical interfaces.

Three interface modes are supported:

- Training CLI (Scripts 1-7)
- Inference CLI (predict.py)
- GUI interface (phishing_gui.py)

The GUI interface provides a structured visual output including per-model classification results, confidence scores, and a majority-vote summary.

Dashboard outputs serve the Academic Evaluator and include:

- performance metrics
- comparative visualization
- feature importance plots
- confusion matrices

Representative examples of the GUI interface, command-line outputs, and dashboard visualizations are provided in Appendix D to illustrate system behavior across both training and inference workflows.

6. Data Requirements

6.1 Logical Data Model

The system processes data through a series of transformations, beginning with raw input data and progressing through structured intermediate representations to model artifacts and evaluation outputs.

The primary data entities are defined as follows:

- **Raw Data Source**
Represents externally obtained data (PhishTank XML, URLhaus text feed, Tranco CSV). These files contain unprocessed URL records in source-specific formats.
- **URL Record**
Represents a single URL instance with an associated label indicating class membership (malicious=1, legitimate=0)
- **Dataset**
Represents a collection of labeled URL records after merging the malicious and legitimate sources into a balanced dataset.
- **Feature Set**
Represents a transformed dataset where each URL record is converted into a fixed-length feature vector consisting of 24 URL-derived features.
- **Model Artifact**
Represents a trained machine learning model serialized to disk (.pkl), including any preprocessing components (e.g., StandardScaler).
- **Evaluation Result**
Represents performance metrics, including accuracy, precision, recall, F1-score, and confusion matrix values.

Relationships between the entities are as follows:

- Raw Data Source → URL Records
- URL Records → Dataset
- Dataset → BalancedDataset
- BalancedDataset → CleanedDataset
- CleanedDataset → Feature Set

- Feature Set → ModelArtifacts (via train_models.py)
- Feature Set → ModelResults (via train_models.py)
- Feature Set → CrossEvalResult (via cross_dataset_eval.py, both directions)

All data transformations occur sequentially and are persisted between stages.

6.2 Data Dictionary

The system uses structured CSV files as the primary data format. Key data elements are defined below. All datasets are stored within the data/ directory, and all outputs are stored within the outputs/ directory.

6.2.1 Raw Input Data (Normalized Output of Conversion Stage)

Field Name	Data Type	Description	Length	Allowed Values/Constraints
url	String	Extracted URL from source data	<= 2048 characters	Non-null, non-empty
label	Integer	Class label assigned during ingestion	1 digit	{0, 1} where 1= malicious, 0= legitimate

6.2.2 Cleaned Dataset (Post-Validation Stage)

Field Name	Data Type	Description	Length	Allowed Values/Constraints
url	String	Validated and cleaned URL string	<= 2048 characters	Non-null, non-empty, duplicates removed
label	Integer	Normalized class label	1 digit	{0, 1}

6.2.3 Feature Set (Post-Feature Engineering)

Field Name	Data Type	Description	Length	Allowed Values/Constraints
url	String	Original URL string	<= 2048 chars	Non-null
label	Integer	Class Label	1 digit	{0, 1}
url_length	Integer	Total length of URL string	N/A	Integer ≥ 0
hostname_length	Integer	Length of hostname component	N/A	Integer ≥ 0
path_length	Integer	Length of URL path	N/A	Integer ≥ 0
query_length	Integer	Length of query string	N/A	Integer ≥ 0

num_dots	Integer	Count of '.' characters in the URL	N/A	Integer ≥ 0
num_hyphens	Integer	Count of '-' characters in the URL	N/A	Integer ≥ 0
num_underscores	Integer	Count of '_' characters in the URL	N/A	Integer ≥ 0
num_slashes	Integer	Count of '/' characters in the URL	N/A	Integer ≥ 0
num_qmarks	Integer	Count of '?' characters in the URL	N/A	Integer ≥ 0
num_equals	Integer	Count of '=' characters in the URL	N/A	Integer ≥ 0
num_ampersands	Integer	Count of '&' characters in the URL	N/A	Integer ≥ 0
num_at	Integer	Count of '@' characters in the URL	N/A	Integer ≥ 0
num_percent	Integer	Count of '%' characters in the URL	N/A	Integer ≥ 0
num_digits	Integer	Count of numeric characters in the URL	N/A	Integer ≥ 0
has_https	Integer	Indicator of whether the URL uses HTTPS	1 digit	{0, 1}
has_ip	Integer	Indicator of whether the hostname is an IP address	1 digit	{0, 1}
has_port	Integer	Indicator of whether the URL explicitly specifies a port number	1 digit	{0, 1}

has_www	Integer	Indicator of whether the hostname contains 'www'	1 digit	{0, 1}
has_at_in_url	Integer	Indicator of whether '@' appears anywhere in the URL	1 digit	{0, 1}
subdomain_depth	Integer	Number of subdomain levels in the hostname	N/A	Integer ≥ 0
url_entropy	Float	Shannon entropy of the full URL string	N/A	Float ≥ 0
hostname_entropy	Float	Shannon entropy of the hostname component	N/A	Float ≥ 0
digit_ratio	Float	Ratio of digit characters to total length	N/A	Float $\in [0, 1]$
special_char_ratio	Float	Ratio of selected special characters to the total URL length	N/A	Float $\in [0, 1]$

6.2.4 Model Artifacts

Field Name	Data Type	Description	Length	Allowed Values/Constraints
model	Binary (.pkl)	Serialized machine learning model (Logistic Regression, Random Forest, Decision Tree)	N/A	Must be a valid scikit-learn serialized object
scaler	Binary (.pkl)	Serialized StandardScaler object used for feature normalization	N/A	Fitted on training data and applied to Logistic Regression inference only. Random Forest and Decision Tree operate on unscaled features.

6.2.5 Evaluation Results

Field Name	Data Type	Description	Length	Allowed Values/Constraints
accuracy	Float	Classification accuracy score	N/A	Float $\in [0, 1]$
precision	Float	Precision score	N/A	Float $\in [0, 1]$
recall	Float	Recall score	N/A	Float $\in [0, 1]$
f1_score	Float	F1-score	N/A	Float $\in [0, 1]$
confusion_matrix	Array (2x2)	Confusion matrix values (TN, FP, FN, TP)	4 elements	Non-negative integers

6.2.6 Log Artifacts (System Output)

Field Name	Data Type	Description	Length	Allowed Values/Constraints
log_entry	String	Log message generated during execution	Variable	Non-null
Timestamp	String	Time associated with the log event	N/A	Formatted according to configured logging format (e.g., HH:MM:SS)
log_level	String	Severity level of the log message	N/A	Predefined logging levels such as INFO and ERROR

6.3 Data Retention

The system retains all generated data artifacts within the project directory unless explicitly removed by the user.

- **Raw Data Retention:** Raw source files remain unchanged and are preserved for reproducibility. These files serve as the authoritative input for all pipeline executions.
- **Intermediate Data Retention:** Cleaned datasets and feature-engineered datasets are retained within the data/ directory to support inspection, debugging, and reuse without requiring reprocessing.
- **Output Retention:** Model artifacts, evaluation results, dashboards, and logs are stored in the outputs/ directory.

The system does not implement automated archival policies. Data lifecycle management is manual and controlled by the user.

Timestamped log files accumulate across executions, while fixed-name output files (e.g., model_results.csv, dashboard images, serialized models) are overwritten during subsequent runs of the same pipeline stage.

6.4 Data Migration

The system does not depend on legacy systems, external databases, or real-time data ingestion pipelines. Data migration requirements are limited to importing externally obtained data/datasets into the expected directory structure.

- **Input Data Migration:** Users must manually download raw input data from external sources and place them into the data/ directory prior to execution.
- **Format Conversion:** Conversion scripts normalize source-specific formats into a standardized CSV schema consisting of url and label fields.
- **Output Export:** All generated outputs can be manually copied or transferred to external systems for analysis, reporting, or archival purposes.

No automated data migration tools or pipelines are implemented.

6.5 Backup & Recovery

The system relies on the host environment for data backup and recovery.

- **Backup Strategy:** No automated backup mechanism is implemented within the system. Users are responsible for maintaining backups of the project directory if required.
- **Recovery Process:** Data recovery is performed either by restoring files from external backups or by re-running the pipeline from raw data sources.
- **Recovery Time Objective (RTO):** Recovery time is bounded by pipeline execution duration. A full system rebuild, including data processing, model training, and evaluation, can be completed within approximately 20 minutes on recommended hardware referenced in Section 2.4.
- **Recovery Point Objective (RPO):** The system has an RPO of zero for reproducible data, as all intermediate and final outputs can be deterministically regenerated from raw input data.

Because all processing is deterministic and based on static input data, full system recovery is achieved through re-execution rather than state restoration.

7. Other Software Requirements

This section defines additional requirements that do not fall under functional, non-functional, or architectural categories but are necessary to ensure completeness of

the system specification. These requirements reflect both the constraints of the current prototype and considerations for potential future deployment.

7.1 Internationalization & Localization

The current system does not implement internationalization or localization features. All inputs, outputs, logs, and documentation are provided exclusively in English.

The system assumes a standard environment using UTF-8 encoding with basic fallback handling mechanisms, but does not perform language-specific parsing, translation, or locale-aware formatting.

Future deployments may incorporate internationalization and localization capabilities, including:

- Multi-language user interfaces
- Regional formatting (e.g., date, time, and numeric formats)
- Localization of user-facing outputs

These features are outside the scope of the current prototype.

7.2 Installation & Deployment

The system is designed for local execution within a controlled development environment and does not require distributed infrastructure or external services.

Installation Requirements

- Python 3.11.x installed on the host system
- Creation of an isolated Python virtual environment (venv)
- Installation of dependencies using:
 - `pip install -r requirements.txt`

Configuration Requirements

- Placement of raw input data within the `data/` directory
- No additional configuration files, environment variables, or external services are required

Execution

- Pipeline scripts are executed via command-line interface (CLI) in a predefined sequential order
- URL inference may be performed via CLI (`predict.py`) or through the graphical user interface (GUI) implemented in `phishing_gui.py`

- The GUI supports inference-only interaction and does not provide access to training or pipeline execution functionality
- Optional batch execution is supported through the run_pipeline.bat script (Windows environment)
- Equivalent shell script execution may be implemented for macOS/Linux environments

Deployment Environment

- Local machine execution (Windows, macOS, or Linux)
- Python-based runtime with no dependency on cloud platforms or containerization

The system is not deployed as a service and does not support remote access, concurrent users, or multi-user environments in its current form.

7.3 Licensing Requirements

The system is developed as an academic research prototype and does not implement a commercial licensing model.

All third-party dependencies are open-source and distributed under permissive license, including:

- pandas (BSD License) (Oracle, 2026)
- NumPy (BSD License) (Oracle, 2026)
- scikit-learn (BSD License) (Oracle, 2026)
- Matplotlib (PSF license) (matplotlib, 2026)
- seaborn (PSF license) (matplotlib, 2026)

No license key management, activation mechanisms, or usage restrictions are implemented within the system.

Any future distribution, modification, or deployment of the system must comply with the licensing terms and conditions of all included third-party libraries.

7.4 Legal & Regulatory Requirements

The system processes publicly available data and does not handle personal, sensitive, or regulated information.

Privacy: No personally identifiable information (PII) is collected, stored, or processed. The system does not implement any privacy controls because none are required given the nature of the data processed.

Data Usage and Regulatory Scope: All raw data sources (PhishTank, URLhaus, Tranco) are publicly available and intended for security research purposes. The system does not fall under frameworks such as HIPAA, GDPR, PCI-DSS, or other data protection regulations in its current form.

These controls may be required in a future production deployment involving external users, user-generated input, or the processing of regulated data.

7.5 Documentation Requirements

The system includes documentation sufficient to support reproducibility, evaluation, and extension by researchers and developers.

Required Documentation Artifacts

- Software Requirements Specification (SRS) document
- Source code documentation, including:
 - File-level header comments describing purpose, inputs, and outputs
 - Function-level inline comments describing parameters, return values, and behavior
- README file describing system setup, execution procedures, and project structure.

Output Documentation

- Automatically generated execution logs (timestamped)
- CSV files containing model evaluation results and metrics
- Visual dashboards (PNG format) summarizing model performance

The system does not include:

- Interactive help systems
- End-user manual
- API documentation

Future enhancements may include:

- Web-based user documentation
- API reference documentation
- Training materials for non-technical users

These enhancements are outside the scope of the current prototype.

8. Appendices

Appendix A: Detailed Architecture Diagrams

This appendix contains the system diagrams and models referenced throughout Section 5.

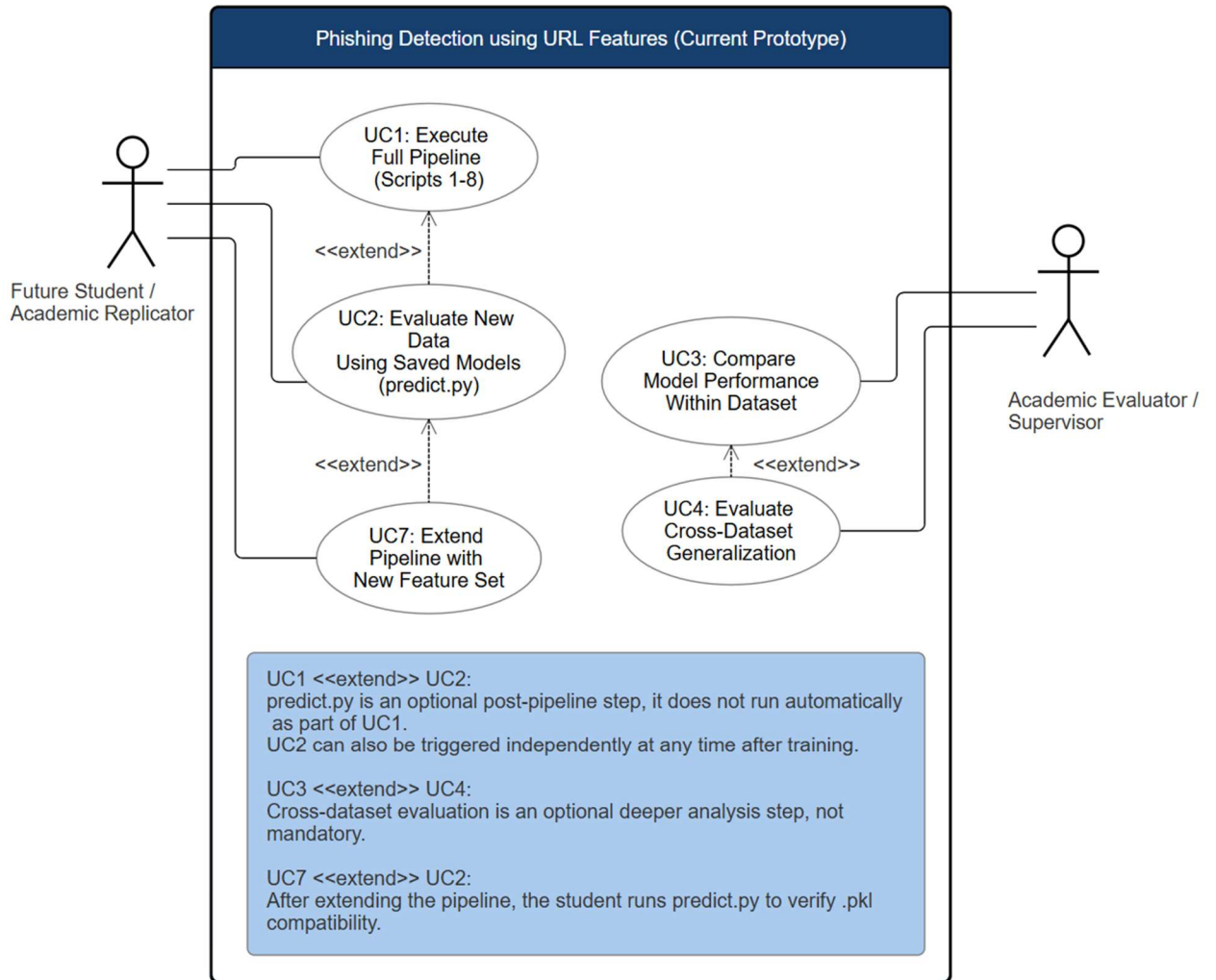


Figure A.1: UML Use Case Diagram- Current Prototype

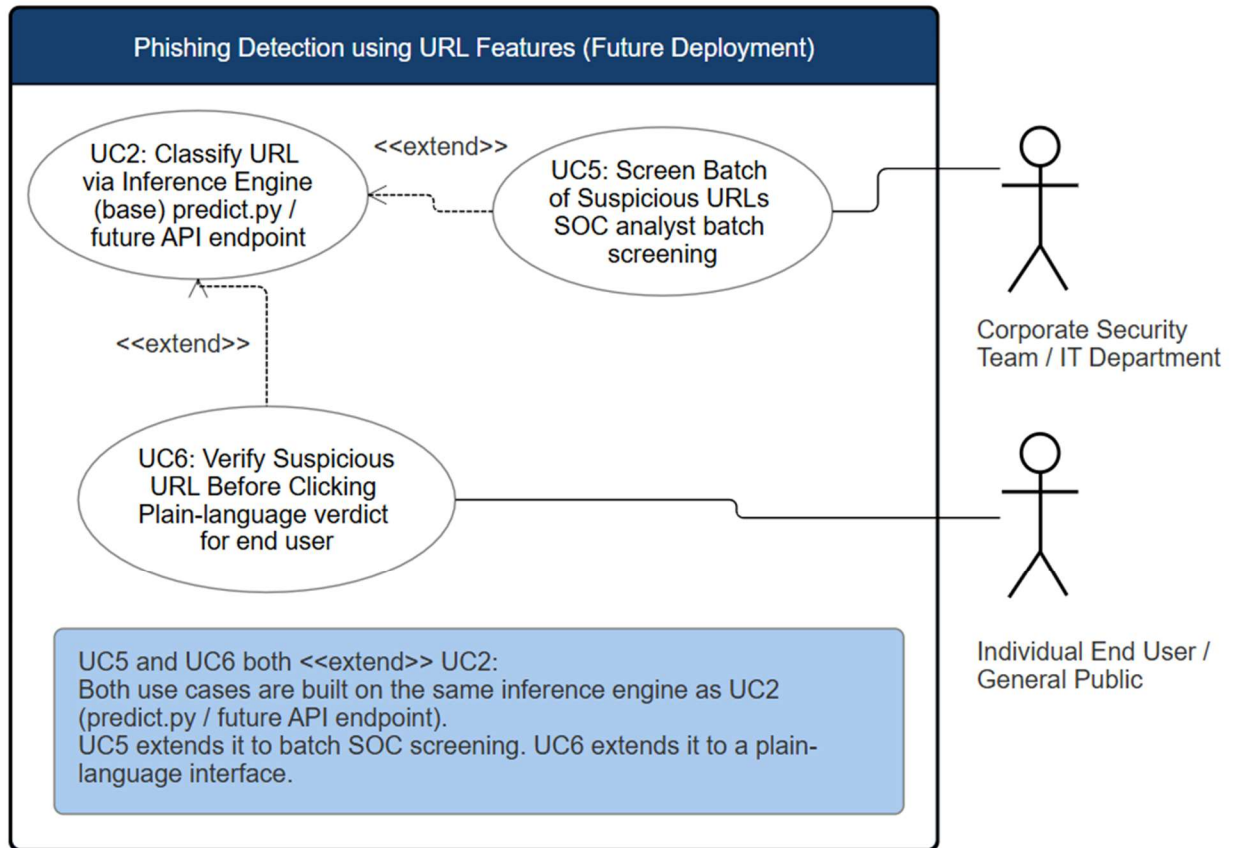


Figure A.2: UML Use Case Diagram- Future Deployment

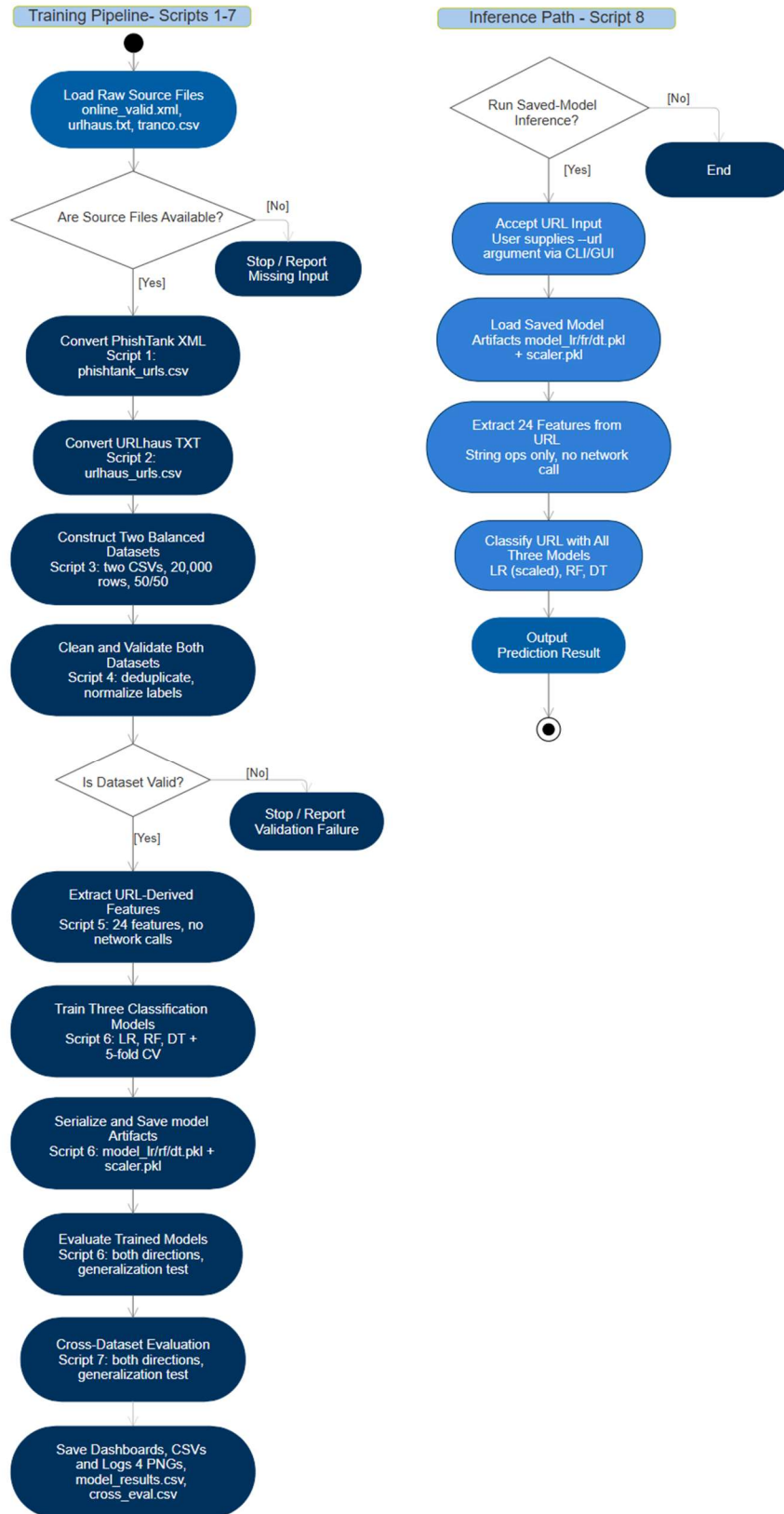


Figure A.3: UML Activity Diagram- Training Pipeline & Inference Path

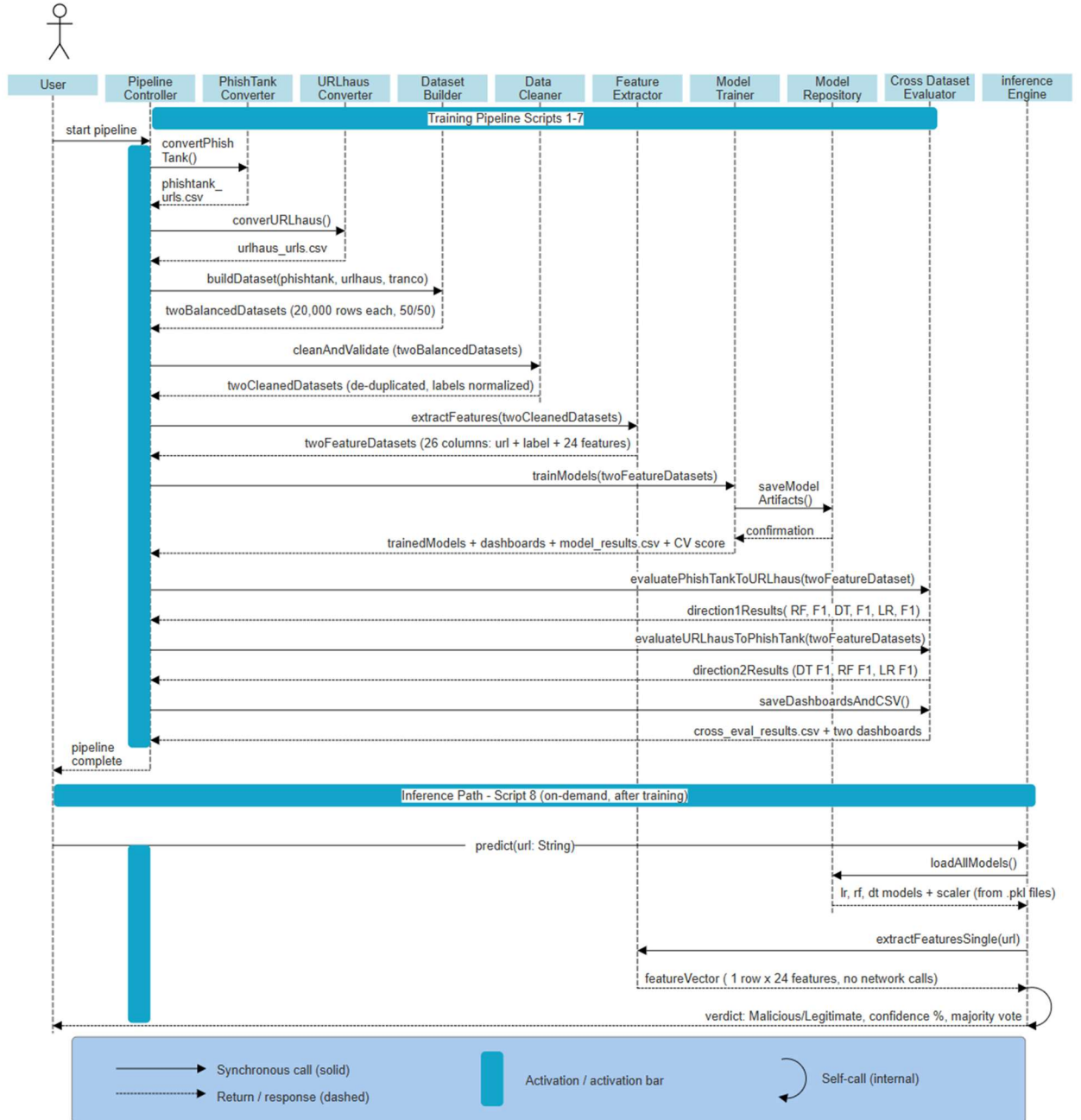


Figure A.4: UML Sequence Diagram- End-to-End Pipeline Interaction

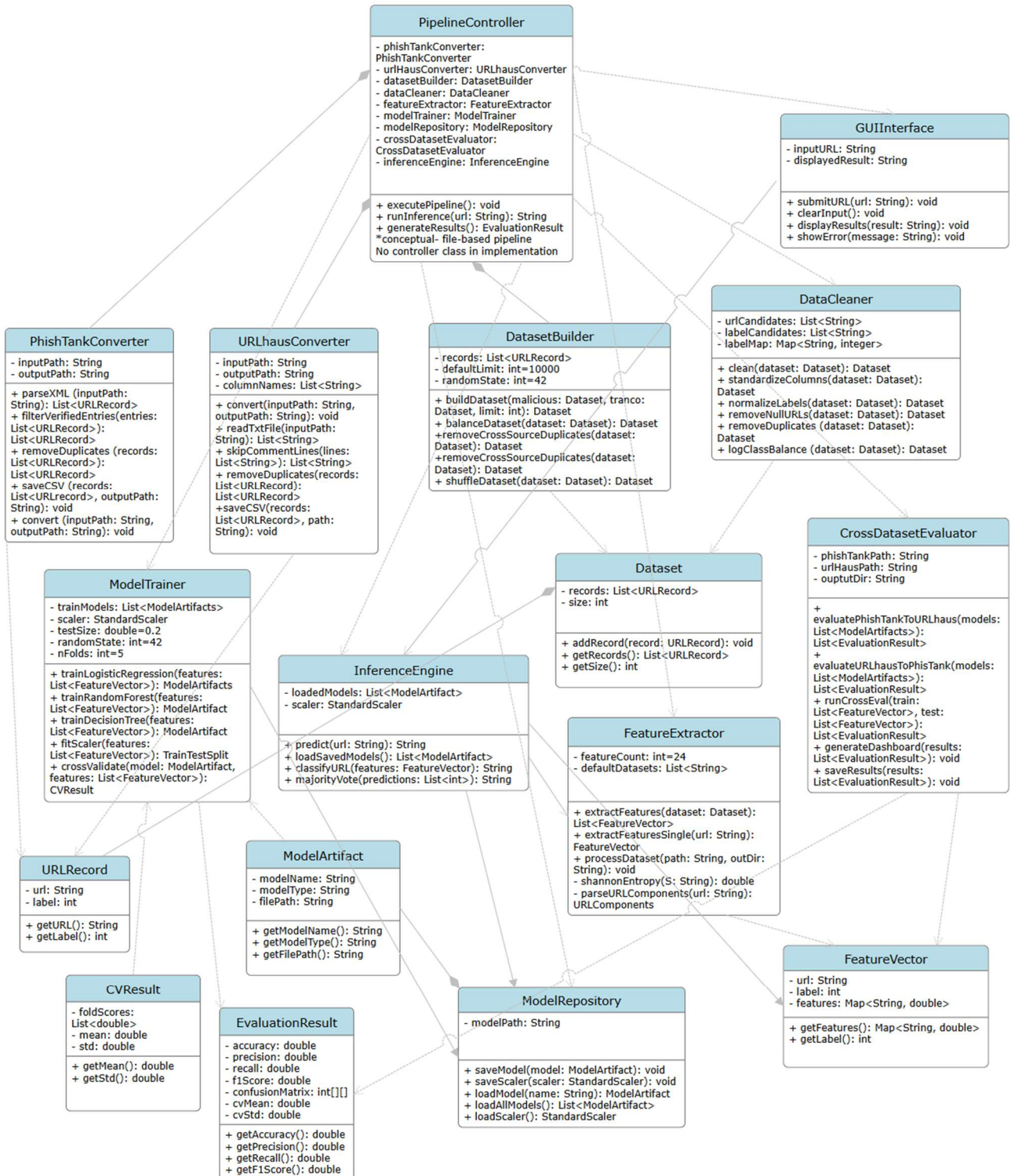


Figure A.5: UML Class Diagram- System Architecture

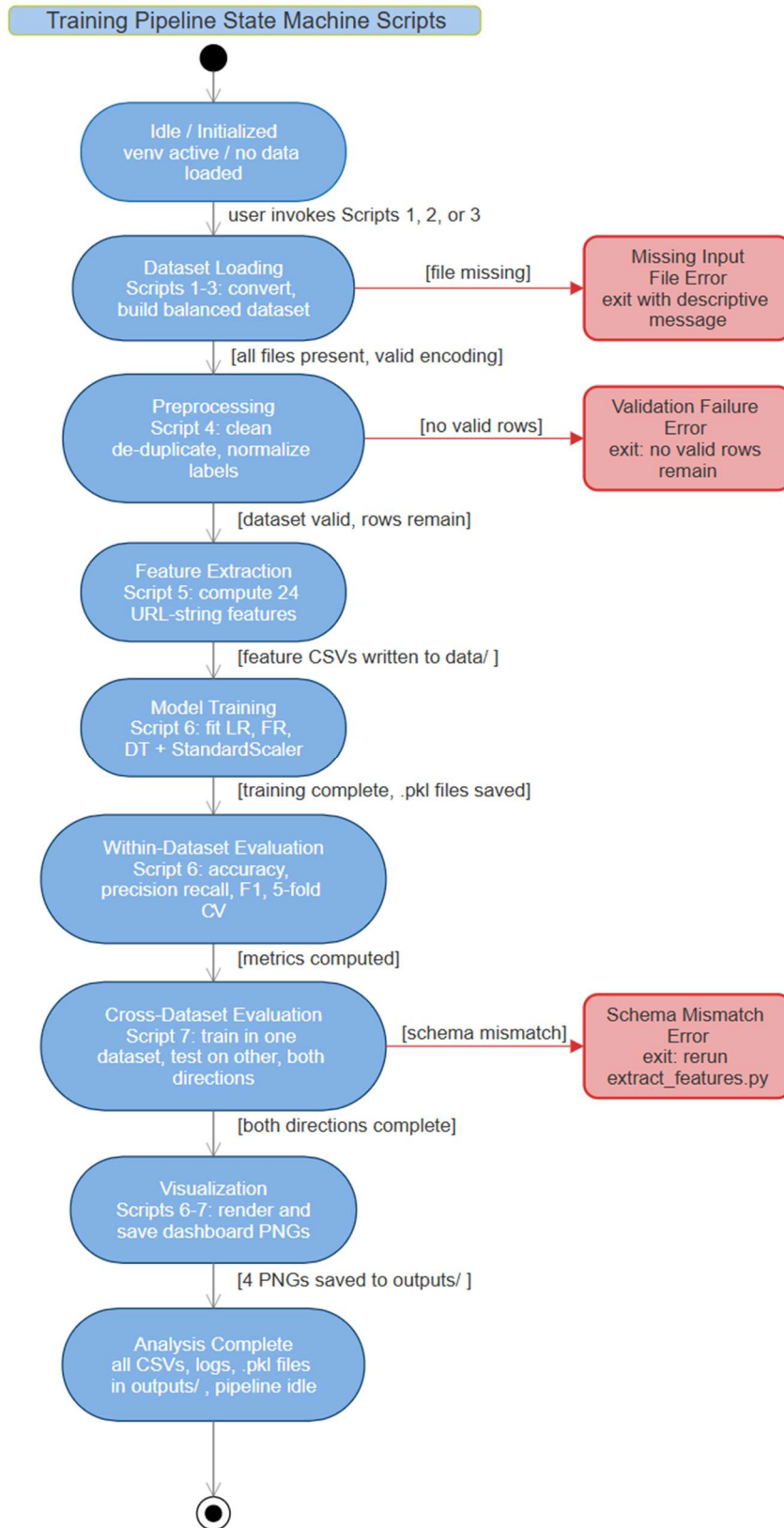


Figure A.6: UML State Diagram- Training Pipeline

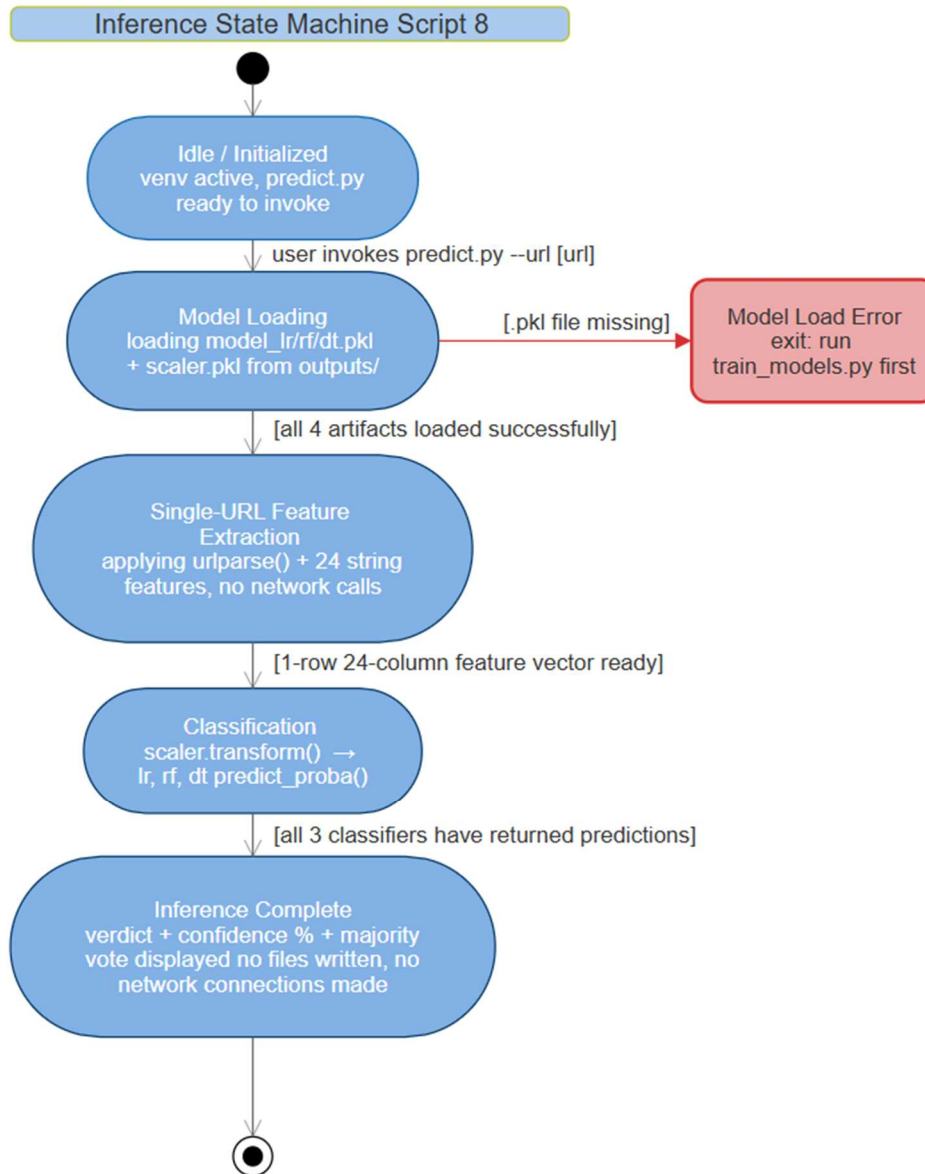


Figure A.7: UML State Diagram- Inference Path

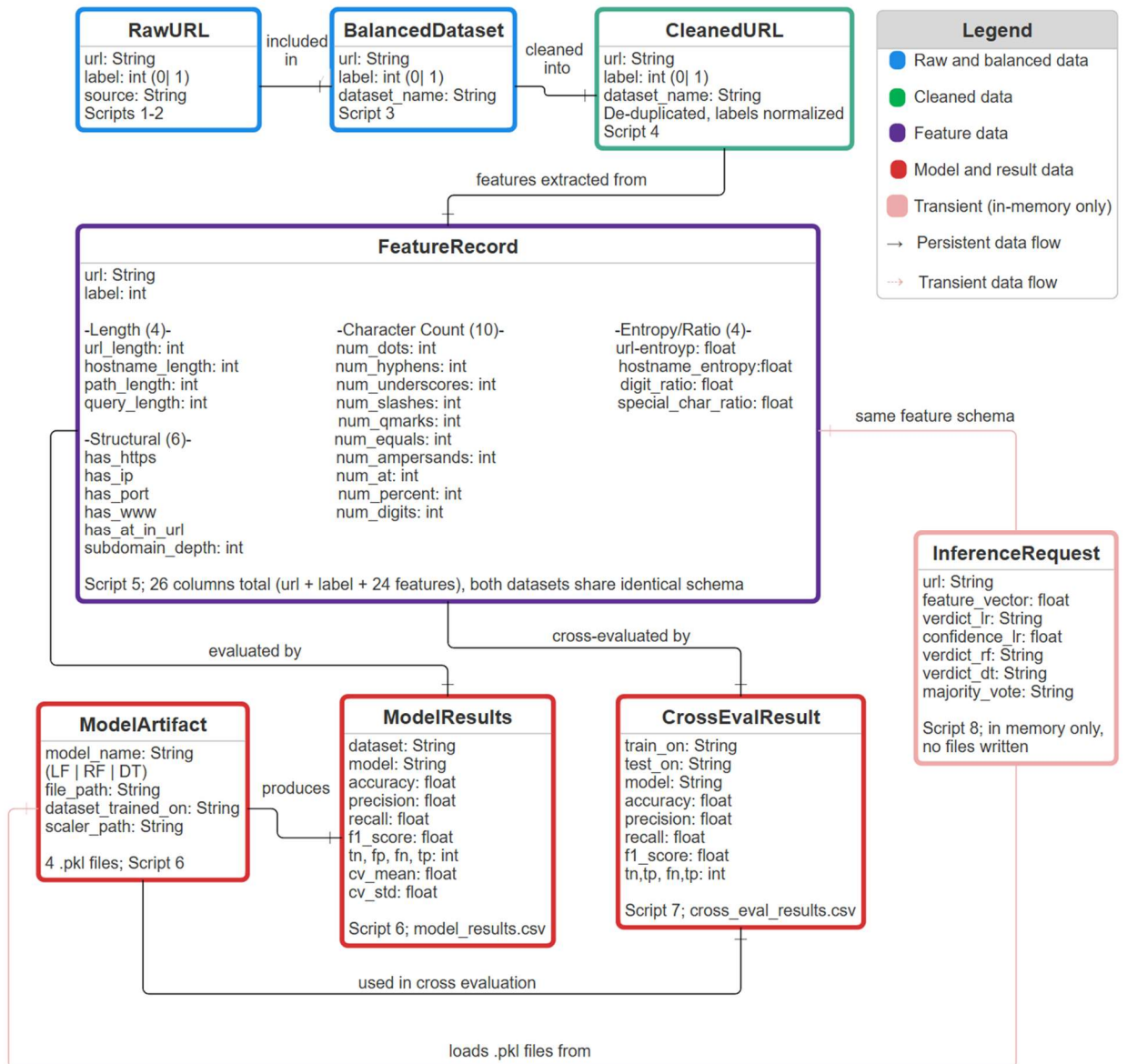


Figure A.8: Entity-Relationship Diagram- Pipeline Data Flow

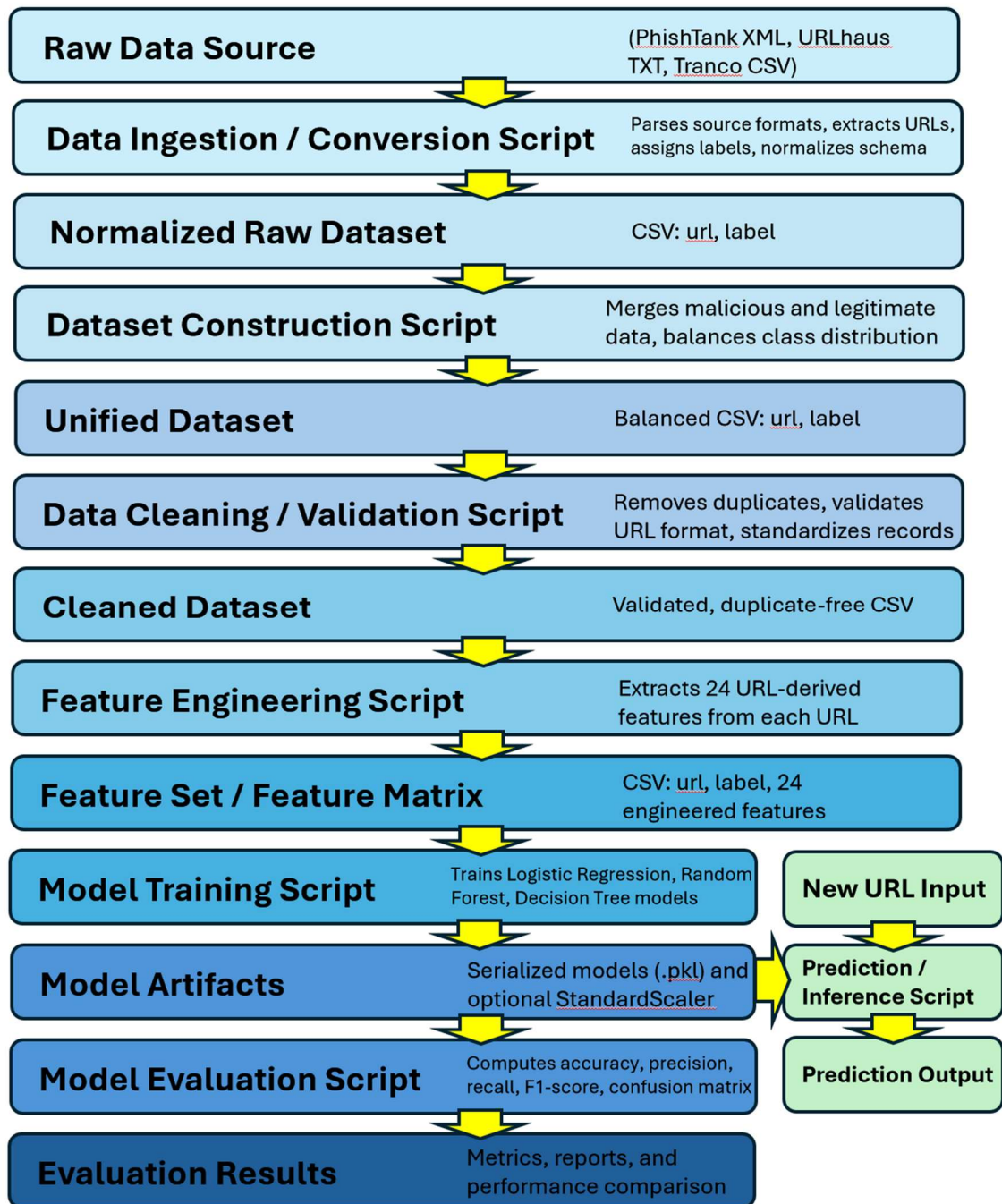


Figure A.9: End-to-End Pipeline Diagram- Phishing Detection System

Appendix B: Technical Specifications and Algorithmic Details

This appendix provides detailed technical information supporting the system design described in the main body of this document. It outlines the feature engineering methodology, machine learning models, data processing logic, evaluation approach, inference workflow, and validation mechanism used within the phishing detection pipeline. These specifications are intended to support reproducibility, transparency, and technical evaluation of the system.

B.1 Feature Engineering Specification

The system extracts a fixed set of 24 URL-string derived features for each input URL. These features are computed using string operations, pattern matching, and statistical measures. No external network queries are performed, ensuring that the system remains fully offline and reproducible.

Each feature is designed to capture structural characteristics commonly associated with malicious URLs.

The feature set described in this appendix matches the implementation in `extract_features.py` exactly and is identical to the schema documented in Section 6.2.3. The same 24 features are applied consistently across both training datasets and all inference inputs, ensuring schema compatibility and enabling valid cross-dataset evaluation. All missing values resulting from zero-length URL strings are resolved by substituting 0 via `pandas fillna(0)` after feature computation.

Feature Name	Description
url_length	Total number of characters in the full URL string, computed using Python string length on raw url field.
hostname_length	Number of characters in the hostname component of the URL, extracted using <code>urlparse()</code> .
path_length	Number of characters in the path component of the URL, extracted using <code>urlparse()</code> . Zero if no path is present.
query_length	Number of characters in the query string component of the URL, extracted using <code>urlparse()</code> . Zero if no query string is present.
num_dots	Count of period (.) characters in the full URL string.

num_hyphens	Count of hyphen (-) characters in the full URL string.
num_underscores	Count of underscore (_) characters in the full URL string.
num_slashes	Count of forward slash (/) characters in the full URL string.
num_qmarks	Count of question mark (?) characters in the full URL string.
num_equals	Count of equals sign (=) characters in the full URL string.
num_ampersands	Count of ampersand (&) characters in the full URL string.
num_at	Count of at symbol (@) characters in the full URL string.
num_percent	Count of percent sign (%) characters in the full URL string, indicating URL-encoded characters.
num_digits	Count of numeric digit character (0-9) in the full URL string.
has_https	Binary indicator (1/0). Value is 1 if the URL scheme is https, 0 otherwise. Computed from the scheme component returned by urlparse().
has_ip	Binary indicator (1/0). Value is 1 if the hostname matches the pattern of an IPv4 address (four dot-separated octets of 1-3 digits each), 0 otherwise. Phishing URLs frequently use raw IP addresses to bypass domain-based filtering.
has_port	Binary indicator (1/0). Value is 1 if the URL explicitly specifies a non-standard port number in the netloc component, 0 otherwise. Computed from the port attribute returned by urlparse().
has_www	Binary indicator (1/0). Value is 1 if the hostname begins with the prefix www., 0 otherwise.
has_at_in_url	Binary indicator (1/0). Value is 1 if the @ character appears anywhere in the full URL string. The @ character in a URL causes browsers to treat everything before it as credentials, potentially masking the true destination.
subdomain_depth	Integer count of subdomain levels present above the registered domain. Computed by

	splitting the hostname on period characters and subtracting 2 (for the registered domain and TLD). Returns 0 for hostnames with no subdomain. Minimum value is 0.
url_entropy	Shannon entropy of the full URL string computed as $H = -\sum p(x) \log_2$. Where $p(x)$ is the relative frequency of each character. Higher entropy values indicate greater character randomness, which is associated with algorithmically generated or obfuscated malicious URLs.
hostname_entropy	Shannon entropy of the hostname component only, computed using the same Shannon entropy formula. Isolates obfuscation patterns in the domain form those in the path or query string.
digit_ratio	Ratio of numeric digit characters to the total URL length, computed as $\text{num_digits} / \text{url_length}$. Represented as a float in the range $[0, 1]$. URLs with a high proportion of digits may indicate IP-based addresses or algorithmically generated domains. Zero-length URLs are assigned a value of 0 to avoid division errors.
special_char_ratio	Ratio of selected special characters to total URL length. The numerator is the sum of <code>num_dots</code> , <code>num_hyphens</code> , <code>num_underscores</code> , <code>num_at</code> , <code>num_percent</code> , <code>num_qmarks</code> , <code>num_equals</code> , and <code>num_ampersands</code> . Represented as a float in the range $[0, 1]$. Captures structural complexity and obfuscation patterns that entropy alone may not have isolated. Zero-length URLs are assigned a value 0 to avoid division errors.

The full feature set is applied consistently across both training datasets and inference inputs, ensuring schema compatibility and enabling valid cross-dataset evaluation.

B.2 Machine Learning Models

The system employs three supervised classification models:

- Logistic Regression
- Random Forest
- Decision Tree

These models were selected to balance interpretability, performance, and computational efficiency.

Logistic Regression

A linear model that estimates the probability of a URL belonging to the malicious class. It is highly interpretable and serves as a baseline classifier.

Random Forest

An ensemble learning method that constructs multiple decision trees and aggregates their predictions. It is resistant to overfitting and capable of capturing non-linear relationships in feature space.

Decision Tree

A rule-based classifier that partitions the feature space into decision regions. It provides clear, interpretable decision paths and serves as a comparative model.

All models are trained using a stratified 80/20 train-test split and evaluated using 5-fold cross-validation.

B.3 Data Processing and Validation

Data processing follows a structured pipeline to ensure consistency and data quality.

- Malicious URLs are sourced from PhishTank and URLhaus
- Legitimate URLs are sourced from the Tranco dataset
- Each dataset is balanced to achieve a 50/50 distribution of malicious and legitimate samples
- Duplicate URLs are removed using string-based matching
- Labels are normalized to binary values:
 - 1= malicious
 - 0= legitimate

Validation checks are applied at each stage to ensure:

- Required columns (url, label) are present
- Datasets are non-empty

- Schema consistency is maintained across datasets

This preprocessing ensures that all downstream components operate on clean, standardized data.

B.4 Cross-Dataset Evaluation Methodology

The system implements cross-dataset evaluation to assess model generalization across independently sourced datasets.

Two evaluation scenarios are performed:

1. Train on PhishTank + Tranco, test on URLhaus + Tranco
2. Train on URLhaus + Tranco, test on PhishTank + Tranco

This approach evaluates the ability of trained models to detect previously unseen phishing patterns originating from different data sources.

Feature schema consistency between datasets is strictly enforced to ensure valid comparison.

B.5 Inference Logic (predict.py)

The inference process operates independently of the training pipeline and is executed on demand.

The workflow is as follows:

1. The user provides a URL via the --url command-line argument.
2. Serialized model artifacts (.pkl files) and preprocessing components are loaded from the outputs/ directory.
3. The system applies the same 24-feature engineering logic used during training.
4. Each model produces a classification and associated confidence score.
5. A majority voting mechanism determines the final classification result.

The output includes

- Per-model classification (malicious/ legitimate)
- Confidence score
- Majority vote summary

No files are written during inference, and no external network communication is performed.

B.6 Error Handling and Validation

The system implements a fail-fast validation strategy to ensure reliability and reproducibility.

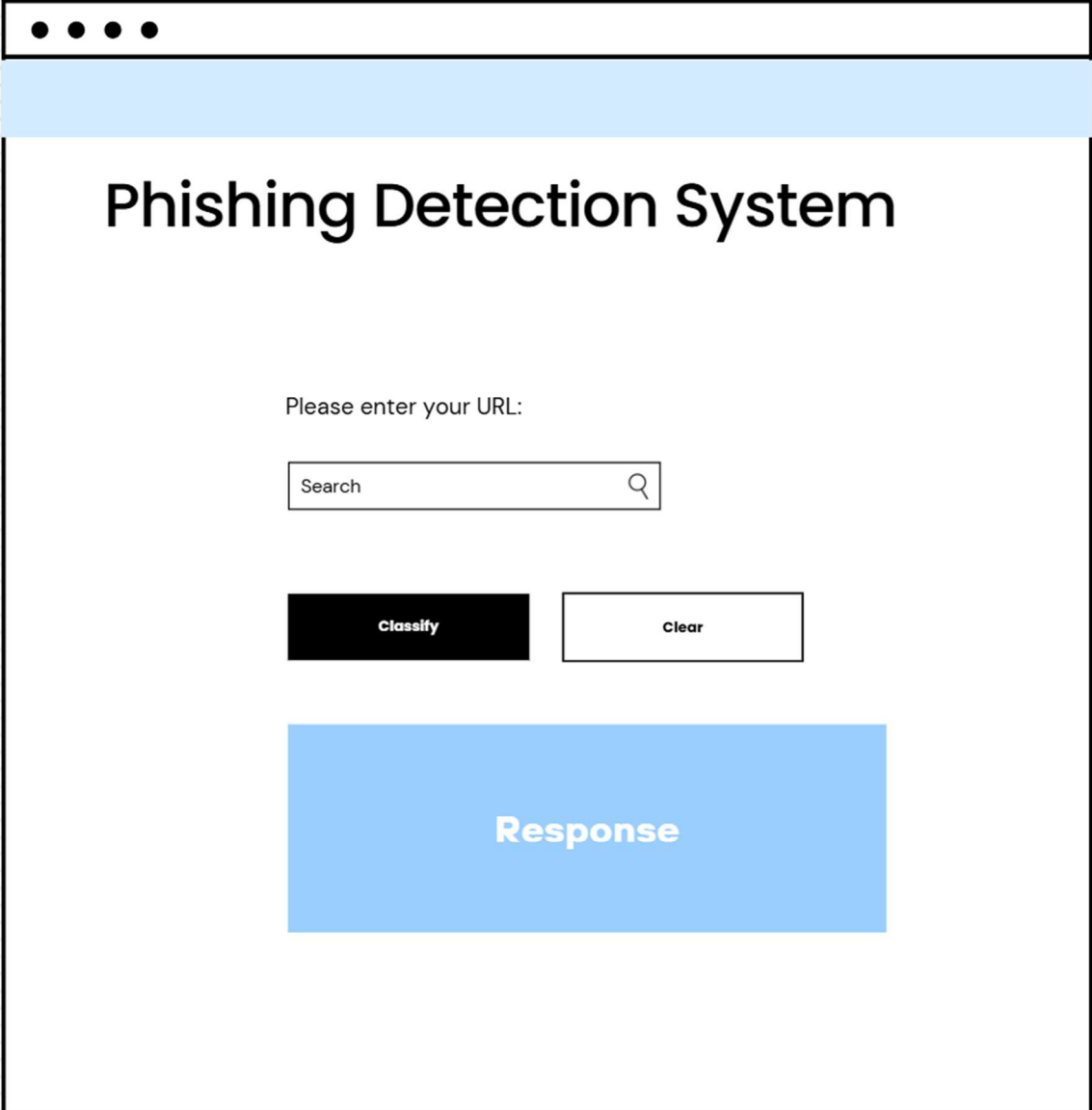
Error handling mechanisms include:

- **Missing Input Files**
Scripts determine with descriptive error messages identifying missing datasets and required download sources.
- **Schema Validation Failure**
Execution halts if required fields are missing or improperly formatted.
- **Model Load Failure**
The inference script verifies the presence of serialized model artifacts and instructs the user to execute the training pipeline if artifacts are absent.
- **Invalid Input Handling**
The inference process validates the presence and format of the input URL before execution.

These mechanisms prevent invalid states and ensure that errors are detected and reported early in the execution process.

Appendix C: User Interface Prototype

This appendix presents the user interface prototype illustrating the intended interaction and layout of the system.



The wireframe depicts a web browser window with a title bar containing four dots. Below the title bar is a light blue header bar. The main content area has a white background and features the title "Phishing Detection System" in a large, bold, black font. Below the title, the text "Please enter your URL:" is displayed. Underneath this text is a search input field with the placeholder text "Search" and a magnifying glass icon on the right. Below the search field are two buttons: a black button with the text "Classify" in white, and a white button with a black border and the text "Clear". Below these buttons is a large, light blue rectangular box with the word "Response" centered in white text.

Figure C.1: User Interface Wireframe- Future Web Interface

```
=====
PHISHING DETECTION PIPELINE — TRAINING EXECUTION LOG
=====

[INFO] Loading datasets...
  - PhishTank dataset loaded (10,000 entries)
  - Tranco dataset loaded (10,000 entries)

[INFO] Constructing balanced dataset...
  - Malicious: 10,000
  - Legitimate: 10,000

[INFO] Cleaning and validating data...
  - Duplicates removed: 1,234
  - Invalid entries removed: 56

[INFO] Extracting features...
  - Total features extracted: 24

[INFO] Splitting dataset (80/20 stratified)...

[INFO] Training models...
  - Logistic Regression: COMPLETE
  - Decision Tree: COMPLETE
  - Random Forest: COMPLETE

[INFO] Performing cross-validation (5-fold)...

[RESULTS]
-----
Model                Accuracy    Precision    Recall    F1
-----
Logistic Regression   0.99        0.99        0.99      0.99
Decision Tree         0.99        0.99        0.99      0.99
Random Forest         0.99        0.99        0.99      0.99
-----

[INFO] Saving model artifacts to /outputs/
[INFO] Training pipeline completed successfully
=====
```

Figure C.2: Training CLI Output Wireframe- Model Training Execution Log

```
=====
PHISHING DETECTION — INFERENCE RESULT
=====

Input URL:
http://secure-login-account-update.com/verify

[INFO] Extracting features...
[INFO] Loading trained models...

-----
Model                Prediction    Confidence
-----
Logistic Regression   Malicious    0.99
Decision Tree         Malicious    0.99
Random Forest         Malicious    0.99
-----

[FINAL RESULT]
Majority Vote: MALICIOUS

Confidence Summary:
- Average Confidence: 0.99
- Agreement: 3/3 models

=====
```

Figure C.3: Inference CLI Output Wireframe- URL Classification Result

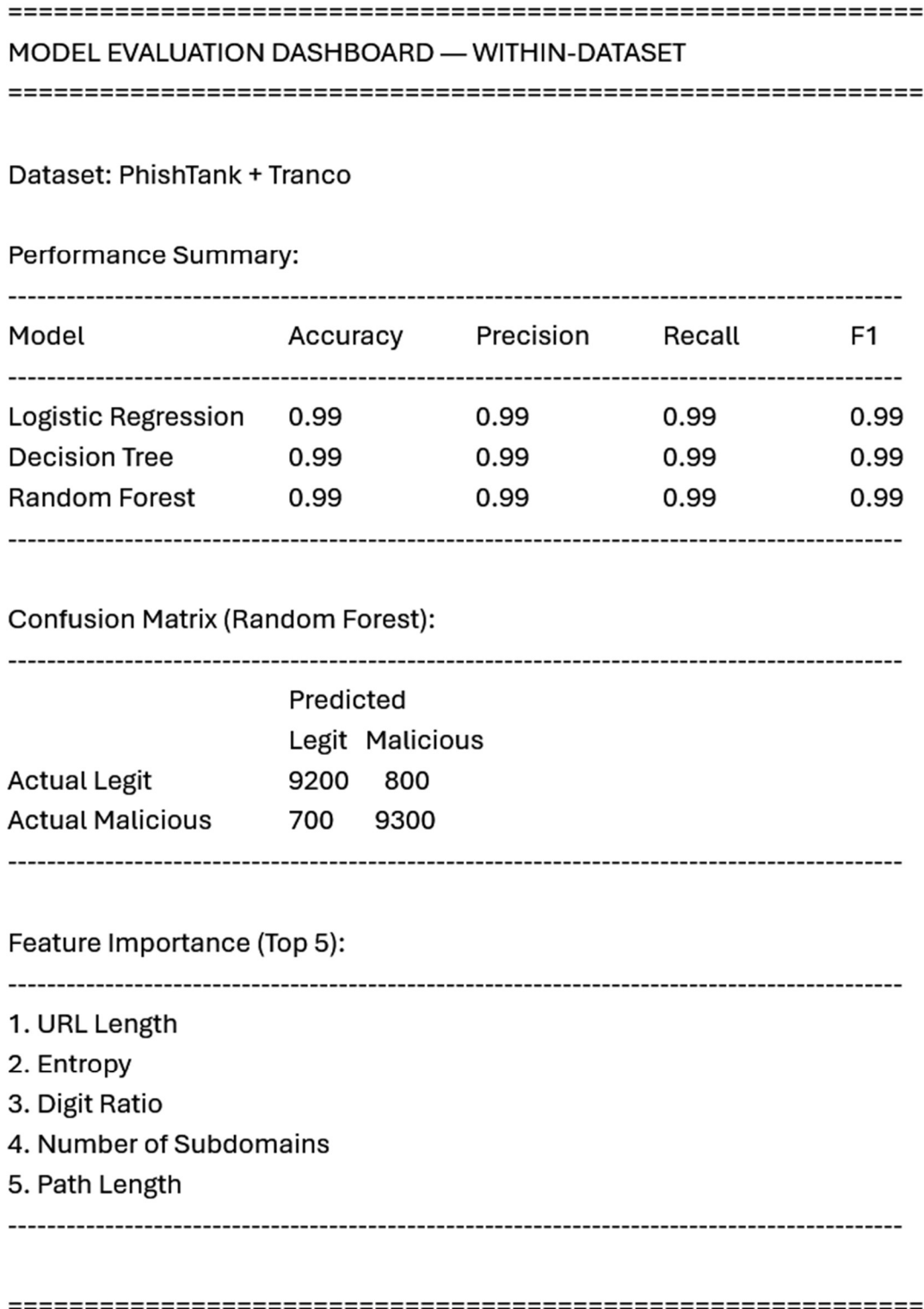


Figure C.4: Dashboard Output Wireframe- Within-Dataset Evaluation

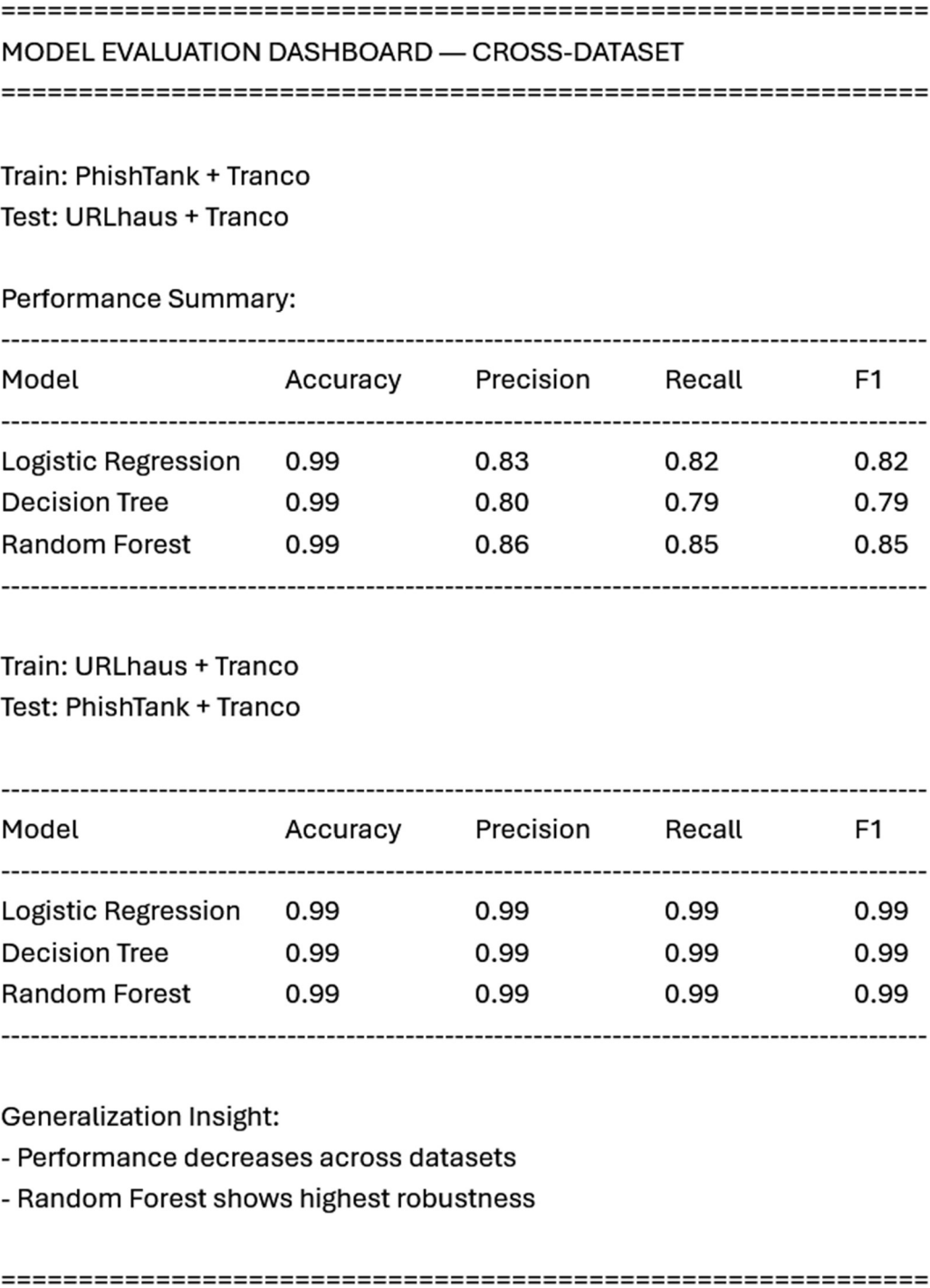


Figure C.5: Dashboard Output Wireframe- Cross-Dataset Evaluation

Figure D.1: Sample Raw Input- PhishTank XML Source File (online_validate.xml)


```
#####
# abuse.ch URLhaus Database Dump (CSV - recent URLs only) #
# Last updated: 2026-03-06 06:04:19 (UTC) #
# #
# Terms Of Use: https://urlhaus.abuse.ch/api/ #
# For questions please contact urlhaus [at] abuse.ch #
#####
# id,dateadded,url,url_status,last_online,threat,tags,urlhaus_link,reporter
"3790452","2026-03-06 06:04:19","http://115.55.35.35:41670/bin.sh","online","2026-03-06 06:04:19","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790452/","geenensp"
"3790451","2026-03-06 06:04:08","https://stre06.aquastream.in.net/verification.google","online","2026-03-06 06:04:08","malware_download","AmateraStealer,ClearFake","https://urlhaus.abuse.ch/url/3790451/","anonymous"
"3790450","2026-03-06 05:57:23","http://39.74.33.196:54757/i","online","2026-03-06 05:57:23","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790450/","geenensp"
"3790449","2026-03-06 05:57:22","http://42.238.142.216:44279/i","online","2026-03-06 05:57:22","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790449/","geenensp"
"3790448","2026-03-06 05:52:19","http://42.238.142.216:44279/bin.sh","online","2026-03-06 05:52:19","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790448/","geenensp"
"3790447","2026-03-06 05:52:08","https://f70zu1t9.flogginguisit.digital/?=check&&actmn=YVIBWnnTogWtApp","offline","","malware_download","ClearFake","https://urlhaus.abuse.ch/url/3790447/","anonymous"
"3790446","2026-03-06 05:50:19","http://42.239.171.246:40751/i","online","2026-03-06 05:50:19","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790446/","geenensp"
"3790445","2026-03-06 05:44:22","http://113.236.150.202:60629/bin.sh","online","2026-03-06 05:44:22","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790445/","geenensp"
"3790444","2026-03-06 05:43:36","http://112.248.160.26:34841/i","online","2026-03-06 05:43:36","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790444/","geenensp"
"3790443","2026-03-06 05:42:16","http://115.56.156.197:49040/bin.sh","online","2026-03-06 05:42:16","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790443/","geenensp"
"3790442","2026-03-06 05:40:16","http://182.115.186.24:50330/i","online","2026-03-06 05:40:16","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790442/","geenensp"
"3790441","2026-03-06 05:36:17","http://117.209.92.160:36673/bin.sh","online","2026-03-06 05:36:17","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790441/","geenensp"
"3790440","2026-03-06 05:33:18","http://110.36.80.163:33806/i","online","2026-03-06 05:33:18","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790440/","geenensp"
"3790439","2026-03-06 05:30:25","http://182.113.36.132:51770/i","online","2026-03-06 05:30:25","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790439/","geenensp"
"3790438","2026-03-06 05:29:18","http://182.116.115.167:52769/i","online","2026-03-06 05:29:18","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790438/","geenensp"
"3790437","2026-03-06 05:26:16","http://110.36.80.163:33806/bin.sh","online","2026-03-06 05:26:16","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790437/","geenensp"
"3790436","2026-03-06 05:25:20","http://123.8.5.188:46066/i","online","2026-03-06 05:25:20","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790436/","geenensp"
"3790435","2026-03-06 05:24:18","http://ip-2422.proline.net.ua:45514/Mozi.m","online","2026-03-06 05:24:18","malware_download","arm,elf,geofenced,mirai,Mozi,ua-wget,USA","https://urlhaus.abuse.ch/url/3790435/","botnetkiller"
"3790434","2026-03-06 05:24:09","http://ip-2422.proline.net.ua:45514/bin.sh","online","2026-03-06 05:24:09","malware_download","arm,elf,geofenced,mirai,Mozi,ua-wget,USA","https://urlhaus.abuse.ch/url/3790434/","botnetkiller"
"3790433","2026-03-06 05:24:09","http://ip-2422.proline.net.ua:45514/Mozi.m","online","2026-03-06 05:24:09","malware_download","arm,elf,geofenced,mirai,Mozi,ua-wget,USA","https://urlhaus.abuse.ch/url/3790433/","botnetkiller"
"3790432","2026-03-06 05:24:08","http://94.244.36.34:45514/Mozi.a","online","2026-03-06 05:24:08","malware_download","arm,elf,geofenced,mirai,Mozi,ua-wget,USA","https://urlhaus.abuse.ch/url/3790432/","botnetkiller"
"3790431","2026-03-06 05:24:08","https://aqua-sync.aquastream.in.net/verification.google","online","2026-03-06 05:24:08","malware_download","AmateraStealer,ClearFake","https://urlhaus.abuse.ch/url/3790431/","anonymous"
"3790430","2026-03-06 05:24:08","http://ip-2422.proline.net.ua:45514/bin.sh","online","2026-03-06 05:24:08","malware_download","arm,elf,geofenced,mirai,Mozi,ua-wget,USA","https://urlhaus.abuse.ch/url/3790430/","botnetkiller"
"3790429","2026-03-06 05:18:27","http://125.42.33.186:60716/i","offline","","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790429/","geenensp"
"3790428","2026-03-06 05:17:10","http://125.42.33.186:60716/bin.sh","online","2026-03-06 05:17:10","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790428/","geenensp"
"3790427","2026-03-06 05:15:11","http://117.209.87.184:52274/bin.sh","online","2026-03-06 05:15:11","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790427/","geenensp"
"3790426","2026-03-06 05:08:17","http://182.116.115.167:52769/bin.sh","online","2026-03-06 05:08:17","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790426/","geenensp"
"3790425","2026-03-06 05:07:20","http://42.239.171.246:40751/bin.sh","online","2026-03-06 05:07:20","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790425/","geenensp"
"3790424","2026-03-06 05:05:13","http://123.8.5.188:46066/bin.sh","online","2026-03-06 05:05:13","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790424/","geenensp"
"3790423","2026-03-06 05:05:07","https://stre-v9.aquastream.in.net/verification.google","online","2026-03-06 05:05:07","malware_download","AmateraStealer,ClearFake","https://urlhaus.abuse.ch/url/3790423/","anonymous"
"3790422","2026-03-06 05:04:24","http://190.55.22.176:45197/bin.sh","online","2026-03-06 05:04:24","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790422/","geenensp"
"3790421","2026-03-06 05:02:23","http://190.55.22.176:45197/i","online","2026-03-06 05:02:23","malware_download","32-bit,elf,mips,Mozi","https://urlhaus.abuse.ch/url/3790421/","geenensp"
"3790420","2026-03-06 04:59:07","http://110.37.103.207:34864/i","online","2026-03-06 04:59:07","malware_download","32-bit,arm,elf,mirai,Mozi","https://urlhaus.abuse.ch/url/3790420/","geenensp"
"3790419","2026-03-06 04:58:18","http://123.8.51.229:34208/i","online","2026-03-06 04:58:18","malware_download","32-
```

Figure D.2: Sample Raw Input- URLhaus Plain-Text Feed (urlhaus_abuse_ch.txt)

AutoSave Off tranco_top_sites.csv

File Home Insert Draw Page Layout Formulas Data Rev

Paste Clipboard Font Alignme

M13

	A	B	C	D	E	F
1	1	google.com				
2	2	microsoft.com				
3	3	gtld-servers.net				
4	4	facebook.com				
5	5	mail.ru				
6	6	googleapis.com				
7	7	cloudflare.com				
8	8	apple.com				
9	9	youtube.com				
10	10	amazonaws.com				
11	11	gstatic.com				
12	12	dzen.ru				
13	13	instagram.com				
14	14	akamai.net				
15	15	office.com				
16	16	fbcdn.net				
17	17	twitter.com				
18	18	linkedin.com				
19	19	googlevideo.com				
20	20	live.com				
21	21	googletagmanager.com				
22	22	akamaiedge.net				
23	23	amazon.com				
24	24	akadns.net				
25	25	azure.com				
26	26	wikipedia.org				
27	27	windowsupdate.com				
28	28	microsoftonline.com				
29	29	github.com				
30	30	e2ro.com				
31	31	googleusercontent.com				
32	32	doubleclick.net				
33	33	bing.com				
34	34	workers.dev				
35	35	netflix.com				
36	36	domaincontrol.com				
37	37	fastly.net				

Figure D.3: Sample Raw Input- Tranco Top Sites List (tranco_top_sites.csv)

	A	B
1	url	label
2	https://allegrolokalnie.ogloszenia-popularne-17255.sbs	1
3	https://allegrolokalnie.ogloszenia-popularne-18257.sbs	1
4	https://allegrolokalnie.ogloszenia-popularne-19251.sbs	1
5	https://allegrolokalnie.ogloszenia-popularne-16253.sbs	1
6	http://trzor-io-start-us-en.created.app	1
7	https://trzor-io-start-us-en.created.app/	1
8	https://kucoincom-login.webflow.io/	1
9	https://treezurioosattrat.webflow.io/	1
10	https://outl00-ec-web.webcindario.com/	1
11	https://beneonline.webcindario.com/	1
12	http://allegrolokalnie.pl-oferta-54882394.sbs/	1
13	http://allegro.7960703-oferta-sprzedam-macbook-air-m1.sbs/	1
14	http://allegro.885294026-oferta-ipad-pro-m5-13-wifi-cellular.cfd/	1
15	https://allegrolokalnie.7960703-oferta-sprzedam-macbook-air-m1.sbs/	1
16	http://allegrolokalnie.7960703-oferta-sprzedam-macbook-air-m1.sbs/	1
17	https://allegrolokalnie.7960703-oferta-sprzedam-macbook-air-m1.sbs	1
18	https://sites.google.com/pinkertonacademy.org/documentation-for-your-review?usp=sharing	1
19	http://allegrolokalnie.pl-oferta61752791.cfd	1
20	http://allegrolokalnie.pl-oferta593850.click	1
21	http://allegrolokalnie.pl-kupteraz88328192.online	1
22	http://allegro.pl-oferta61752791.cfd	1
23	https://allegrolokalnie.oferta074813.cyou	1
24	http://allegrolokalnie.pl-oferta67332435.cfd	1
25	http://allegro.pl-kupteraz88328192.online	1
26	http://allegro.pl-oferta593850.click	1
27	https://elster-online-portal-de.com/	1
28	https://storage.googleapis.com/symmd9ce92qgl32gteimei/a4kupphe7rzz9ih9ide5p43c_s1ulajynjqoqbkd_9jq95ed.html	1
29	https://elster-online-portal-de.com/konto/	1
30	http://allegro.885294026-oferta-ipad-pro-m5-13-wifi-cellular.cfd	1
31	http://allegrolokalnie.pl-ogloszenie298532.sbs	1
32	https://allegro.pl-9272637oferta927363783zamowienie827273.biz	1
33	https://allegro.oferta074813.cyou	1
34	https://new.express.adobe.com/webpage/E9lDLe8tKruFv	1
35	https://shopy.com.pk/ssy/web/ali/	1
36	http://allegrolokalnie.pl-oferta7960703-sprzedam-macbook-air-m1.sbs	1
37	http://allegrolokalnie.pl-oferta2318251.click	1

AutoSave Off urlhaus_urls.csv Search

File Home Insert Draw Page Layout Formulas Data Review View Automate

Paste Clipboard Font Alignment Number

L19

	A	B	C
1	url	label	
2	http://115.55.35.35:41670/bin.sh	1	
3	https://stre06.aquastream.in.net/verification.google	1	
4	http://39.74.33.196:54757/i	1	
5	http://42.238.142.216:44279/i	1	
6	http://42.238.142.216:44279/bin.sh	1	
7	https://f70zu1t9.flogginquisit.digital/?=check&&actmn=YVIBwWnnTOgWtApP	1	
8	http://42.239.171.246:40751/i	1	
9	http://113.236.150.202:60629/bin.sh	1	
10	http://112.248.160.26:34841/i	1	
11	http://115.56.156.197:49040/bin.sh	1	
12	http://182.115.186.24:50330/i	1	
13	http://117.209.92.160:36673/bin.sh	1	
14	http://110.36.80.163:33806/i	1	
15	http://182.113.36.132:51770/i	1	
16	http://182.116.115.167:52769/i	1	
17	http://110.36.80.163:33806/bin.sh	1	
18	http://123.8.5.188:46066/i	1	
19	http://ip-2422.proline.net.ua:45514/Mozi.m	1	
20	http://ip-2422.proline.net.ua:45514/bin.sh	1	
21	http://ip-2422.proline.net.ua:45514/i	1	
22	http://94.244.36.34:45514/Mozi.m	1	
23	https://aqua-sync.aquastream.in.net/verification.google	1	
24	http://ip-2422.proline.net.ua:45514/Mozi.a	1	
25	http://125.42.33.186:60716/i	1	
26	http://125.42.33.186:60716/bin.sh	1	
27	http://117.209.87.184:52274/bin.sh	1	
28	http://182.116.115.167:52769/bin.sh	1	
29	http://42.239.171.246:40751/bin.sh	1	
30	http://123.8.5.188:46066/bin.sh	1	
31	https://stre-v9.aquastream.in.net/verification.google	1	
32	http://190.55.22.176:45197/bin.sh	1	
33	http://190.55.22.176:45197/i	1	
34	http://110.37.103.207:34864/i	1	
35	http://123.8.51.229:34208/i	1	
36	http://110.37.3.251:42121/i	1	
37	http://110.37.103.207:34864/bin.sh	1	

Figure D.5: Sample Converter Output- URLhaus URLs CSV (urlhaus_urls.csv)

	A	B
1	url	label
2	https://hicloud.com	0
3	https://cntrlsegitybank.webcindario.com/	1
4	https://pub-688274cc7f6e42babc80554f317de9ee.r2.dev/mail-ac-cgi-bin-storage-support.htm	1
5	https://docs.google.com/presentation/d/e/2PACX-1vS8LFaO6_heecApAWH3h6RGoLUXw2JdkrmCe9GWX_k6qbZgMUUBvF9T6zTyuulXATOi03PveHygcD_/pub?start=false&loop=false&delays=3000	1
6	https://aws.com	0
7	https://nic.media	0
8	https://cloudsink.net	0
9	https://sites.google.com/view/mmmmmmmmmmm55/home	1
10	https://olark.com	0
11	https://docs.google.com/presentation/d/e/2PACX-1vRe29ZjKo8IBIfQikH7xZtTe13_FD8wHQRmSAhKcnhOV-hKKWgQsfdB7UNLzT4nGh9F3eJv1Sr0jw/pub?start=false&loop=false&delays=3000&pli=1	1
12	https://qrco.de/bgCXP	1
13	https://urxecc.webwave.dev/?forcePageWithoutCdn=true	1
14	https://neopets.com	0
15	https://yektanet.com	0
16	https://p-msedge.net	0
17	https://prezo.ai/public/new-prezo--7c893d66-5e25-4092-ae97-755cef6c982a	1
18	https://mainroll.com	0
19	https://warframe.com	0
20	https://rp.mockplus.com/run/TrQynx-RLJ/irffOPqPZP?cps=hide&rps=hide&nav=1&ha=0&la=0&fc=1&out=1&t=1&isFrame=true&moduleID=P_Tgjhvdd	1
21	https://bit.ly/3X5msLV	1

Figure D.8: Sample Cleaned Dataset Output- PhishTank + Tranco Dataset (cleaned_phishtank_tranco_dataset.csv)

	A	B	C
1	url	label	
2	https://hicloud.com	0	
3	http://124.94.81.59:48696/i	1	
4	http://57.131.18.205:8085/via.wsh	1	
5	http://192.109.200.5/webb/encryptedwe.js	1	
6	https://aws.com	0	
7	https://nic.media	0	
8	https://cloudsink.net	0	
9	http://110.36.86.0:43314/i	1	
10	https://olark.com	0	
11	http://182.116.50.175:50779/i	1	
12	http://220.202.91.49:55110/i	1	
13	http://110.36.65.9:34750/bin.sh	1	
14	https://neopets.com	0	
15	https://yektanet.com	0	
16	https://p-msedge.net	0	
17	http://84920433.duckdns.org/00101010101001011010101110101010101010110101011/nwfaiehg4ewijfgrieghgirehaughrarg.spc	1	
18	https://mainroll.com	0	
19	https://warframe.com	0	
20	https://softbreeze.gentlewind.coupons/air_quality	1	
21	http://123.129.133.112:42211/i	1	
22	http://vmi3034867.contaboserver.net/ambatukam/rizzx.ppc	1	

Figure D.9: Sample Cleaned Dataset Output- URLhaus + Tranco Dataset (cleaned_urlhaus_tranco_dataset.csv)

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	aa	r	f
url	label	url_length	hostname	path_len	query_len	num_domains	num_https	num_unde	num_slash	num_qmark	num_equa	num_amp	num_at	num_percr	num_digits	has_https	has_ip	has_port	has_wwww	has_at_in_subdomain	url_entropy	hostnme_digiti	splatio	ratio_splati	char_aa	r	f		
https://hicloud.com	0	19	11	0	0	1	0	0	2	0	0	0	0	0	0	1	0	0	0	0	3.721613	0.0957952	0	0.052632					
https://ctrlroseighty	1	41	32	1	0	2	0	0	3	0	0	0	0	0	0	0	1	0	0	0	1.4268882	0.4306390	0	0.04878					
https://pub-68627d4	1	87	43	36	0	3	6	0	3	0	0	0	0	0	0	19	1	0	0	0	1.4826363	0.431286	0.218390	0.103482	7586	206896			
https://docs.google	1	167	15	108	35	2	1	3	7	1	3	2	0	0	17	1	0	0	0	0	1.5652986	0.2735726	0.1017964	0.071856					
https://aws.com	0	15	7	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0.5586905	0.9073546	0	0.08667					
https://nic.mie	1	17	9	0	0	1	0	0	2	0	0	0	0	0	0	0	1	0	0	0	0.5734521	0.2347703	0	0.058823294	11764705				
https://cloudsink.ne	0	21	13	0	0	1	0	0	2	0	0	0	0	0	0	0	1	0	0	0	0.38801796	0.3465935	0	0.0476190476	19047616				
https://sites.google	1	49	16	25	0	2	0	0	5	0	0	0	0	0	0	0	0	0	0	0	1.35627343	0.2028195	0.040816	0.040816					
https://olarik.com	0	17	9	0	0	1	0	0	2	0	0	0	0	0	0	1	0	0	0	0	0.7345216	0.2347703	0	0.058823294	11764705				
https://docs.google	1	185	15	108	52	2	1	7	1	0	0	0	0	0	0	20	1	0	0	0	1.5473452	0.2735726	0.1081081	0.0710277					
https://ipco.de/virt/g	1	22	7	7	0	1	0	0	3	0	0	0	0	0	0	0	0	0	0	0	0.0614821	0.8073546	0	0.045454545	45454556				
https://urxccc.webtv	1	52	18	1	24	2	0	0	3	1	1	0	0	0	0	0	0	0	0	0	1.46427263	0.2810361	0	0.078923					
https://neopets.com	0	19	11	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0.34713543	0.0957952	0	0.052632					
https://lykattantef.co	0	20	12	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0.73192633	0.2518291	0	0.05					
https://jp-nedisp.jp	0	20	12	0	0	1	1	0	2	0	0	0	0	0	0	0	0	0	0	0	0.54643953	0.1887218	0	0.01					
https://prezo.air/pub	1	71	8	55	0	1	7	4	0	4	0	0	0	0	0	22	1	0	0	0	0.6544227	0.330895915	0.112676063	0.3802817					
https://mainrol.cor	0	20	12	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0.82192633	0.0849625	0	0.05					
https://warframe.co	0	20	12	0	0	1	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0.82192633	0.0849625	0	0.05					
https://rp-mockjugs	1	131	15	26	81	2	1	1	5																				

Figure D.10: Sample Feature Record- PhishTank Extracted 24 URL-String Features (features_phishtank_tranco_dataset.csv)

[illegible]

Figure D.11: Sample Feature Record- URLhaus Extracted 24 URL-String Features (features_urlhaus_tranco_dataset.csv)

```
training_results_20260321_161811.t × +
File Edit View H1 ≡ B I S ↺ ↻ ⌂
16:18:11 [INFO] Results will be saved -> C:\Users\Connie's Build\phishing-detection\PhishingDe
20260321_161811.txt
16:18:11 [INFO] Loading: data\features_phishtank_tranco_dataset.csv
16:18:11 [INFO] 20000 rows, 24 features | malicious: 10000 (50.0%) legitimate: 10000 (50.0%)

=====
DATASET: features_phishtank_tranco_dataset
Train : 16000 rows | Test: 4000 rows
Features: 24
=====

--- Logistic Regression ---
Accuracy : 0.9972
Precision : 0.9990
Recall : 0.9955
F1-score : 0.9972
      precision    recall  f1-score   support

Legitimate      1.00      1.00      1.00      2000
Malicious       1.00      1.00      1.00      2000

   accuracy
macro avg      1.00      1.00      1.00      4000
weighted avg    1.00      1.00      1.00      4000

--- Random Forest ---
Accuracy : 0.9975
Precision : 0.9980
Recall : 0.9970
F1-score : 0.9975
      precision    recall  f1-score   support

Legitimate      1.00      1.00      1.00      2000
Malicious       1.00      1.00      1.00      2000

   accuracy
macro avg      1.00      1.00      1.00      4000
weighted avg    1.00      1.00      1.00      4000

Top 10 Feature Importances (Random Forest):
path_length      0.3144
num_slashes      0.2776
```

Figure D.12: Training Results- Terminal Screenshot
(training_results_YYYYMMDD_HHMMSS.txt)

	A	B	C	D	E	F	G	H	I	J
1	dataset	model	accuracy	precision	recall	f1_score	tn	fp	fn	tp
2	features_phishtank_tranco_dataset	Logistic Regression	0.9972	0.999	0.9955	0.9972	1998	2	9	1991
3	features_phishtank_tranco_dataset	Random Forest	0.9975	0.998	0.997	0.9975	1996	4	6	1994
4	features_phishtank_tranco_dataset	Decision Tree (depth=10)	0.998	0.9985	0.9975	0.998	1997	3	5	1995
5	features_urlhaus_tranco_dataset	Logistic Regression	0.9998	1	0.9995	0.9997	2000	0	1	1999
6	features_urlhaus_tranco_dataset	Random Forest	0.9998	1	0.9995	0.9997	2000	0	1	1999
7	features_urlhaus_tranco_dataset	Decision Tree (depth=10)	1	1	1	1	2000	0	0	2000
8										

Figure D.13: Training Results- Within Dataset Model Performance Metrics (model_results.csv)

Feature	Importance
path_length	0.3144
num_slashes	0.2776
url_entropy	0.1005
url_length	0.0684
num_dots	0.0550
subdomain_depth	0.0433
hostname_length	0.0323
num_digits	0.0290
hostname_entropy	0.0221
digit_ratio	0.0216

Figure D.14: Training Results- Top Feature Importances, Random Forest Both Datasets (training_results_YYYYMMDD_HHMMSS.txt)

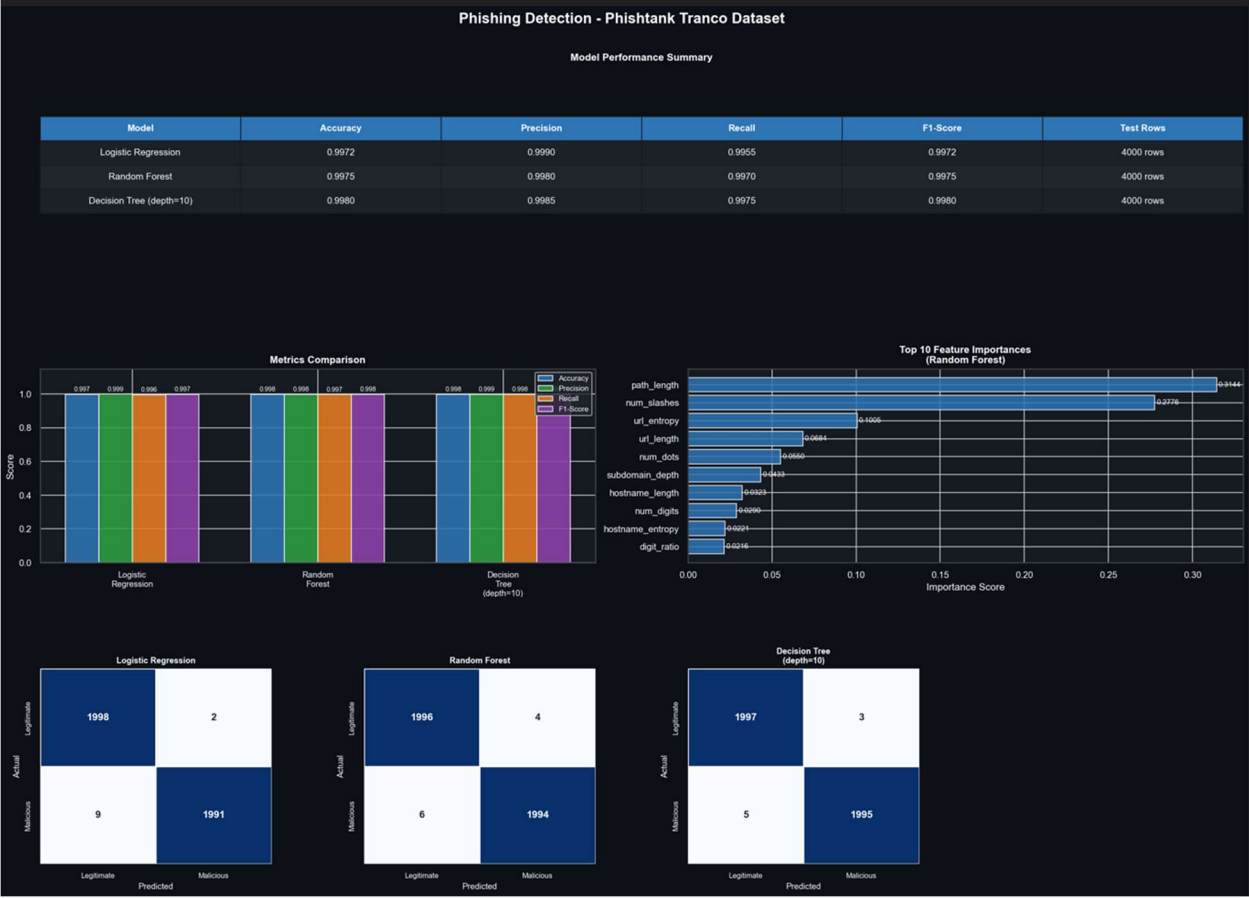


Figure D.15: Dashboard PNG- Within-Dataset Evaluation (PhishTank + Tranco, Blue Theme, dashboard_features_phishtank_tranco_dataset.png)

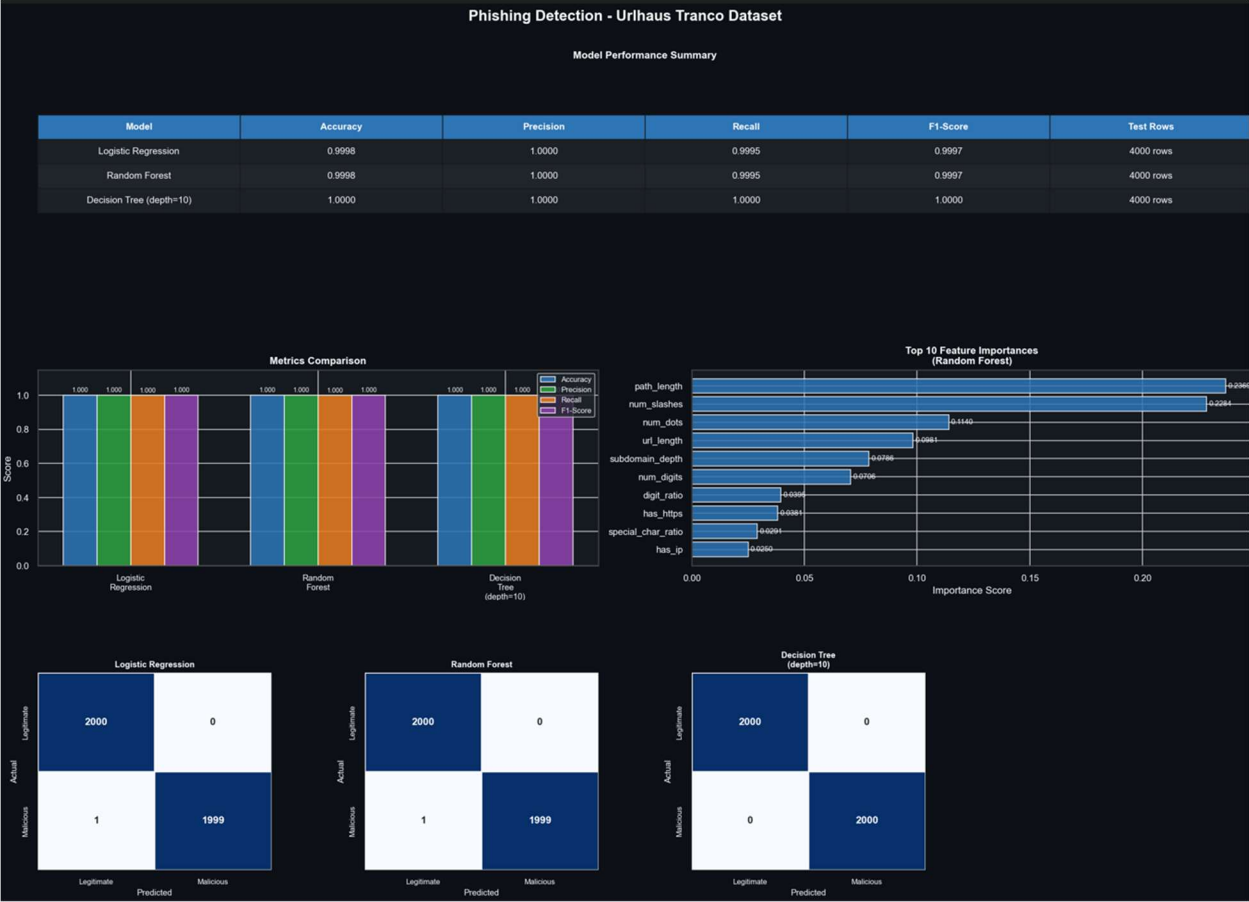


Figure D.16: Dashboard PNG- Within-Dataset Evaluation (URLhaus + Tranco, Blue Theme, dashboard_features_urlhaus_tranco_dataset.png)

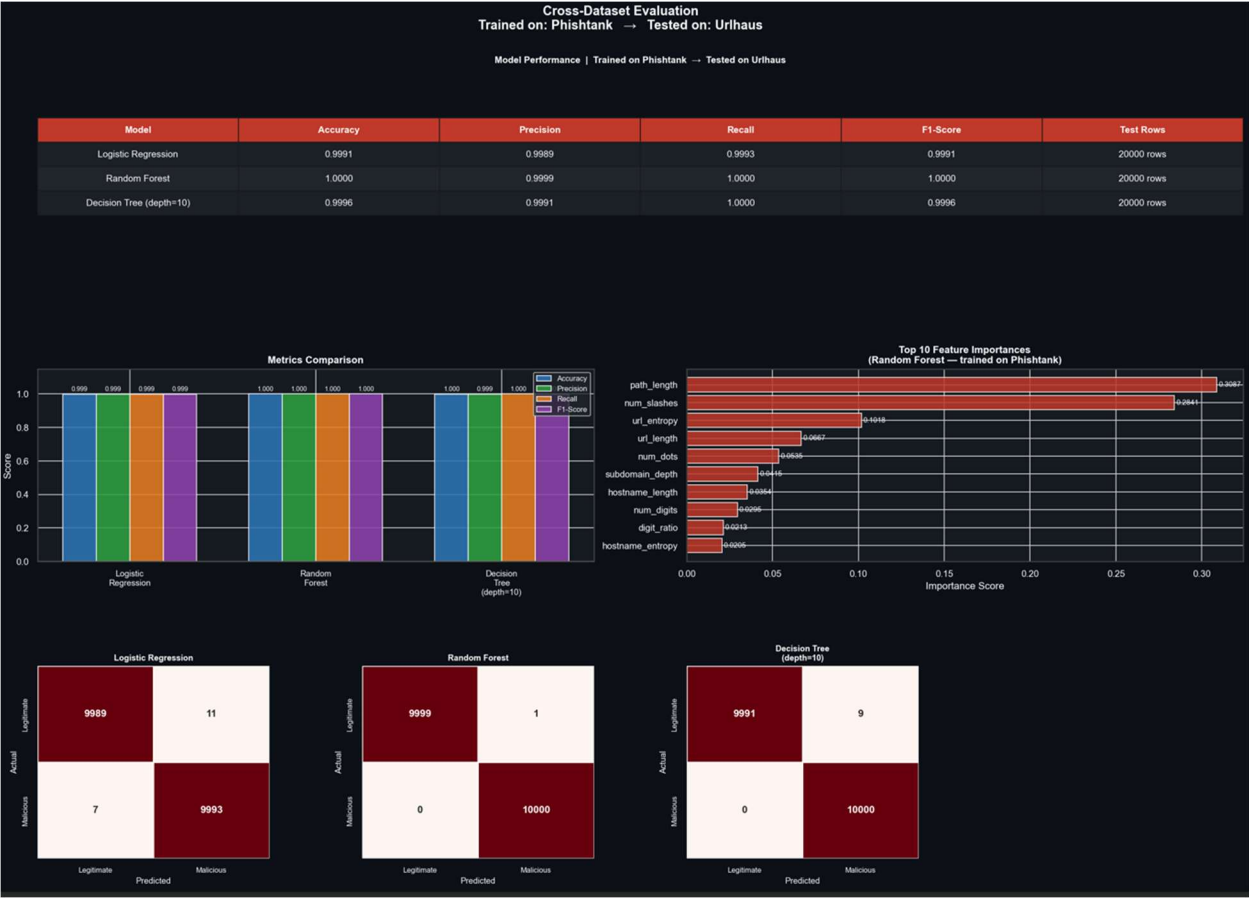


Figure D.18: Dashboard PNG- Cross-Dataset Evaluation (PhishTank to URLhaus, Red Theme, dashboard_cross_eval_PhishTank_to_URLhaus.png)



Figure D.19: Dashboard PNG- Cross-Dataset Evaluation (URLhaus to PhishTank, Red Theme, dashboard_cross_eval_URLhaus_to_PhishTank.png)

```
(venv) PS C:\Users\Connie's Build\phishing-detection\PhishingDetection\phishingdetection> python src/predict.py
--url https://www.google.com
19:01:46 [INFO] =====
19:01:46 [INFO] predict.py -- URL inference only
19:01:46 [INFO] =====
19:01:46 [INFO] Loading model artifacts from outputs/
19:01:50 [INFO] All model artifacts loaded successfully.
19:01:50 [INFO] Extracting 24 URL-string features...
19:01:50 [INFO] Running classifiers...

=====
PHISHING DETECTION - URL CLASSIFICATION RESULT
=====
URL: https://www.google.com
=====
Logistic Regression      Legitimate (99.9% confidence)
Random Forest           Legitimate (99.5% confidence)
Decision Tree (depth=10) Legitimate (100.0% confidence)
=====
Majority Vote (2 of 3):  Legitimate
=====
```

Figure D.20: Inference CLI Output- Legitimate Classification (https://www.google.com)


```
(venv) PS C:\Users\Connie's Build\phishing-detection\PhishingDetection\phishingdetection> python src/predict.py
--url http://free-5507007.webadorsite.com
19:06:05 [INFO] =====
19:06:05 [INFO] predict.py -- URL inference only
19:06:05 [INFO] =====
19:06:05 [INFO] Loading model artifacts from outputs/
19:06:06 [INFO] All model artifacts loaded successfully.
19:06:06 [INFO] Extracting 24 URL-string features...
19:06:06 [INFO] Running classifiers...

=====
PHISHING DETECTION - URL CLASSIFICATION RESULT
=====
URL: http://free-5507007.webadorsite.com
-----
Logistic Regression      Malicious      (79.7% confidence)
Random Forest            Legitimate     (71.5% confidence)
Decision Tree (depth=10) Malicious      (100.0% confidence)
-----
Majority Vote (2 of 3):  Malicious
=====

NOTE: Classifiers disagree on this URL. Treat the majority verdict with caution and
consider manual review before acting on this result.
```

Figure D.21: Inference CLI Output- Malicious URL Classification (Phishing Pattern URL)

Phishing Detection System — URL Classifier

Phishing Detection System

URL Classification using Machine Learning — 24 URL-String Features

Enter URL to classify:
No network connection is made — the URL is analysed as a string only.

Classify Clear

Classification Results

Enter a URL above and click Classify

Ready — enter a URL to classify. ✓ Models loaded

Figure D.22: GUI Prototype- URL Classification Interface (empty state)

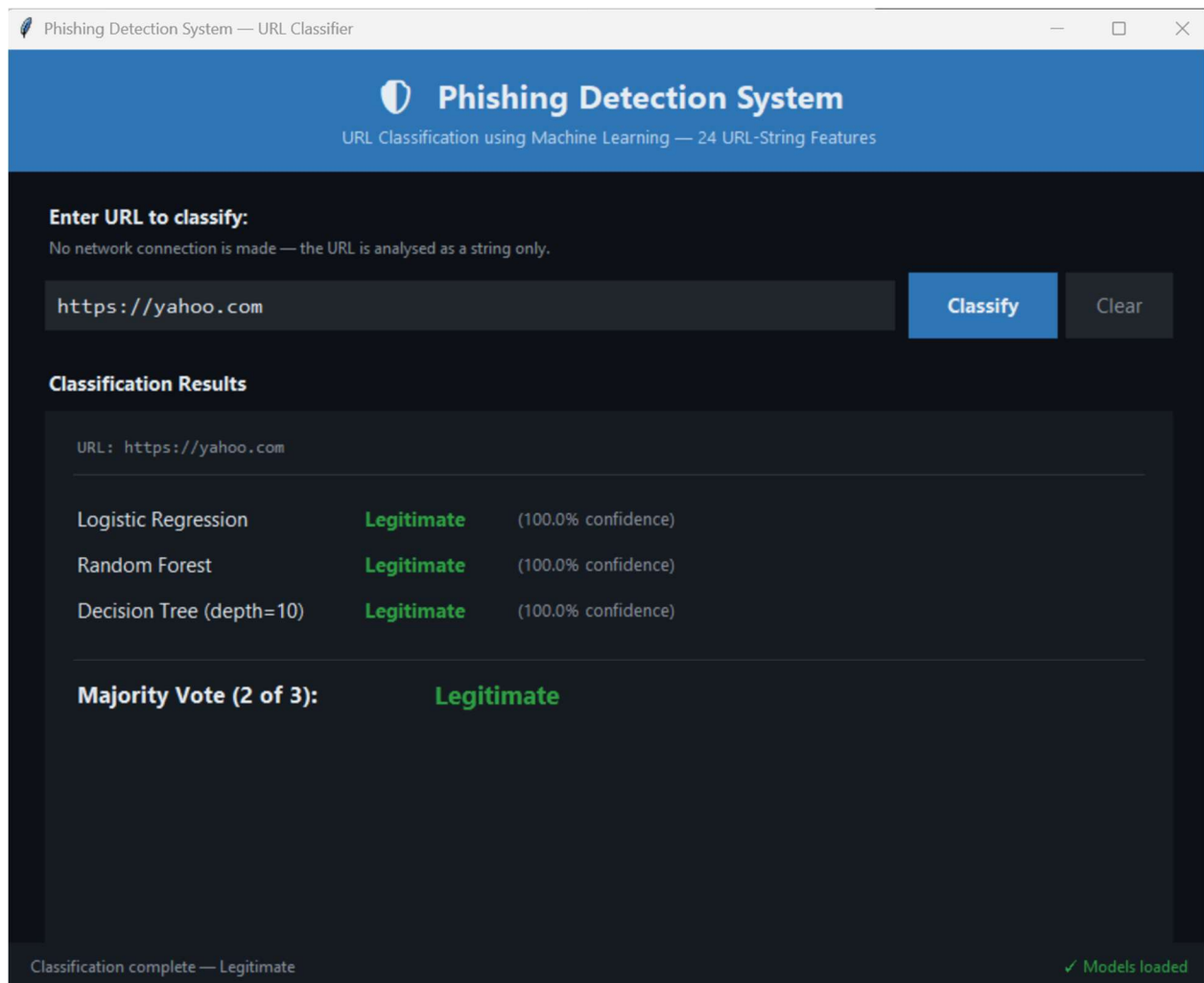


Figure D.23: GUI Prototype- Legitimate URL Classification Result

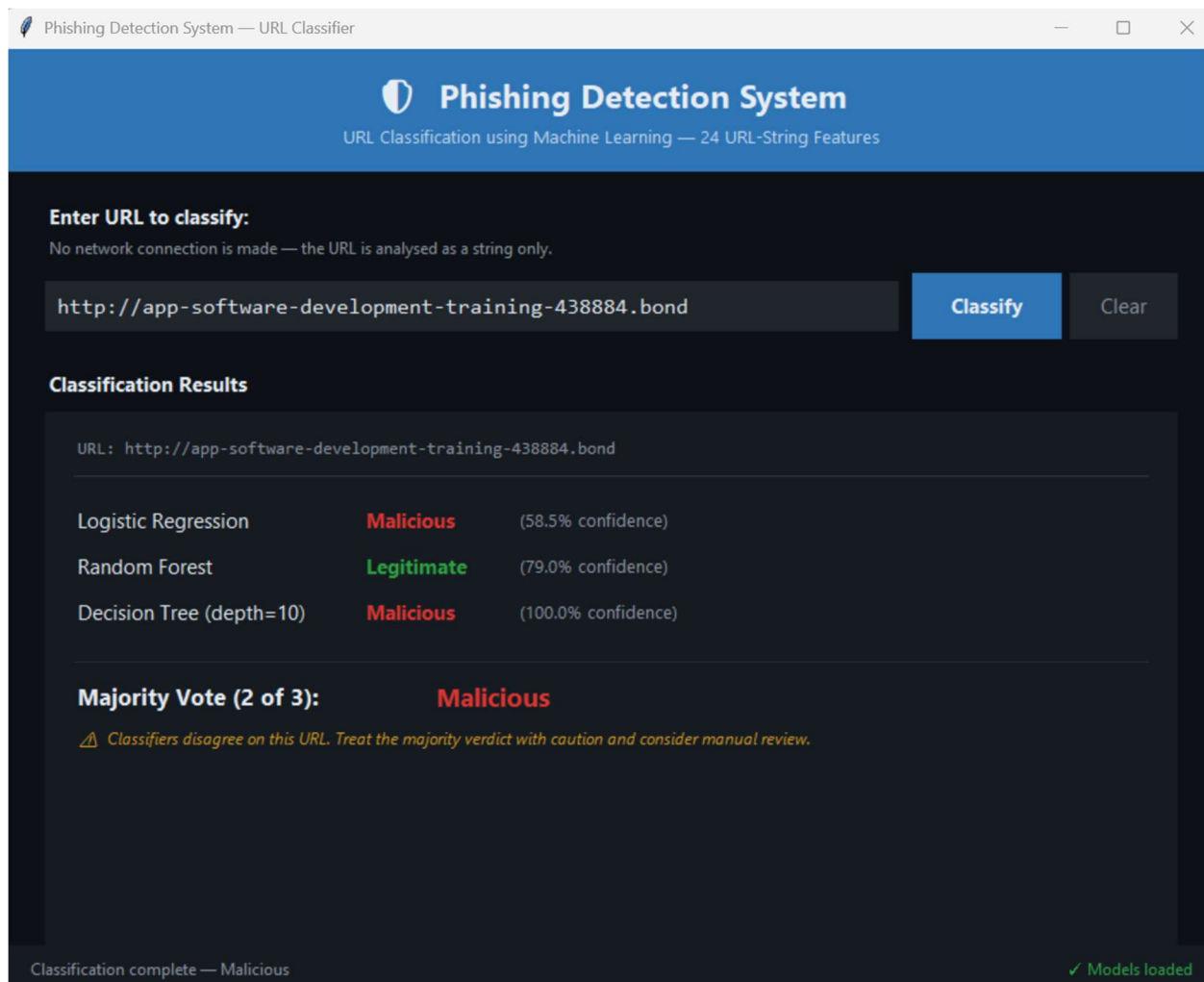


Figure D.24: GUI Prototype- Malicious URL Classification Result

Appendix E: Definitions, Acronyms, and Abbreviations

This appendix defines key terms, acronyms, and abbreviations used throughout this document. These definitions are provided to ensure clarity, consistency, and a shared understanding of terminology across all audiences.

E.1 Acronyms and Abbreviations

Acronym	Definition
API	Application Programming Interface
CLI	Command-Line Interface
CSV	Comma-Separated Values
DNS	Domain Name System
F1 Score	Harmonic mean of precision and recall
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ML	Machine Learning
SRS	Software Requirements Specification
URL	Uniform Resource Locator
WHOIS	Internet domain registration lookup protocol

E.2 Definitions

Term	Definition
Balanced Dataset	A dataset constructed to contain an equal number of malicious and legitimate samples.
Classification	The process of assigning a label (malicious or legitimate) to a given URL based on model output.
Confidence Score	A numerical value representing the probability or certainty of a model's prediction.
Cross-Dataset Evaluation	Evaluation method in which a model is trained on one dataset and tested on a different dataset to assess generalization.
Dataset	A structured collection of URL records used for training or evaluation.
Feature	A measurable property or characteristic extracted from a URL string and used as input to a model.
Feature Engineering	The process of transforming raw URL data into a structured feature set suitable for machine learning.
Feature Record	A structured representation of a single URL after feature extraction, consisting of 24 features and a corresponding label.

Feature Vector	An ordered set of numerical feature values representing a URL instance.
Inference	The process of applying trained models to new input data to generated predictions.
Legitimate URL	A URL that is classified as safe or non-malicious.
Malicious URL	A URL associated with phishing, malware, or other harmful activity.
Majority Voting	An ensemble decision method in which the final classification is determined by the majority of model predictions.
Model Artifact	A serialized representation of a trained machine learning model (e.g., .pkl file).
Model Evaluation	The process of assessing model performance using metrics such as accuracy, precision, recall, and F1.
Offline Pipeline	A system design in which all processing occurs locally without network calls or dependencies.
Pipeline	A sequence of processing stages in which data flows from ingestion through transformation, modeling, and output.
Prediction	The output generated by a model indicating whether a URL is malicious or legitimate.
Raw URL	The original, unprocessed URL string obtained from dataset.
Stratified Split	A dataset partitioning method that preserves class distribution across training and testing sets.
Training	The process of fitting a machine learning model to labeled data.
URL Entropy	A statistical measure of randomness within a URL string, often used to detect obfuscation.
Validation	The process of ensuring that input data meets required format schema constraints.

Appendix F: References

- Cisco Talos Intelligence Group (2024). *PhishTank out of the Net into the Tank*.
https://phishtank.org/developer_info.php
- Abuse.ch (2024). *URLhaus Malware URL exchange*. Bern University of Applied Sciences. <https://urlhaus.abuse.ch/>
- Le Pochat, V., Van Goethem, T., Tajalizadehkhood, S., Korczynski, M. & Joosen, W. (2023). *Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation*. Tranco. <https://tranco-list.eu/>
- IBM (n.d.). *What is ETL (extract, transform, load)?* IBM.
<https://www.ibm.com/think/topics/etl>
- Wang, J., KUO, T., Li, L., & Zeller, A. (2020). Assessing and Restoring Reproducibility of Jupyter Notebooks. *2020 35th IEEE/ACM International Conference on Automated Software Engineering*, 138-149.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9286024>
- Amazon Web Services (2026). *Monolithic vs microservices: Difference between software development architecture*. AWS.
<https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/>
- Microsoft Build 2026 (2026a). *Event-driven architecture style*. Microsoft.
<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>
- Microsoft Build 2026 (2026b). *Architectural principles*. Microsoft.
<https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/architectural-principles>
- Huyen, C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media.
- Wiegers, K. & Beatty, J. (n.d.). *Guidance for Data Dictionaries*. University of Kansas.
<https://view.officeapps.live.com/op/view.aspx?src=https%3A%2F%2Fpeople.eecs.ku.edu%2F~saiedian%2F812%2FProject%2FWiegers->

[Resources%2FChapter%252013%2FGuidance%2520for%2520Data%2520Dictionaries.docx&wdOrigin=BROWSELINK](#)

Dhingra, P. (2026). *CTAL-TA Quality Characteristics: ISO 25010, Usability, Reliability, and Maintainability Testing*. Master Software Testing.

<https://mastersoftwaretesting.com/certification-guides/istqb/ctal-ta/ctal-ta-quality-characteristics#iso-25010-quality-model>

ISO (n.d.) ISO 8601 Date and time format. ISO. <https://www.iso.org/iso-8601-date-and-time-format.html>

Microsoft (2026c). *Open-source libraries and Python in Excel*. Microsoft.

<https://support.microsoft.com/en-us/office/open-source-libraries-and-python-in-excel-c817c897-41db-40a1-b9f3-d5ffe6d1bf3e?>

Purdue University (n.d.) *IEEE General Format*. Purdue University.

https://owl.purdue.edu/owl/research_and_citation/ieee_style/ieee_general_format.html

Oracle (2026). *Licensing Information User Manual for Oracle Fusion Data Intelligence*. Oracle Communications.

https://docs.oracle.com/en/cloud/saas/analytics/26r1/fawlg/AQR_Capital_Management_LLC_Lambda_Foundry_Inc_and_PyData_Development_Team_Pandas.html

Matplotlib (2026). *License*. Matplotlib.

<https://matplotlib.org/stable/project/license.html>

Appendix G: AI Use

	Tool/Version	Query/Prompt	Output Summary	How It Was Used
1	OpenAI ChatGPT-4 (Nov 5, 2023)	“Can you please proofread the following proofread the following section for grammar, punctuation, and sentence clarity at a master’s level academic standard” (section 1)	Suggested corrections to title wording, punctuation, grammar, typos, and citation consistency.	Reviewed all suggestions independently. Accepted only minor grammatical corrections that did not alter my original works meaning. Corrected typos.
2	OpenAI ChatGPT (GPT-4, 2023)	“Please proofread the following section for grammar, punctuation, and sentence clarity at a master’s level academic standard, please format your response in bullet points thank you” (section 2)	Suggested corrections for grammar, data source, typos, and phrasing.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original works meaning. Corrected typos.
3	OpenAI ChatGPT	“Please proofread the following section for	Suggested reordering contributions to lead	Evaluated the suggested order

	(GPT-4, 2023)	grammar, punctuation, and sentence clarity at a master's level academic standard, please format your response in bullet points thank you" (section 3)	with the most novel methodological decision before listing supporting contributions.	against my own judgement and made selective adjustments, as not all adjustments seemed appropriate of logical.
4	OpenAI ChatGPT (GPT-4, 2023)	"Please proofread the following section for grammar, punctuation, and sentence clarity at a master's level academic standard, please format your response in bullet points thank you" (section 4)	Suggested corrections for grammar, data source, typos, and phrasing.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original works meaning. Corrected typos.
5	OpenAI ChatGPT (GPT-4, 2023)	"Please proofread the following section for grammar, punctuation, and sentence clarity at a master's level academic standard, please format your response in bullet points thank you" (section 5)	Suggested corrections for grammar, data source, typos, and phrasing.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original

				works meaning. Corrected typos.
6	OpenAI ChatGPT (GPT-4, 2023)	“Please proofread the following section for grammar, punctuation, and sentence clarity at a master’s level academic standard, please format your response in bullet points thank you” (section 6)	Suggested corrections for grammar, data source, typos, and phrasing.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original works meaning. Corrected typos.
7	OpenAI ChatGPT (GPT-4, 2023)	“Please proofread the following section for grammar, punctuation, and sentence clarity at a master’s level academic standard, please format your response in bullet points thank you” (section 7)	Suggested corrections for grammar, data source, typos, and phrasing.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original works meaning. Corrected typos.
8	OpenAI ChatGPT	Are the following APA references formatted correctly?	Recommended large number of things that were not necessary.	Changed authors name to match.

	(GPT-4, 2023)		Suggested correction to one author's name.	
9	OpenAI ChatGPT (GPT-4, 2023)	"Please proofread the following executive statement for grammar, punctuation, and sentence clarity at a master's level academic standard, please format your response in bullet points thank you"	Suggested corrections for capitalization, grammatical fixes, typos, and repetitive phrasing.	Reviewed all suggestions independently. Accepted minor grammatical corrections and phrasing that did not change my original works meaning. Corrected typos.